구조 설계 및 평가

조 용 진

(drajin.cho@bosornd.com)

주식회사 보이는 소프트웨어 연구소
http://www.bosornd.com

CC BY NC ND

강사 소개

성명	조용진(CHO YONG JIN) – 소프트웨어 플랫폼 전문가/아키텍트
연락처	drajin.cho@bosornd.com (+82-10-3754-2879)
학력	1994년 2월 한국과학기술대학 전산학과 졸업 1996년 2월 한국과학기술원 전산학과 석사 졸업 1998년 2월 한국과학기술원 전산학과 박사 수료
경력	2003년 ~ 2016년 2월, (주)삼성전자 근무 스마트 홈 & 상황인지 서비스 프레임워크 개발(2003년~2004년) 디지털 TV 소프트웨어 플랫폼 개발 및 상품화(2005년~2009년) 인도 델리 연구소 주재(2010년~2014년) 사물인터넷 기기를 위한 Tizen 플랫폼 개발(2015년) 2016년 11월 22일 주식회사 보이는소프트웨어연구소 창업 삼성전자 소프트웨어 구조설계자 양성 및 인증(2016년~현재)



목차 – 구조 설계 및 평가(핵심 개념)

캐디 로봇 과제

1) 구조 명세 – 구조도

2) 구조 평가 – 품질 요구사항, 구조적 의사결정

3) 재설계 – ADR (Architecture Decision Records)

4) 구조 설계와 문서화

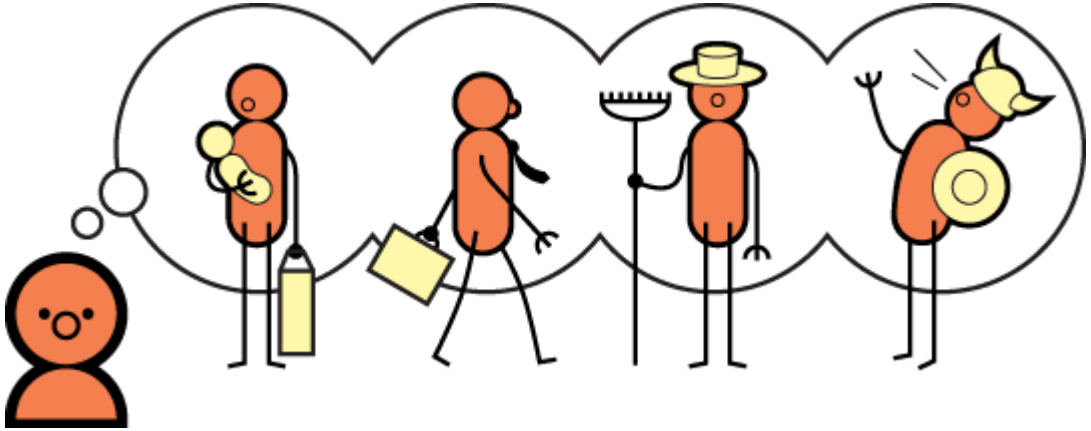
repository: https://github.com/bosornd/caddybot

C:₩> docker pull **tiryoh/ros2-desktop-vnc:humble**

Role of Architect

cc BY NC ND

Role of Architect?



Whoever knows SW cannot say it’s easy.

Architect makes SW easier.

Writing vs. Programming







Learning English	Learning programming language
Writing diary	Writing sample program
Writing a short story	Developing an Android app
Writing a novel	Developing an JavaScript engine
Writing an epic	Developing Web browser

Writing vs. Programming







Learning English

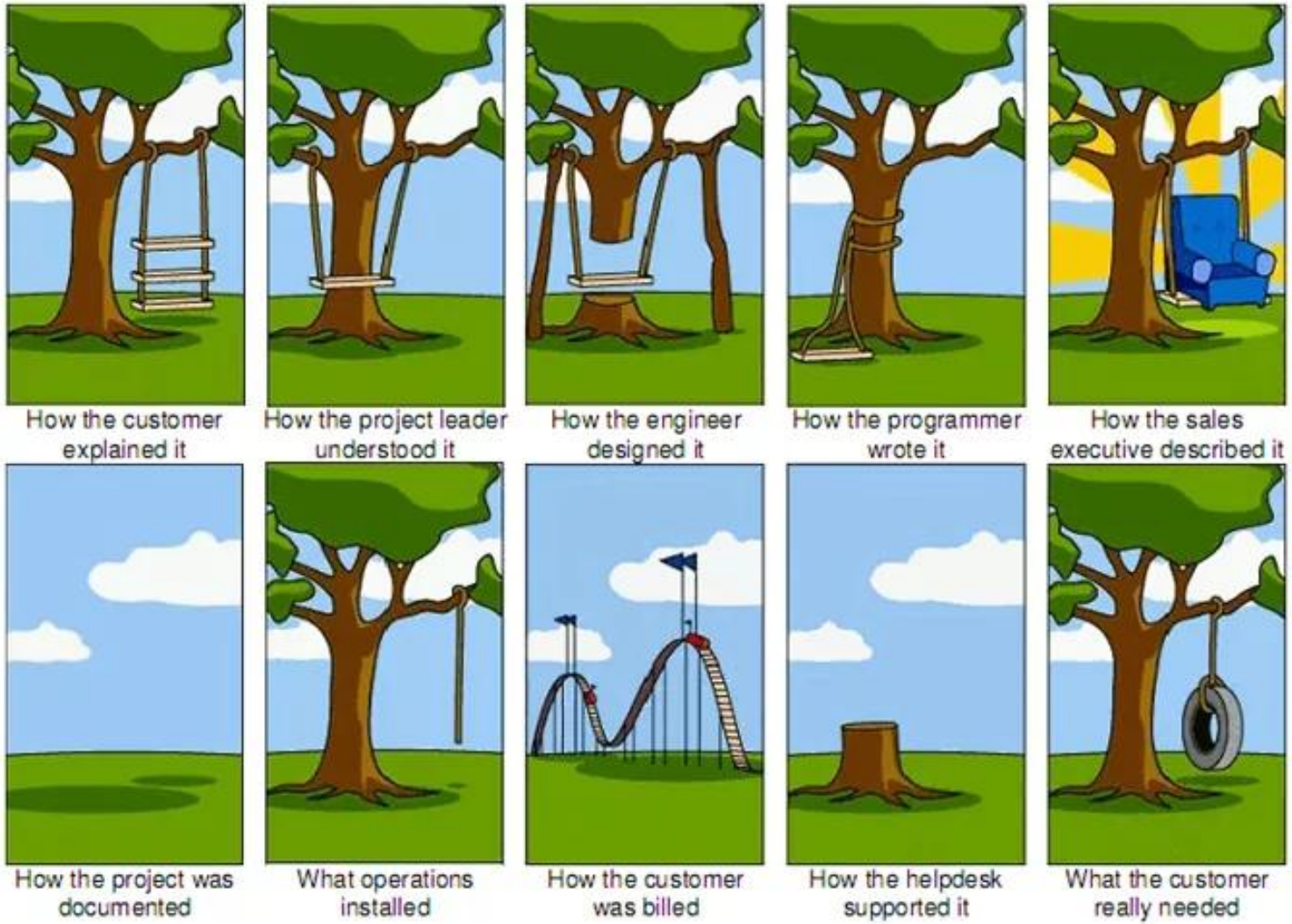
Writing a novel

Learning programming language

Developing Software

more difficult

Why SW is Difficult?



Why SW is Difficult?

Because of Requirements

- Ambiguous
- Incomplete / missing
- Changing

How about requirements of the building?

- Nothing ambiguous?
- Everything complete?
- No changes in the construction?



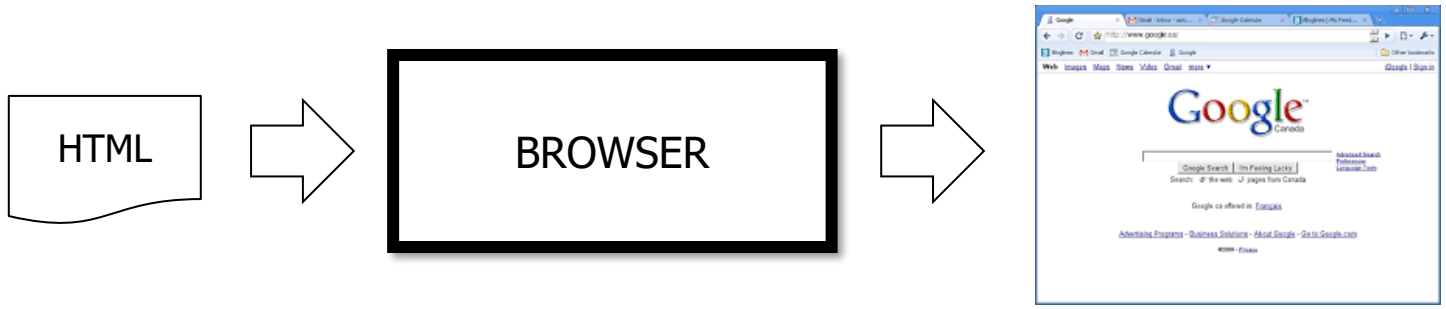
Why SW is Difficult?

If requirements are clear, …



Why SW is Difficult?

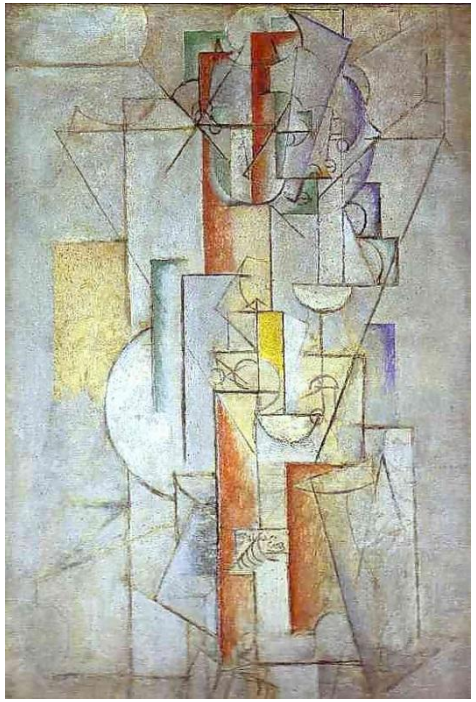
If requirements are clear, …



Something rather than requirements makes SW difficult.

What makes SW more Difficult?

Picasso Paintings



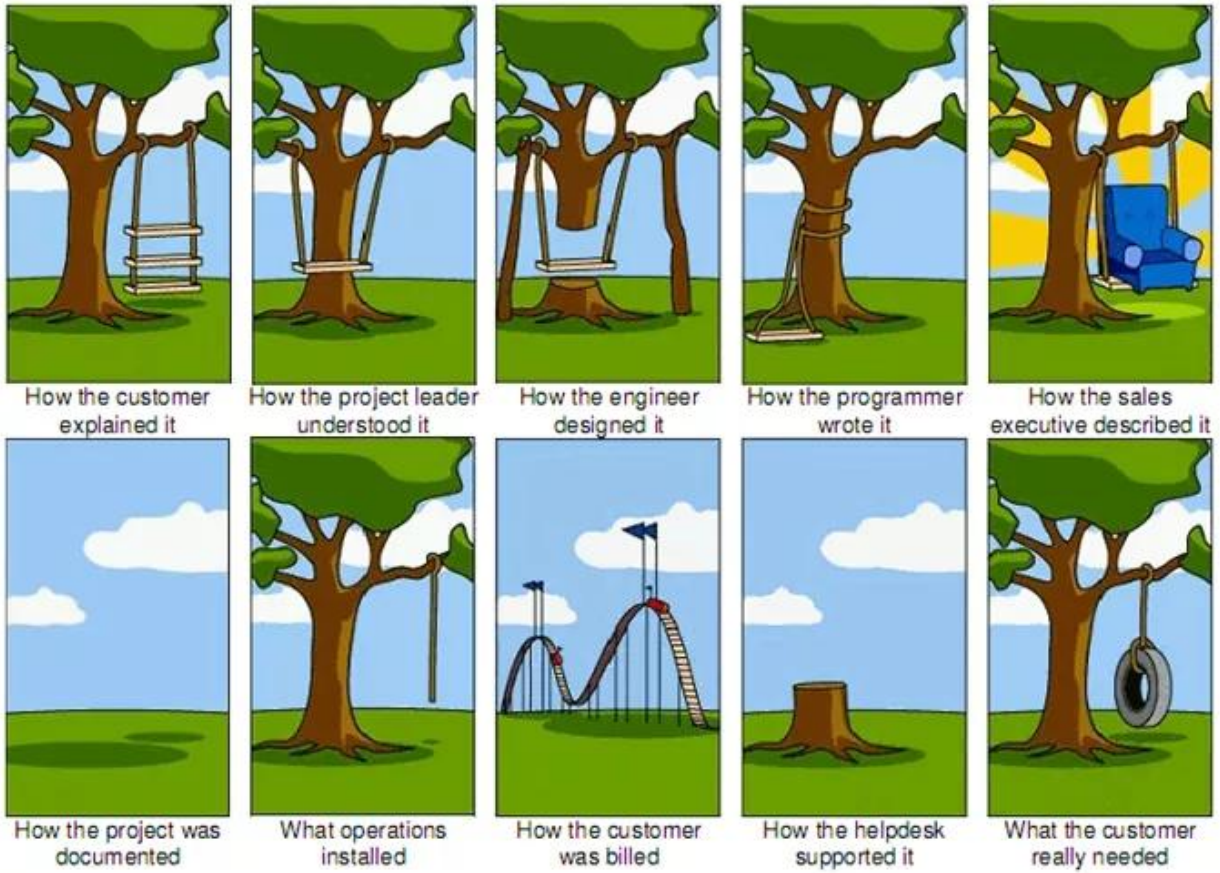
Nude, I love Eva



Woman Playing the Mandolin

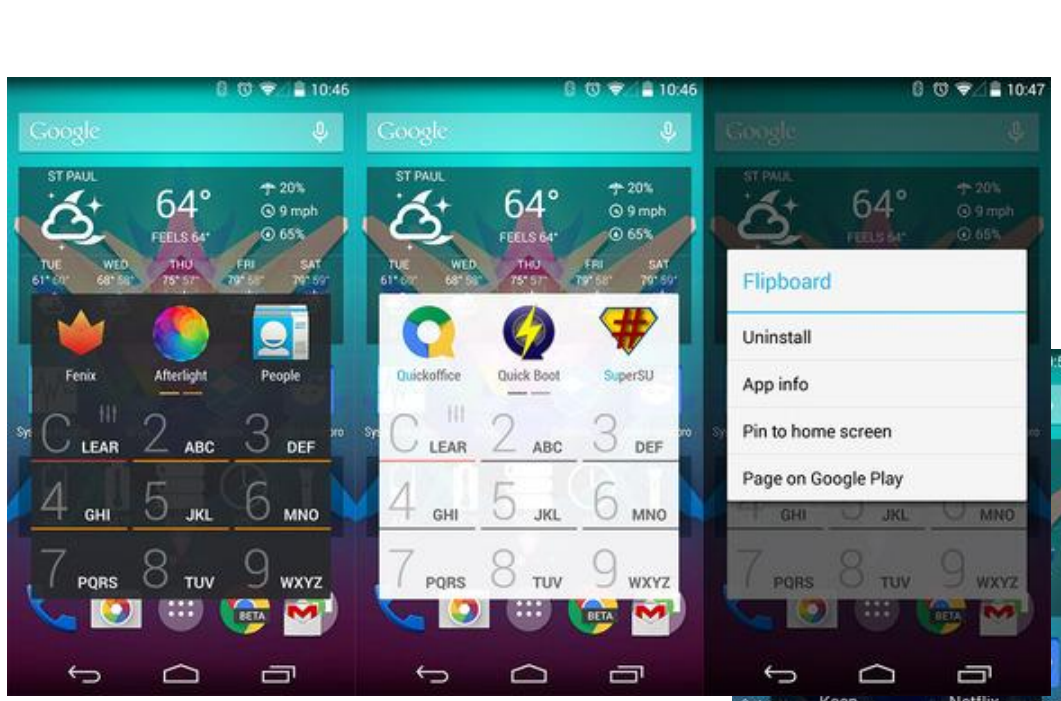
What makes SW more Difficult?

SW is [ ] in nature.

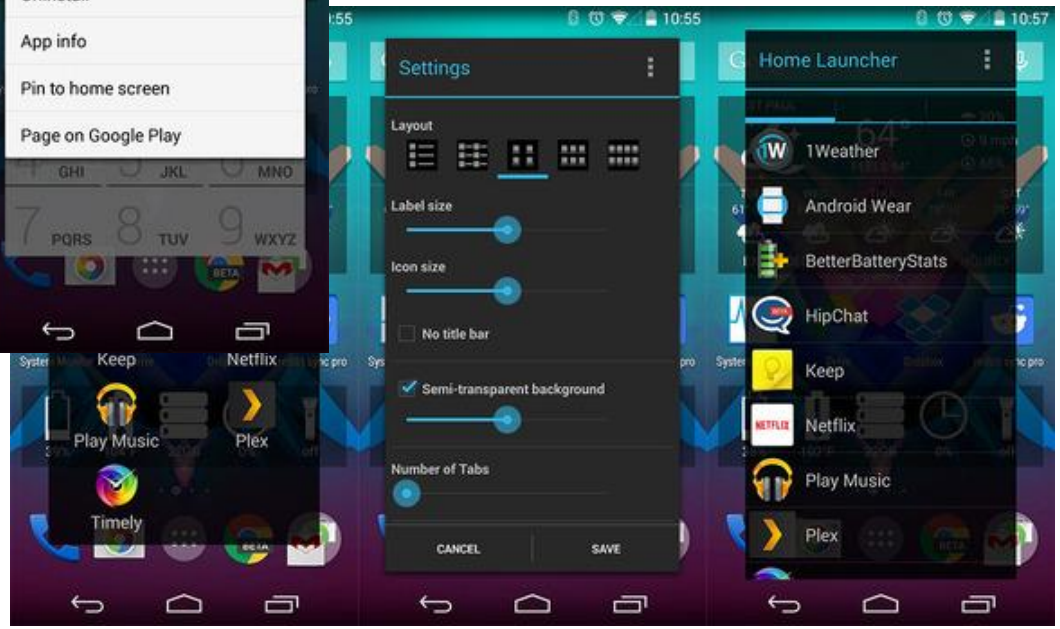


What makes SW more Difficult?

SW is invisible in nature.

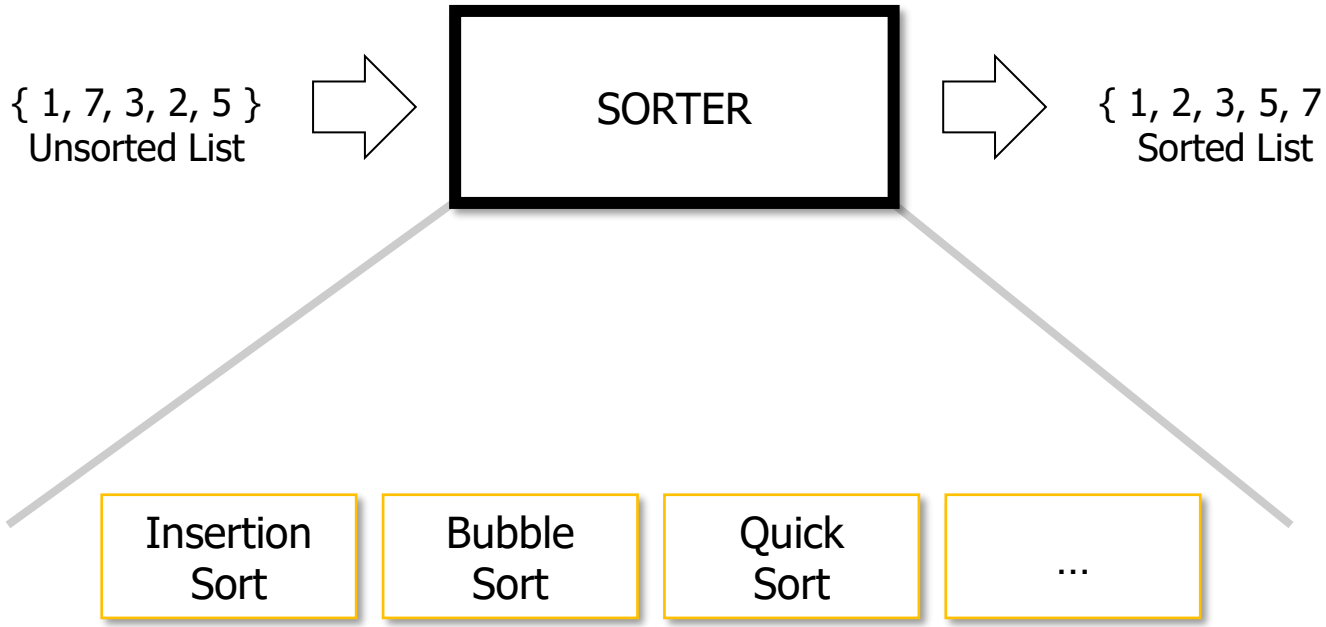






What makes SW more Difficult?

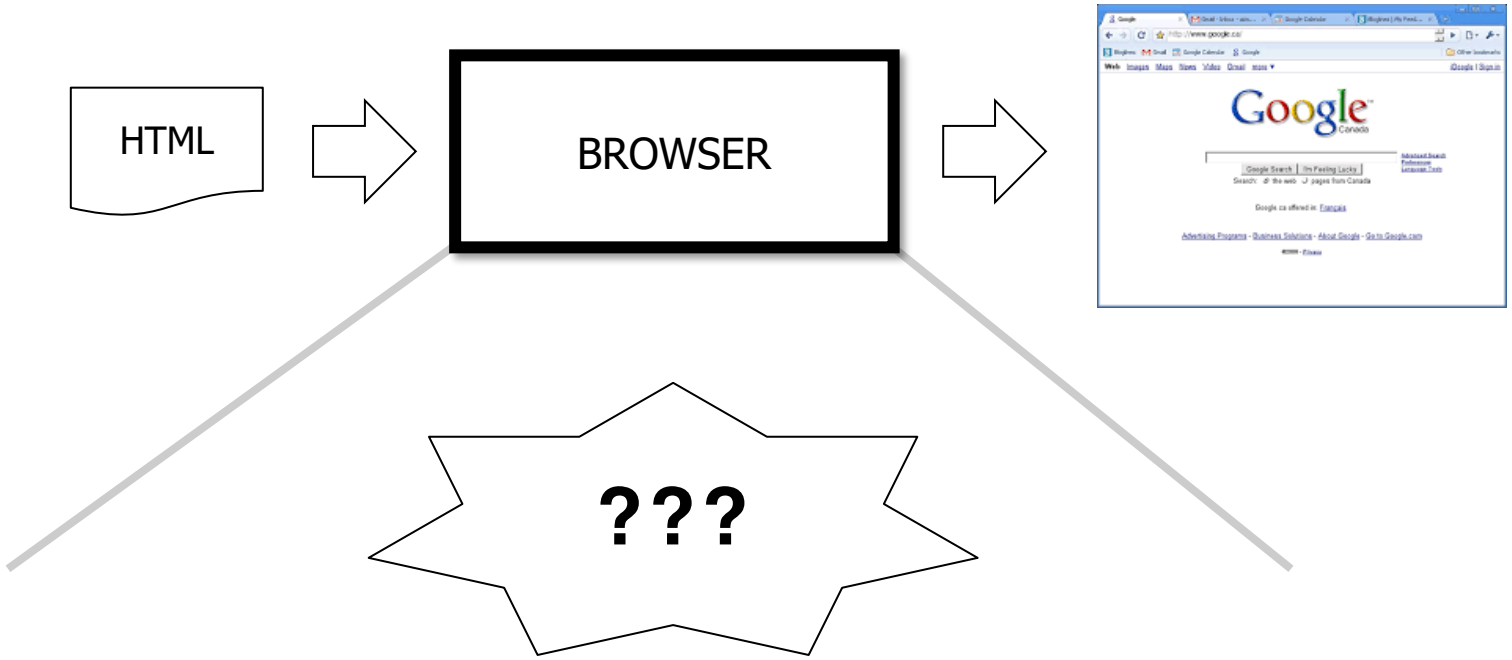
SW is invisible in nature.



What makes SW more Difficult?

SW is invisible in nature.

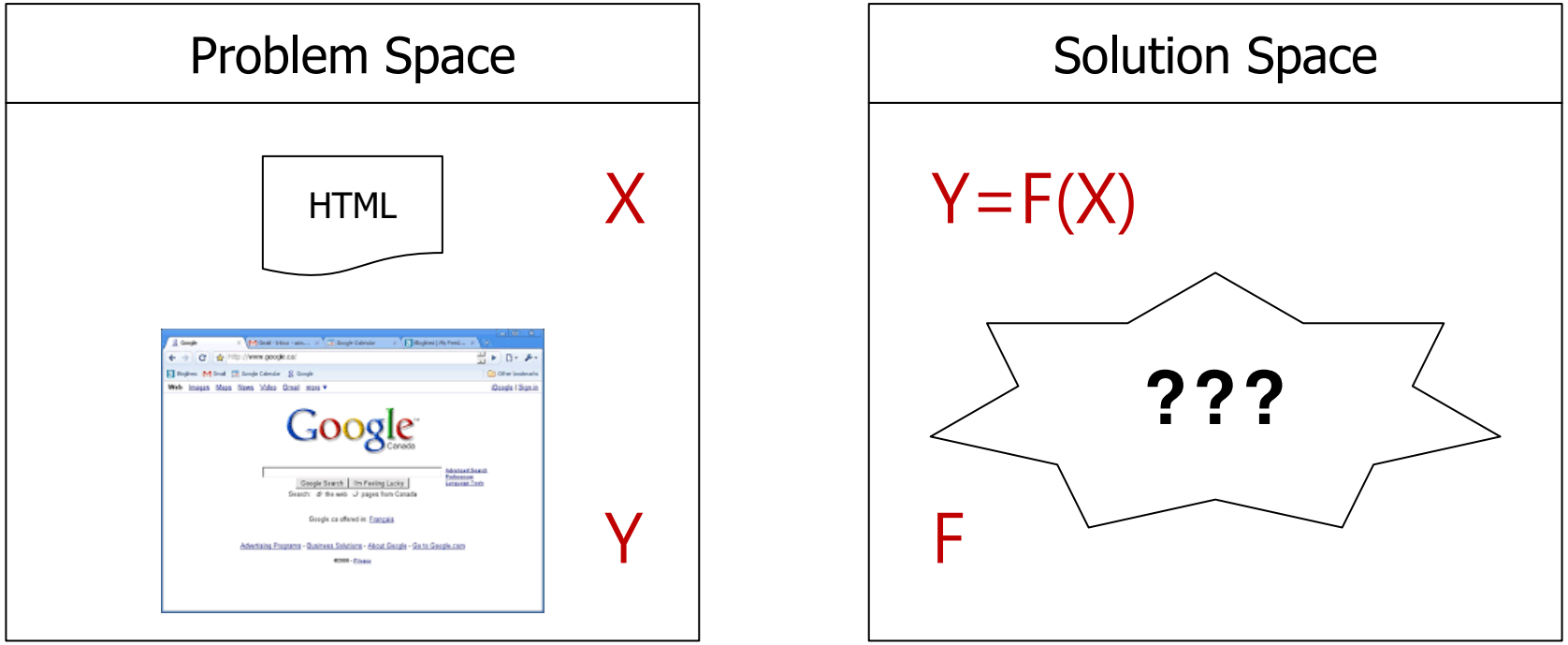
➔ Hard to imagine what it's composed of and how they work



What makes SW more Difficult?

SW is invisible in nature.

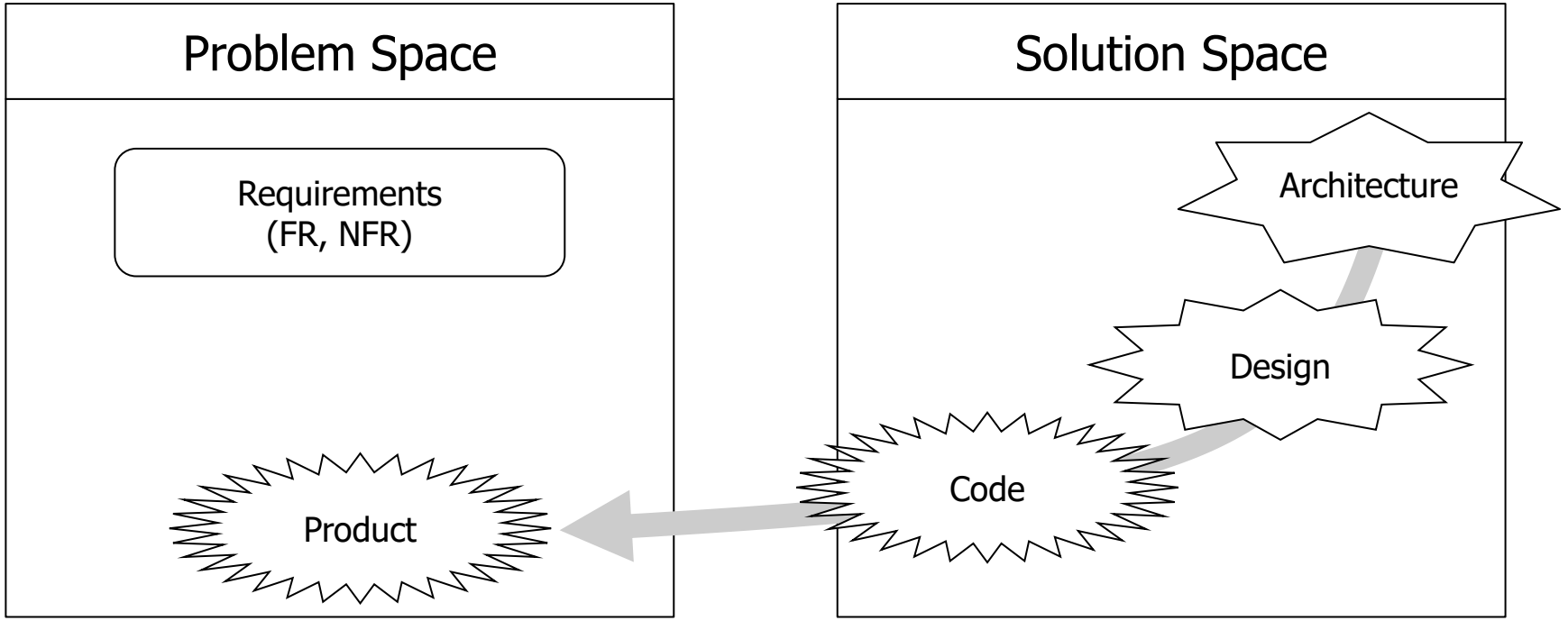
➔ Hard to imagine what it's composed of and how they work



What makes SW more Difficult?

SW is invisible in nature.

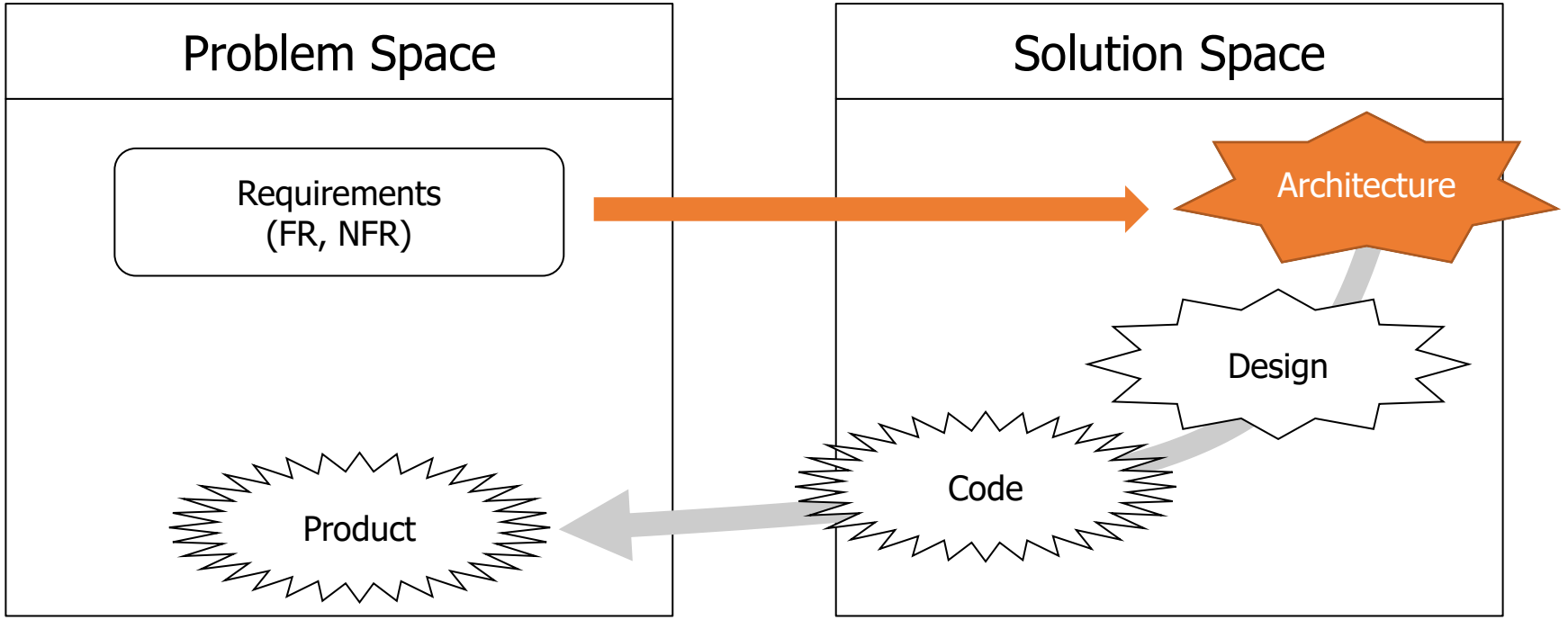
➔ Hard to imagine what it's composed of and how they work



Role of Architect

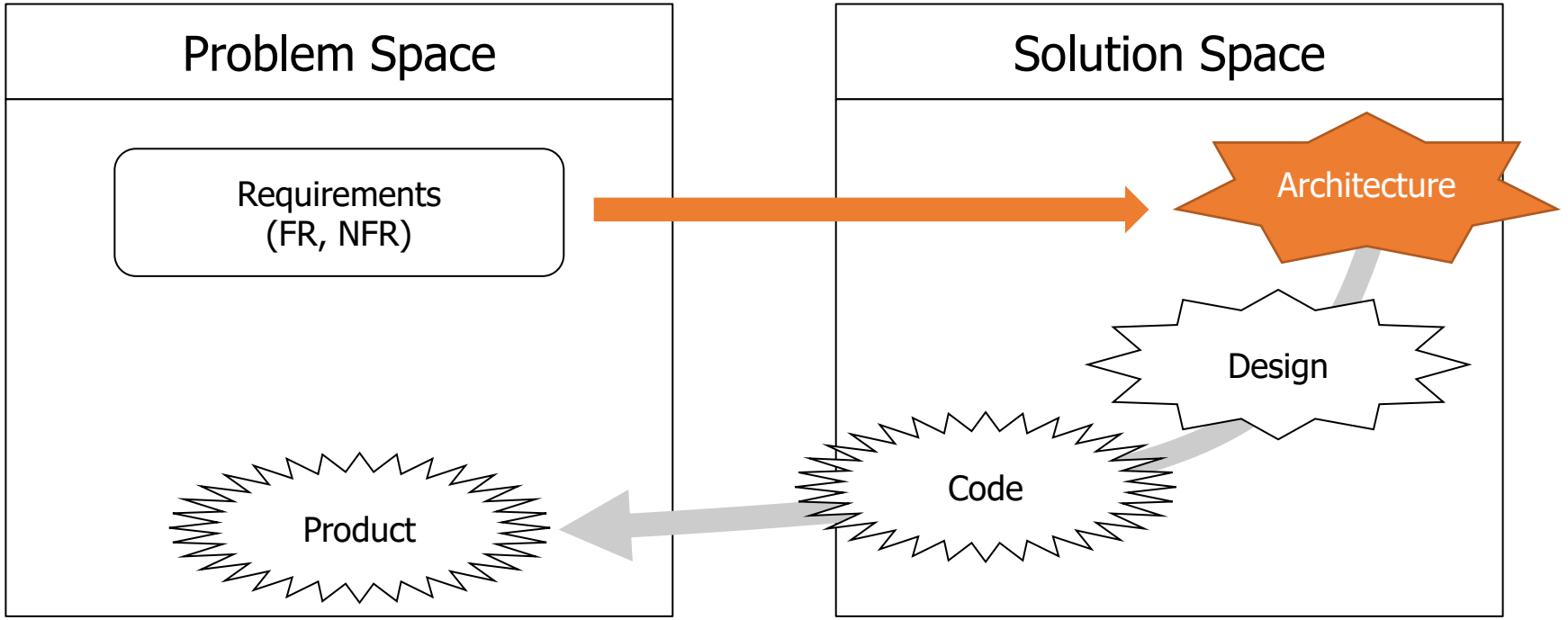
Architect shows what it's composed of and how they work (architecture)

➔ Make SW imaginable and easier to develop



Way of Architect

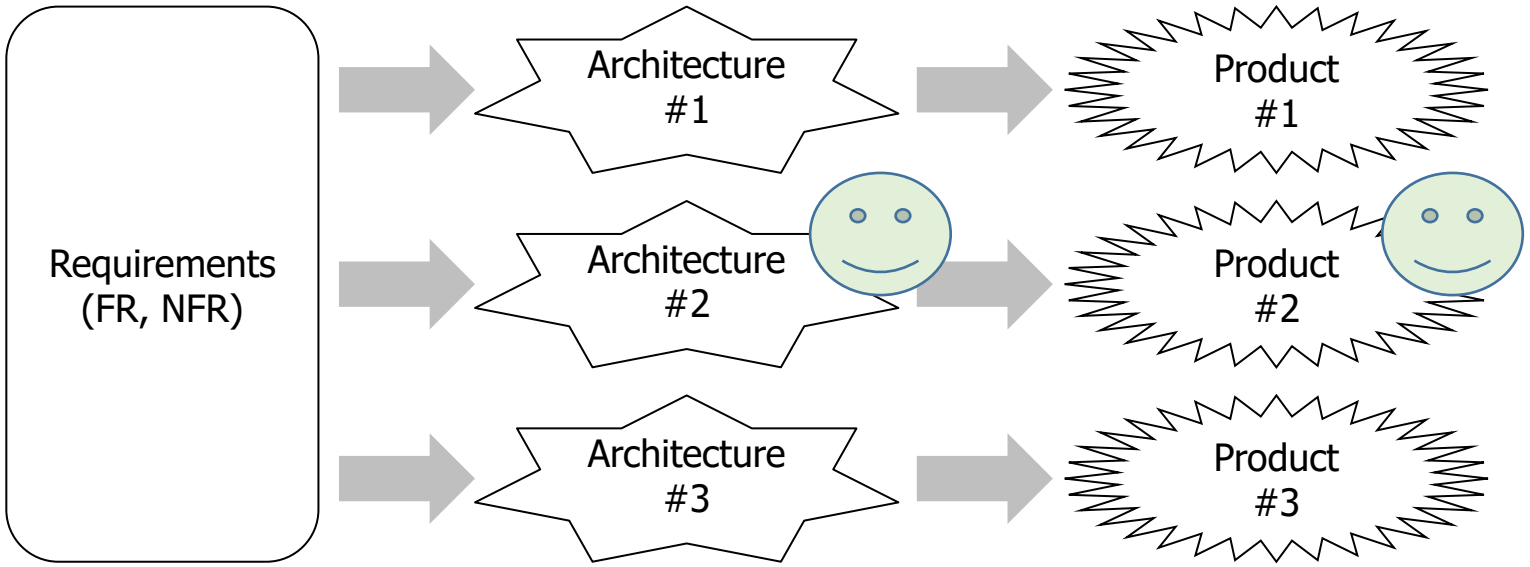
Architect tries to [ ] what it's composed of and how they work and find the best one for the project



Way of Architect

To find the best architecture,

architect evaluates architecture candidates by



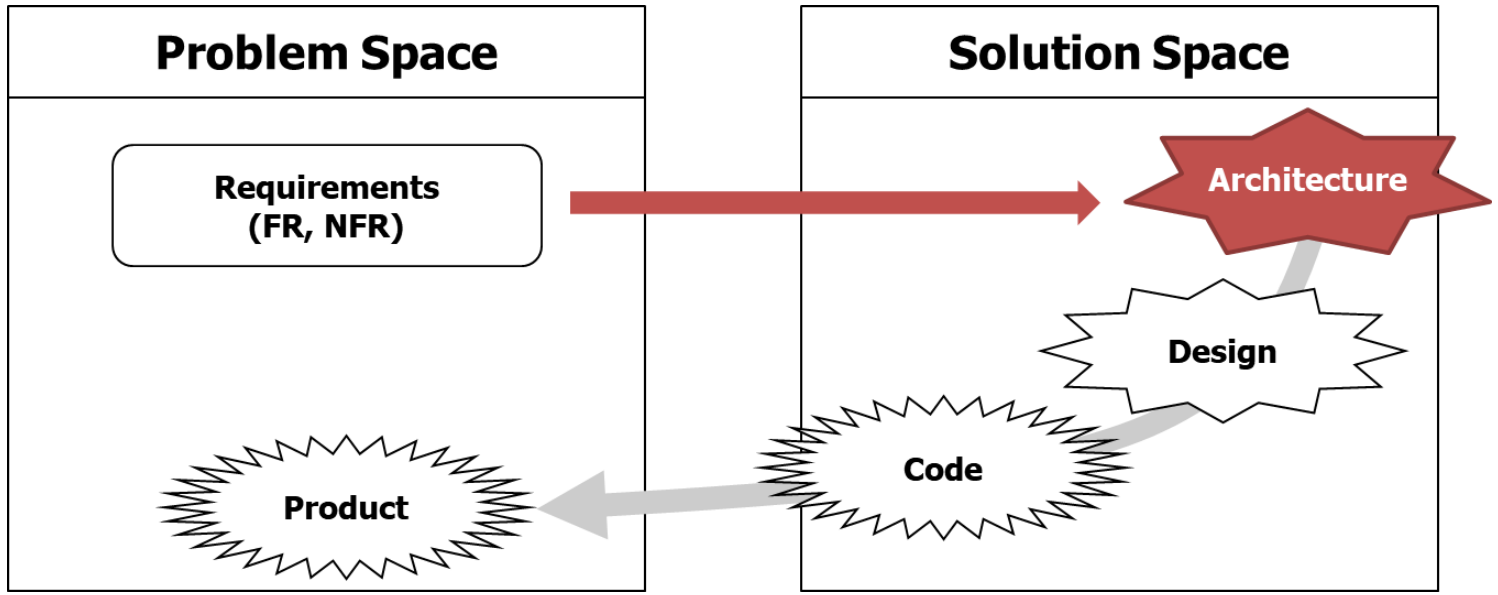
Why Architecture more concerned?

Architecture change is more expensive than design or code change.

How about requirement changes?

Cost depends on implementation change.

- architecture change
- design change
- code change



Summary

Architect shows

what the required SW system is composed of and how they work

← Architecture

evaluates architecture candidates and finds the best one for the project [business]

makes easier to construct and maintain the required SW system [development]

SW is More Difficult, Why?



VS



Learning English

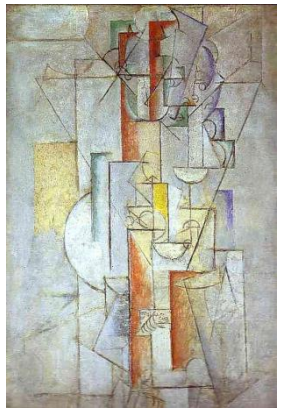
Learning programming language



Writing a novel

more difficult

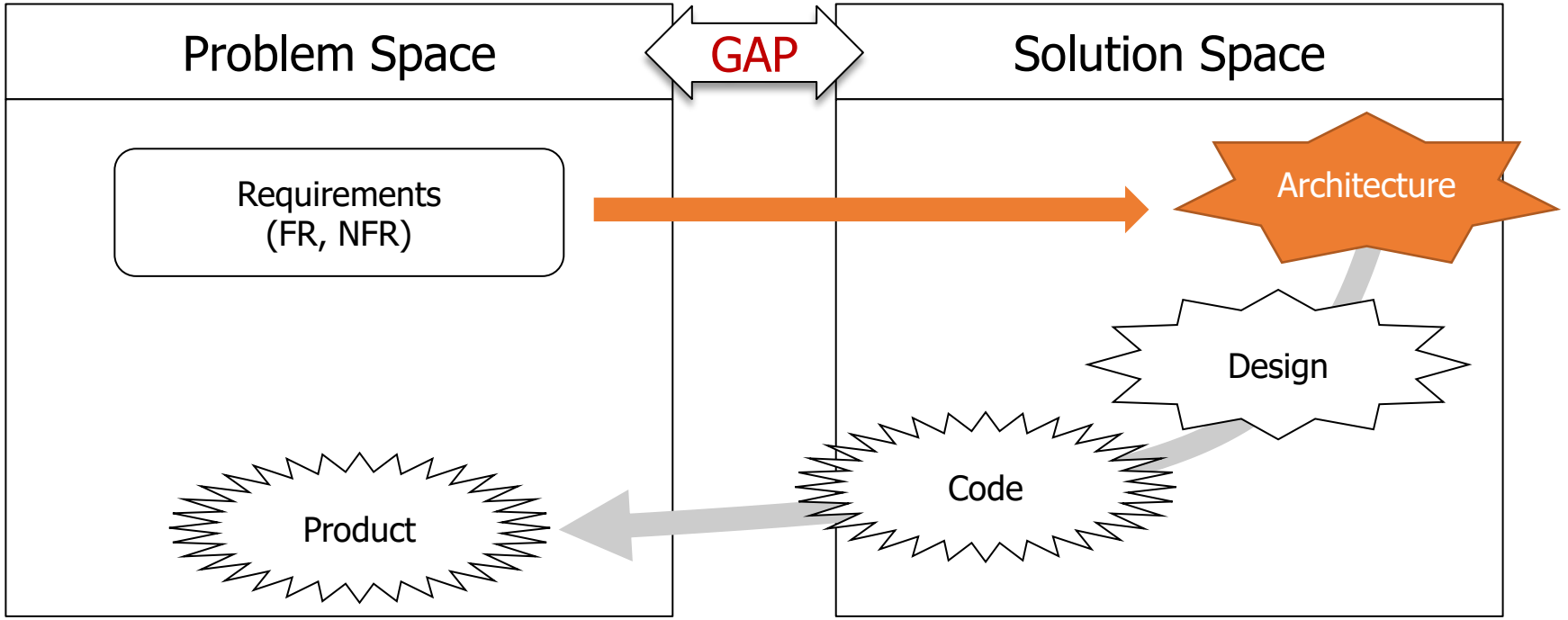
Developing software



More Difficult?

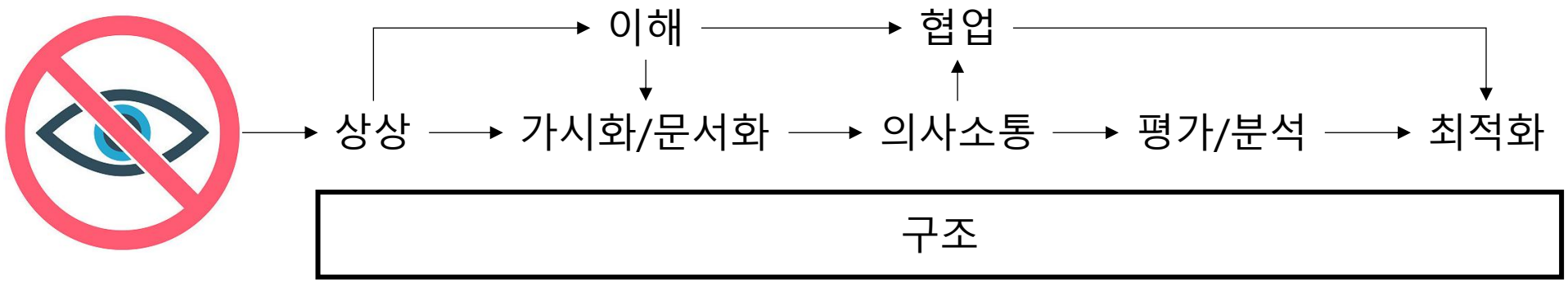
Difficulty ∝ GAP between problem space and solution space

Expert(Architect) is familiar with solution space



아키텍트의 역할

- 구조도를 그린다.



- 최적 설계/개발을 이끈다.

캐디 로봇



https://github.com/bosornd/caddybot

CC BY NC ND

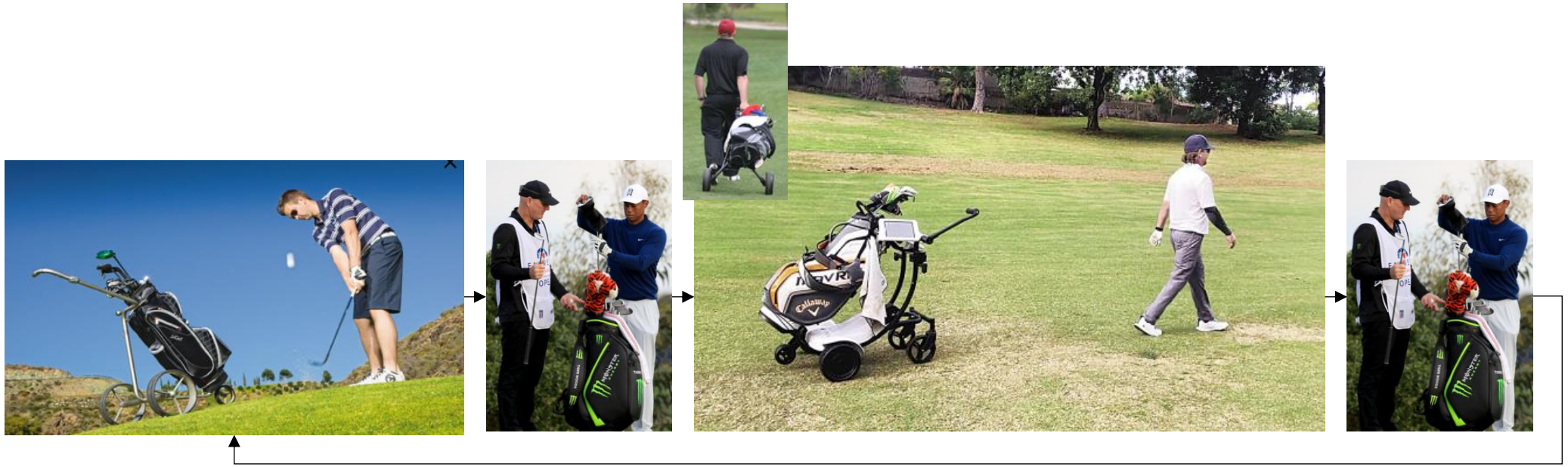
캐디 로봇





http://www.talktalkgolf.com/bbs/content.php?co_id=hello_caddy_ko

기본 기능



1. 사용자는 공을 친다. 캐디 로봇은 스윙에 방해되지 않을 만큼 떨어져 있어야 한다.
2. 사용자가 캐디 로봇에 다가간다. 사용자는 사용한 클럽을 가방에 넣는다.
3. 사용자는 다음 위치(공의 낙하지점, 다음 홀)로 이동한다. **캐디 로봇은 사용자를 추적한다.**
4. 사용자가 캐디 로봇에 다가간다. 사용자는 가방에서 클럽을 꺼낸다.







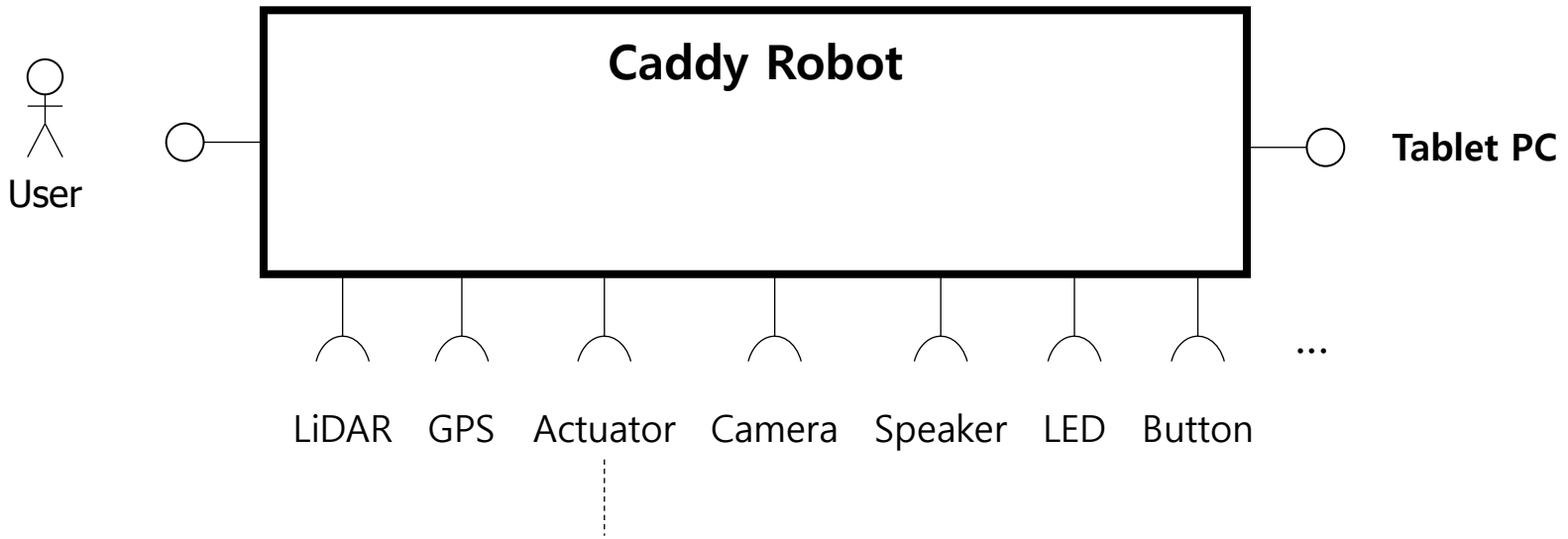
캐디 로봇의 사용성에 관한 평가 기준

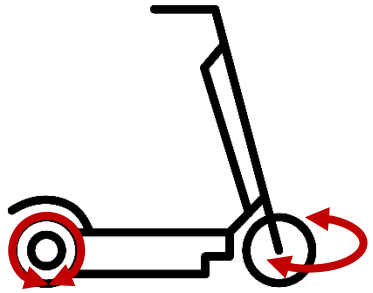
사용자 추적

- 사용자를 놓치지 않아야 한다.
- 장애물이나 출입 불가 지역을 회피할 수 있어야 한다.

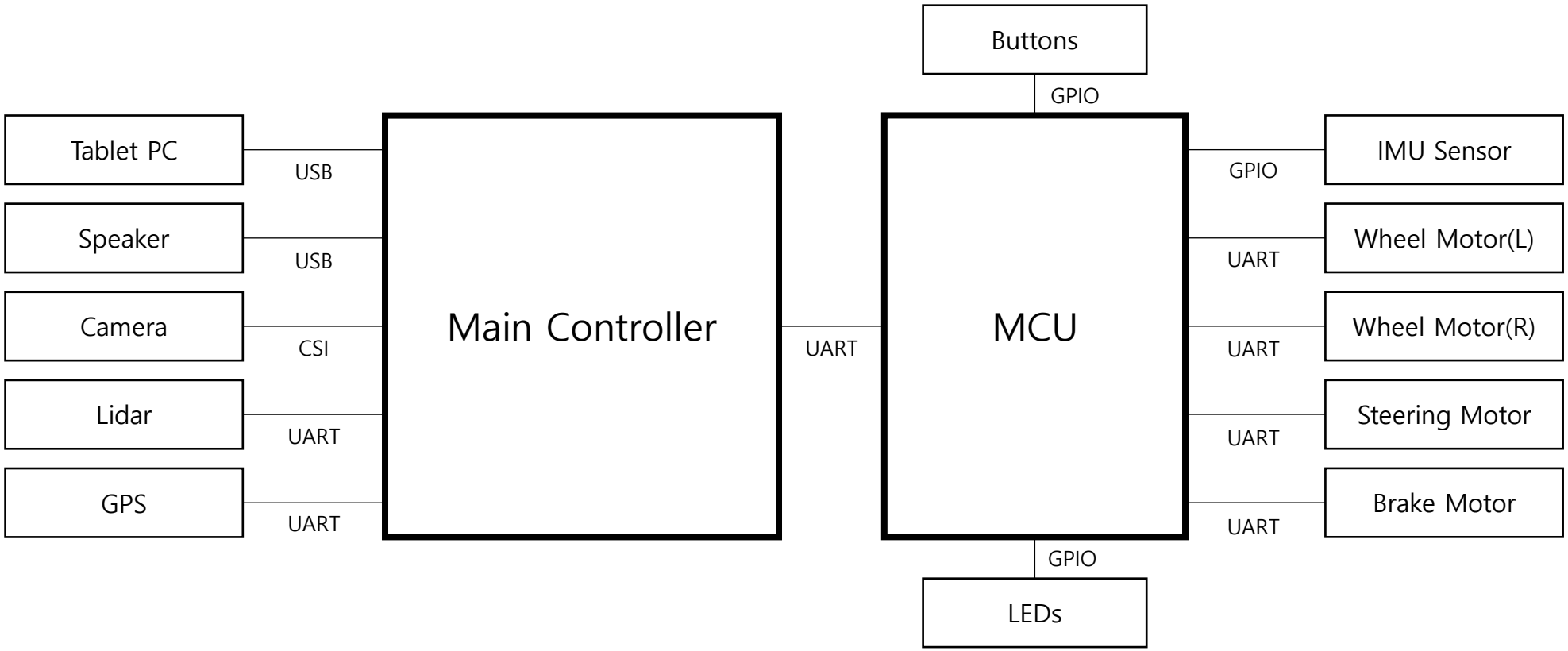


시스템 정의

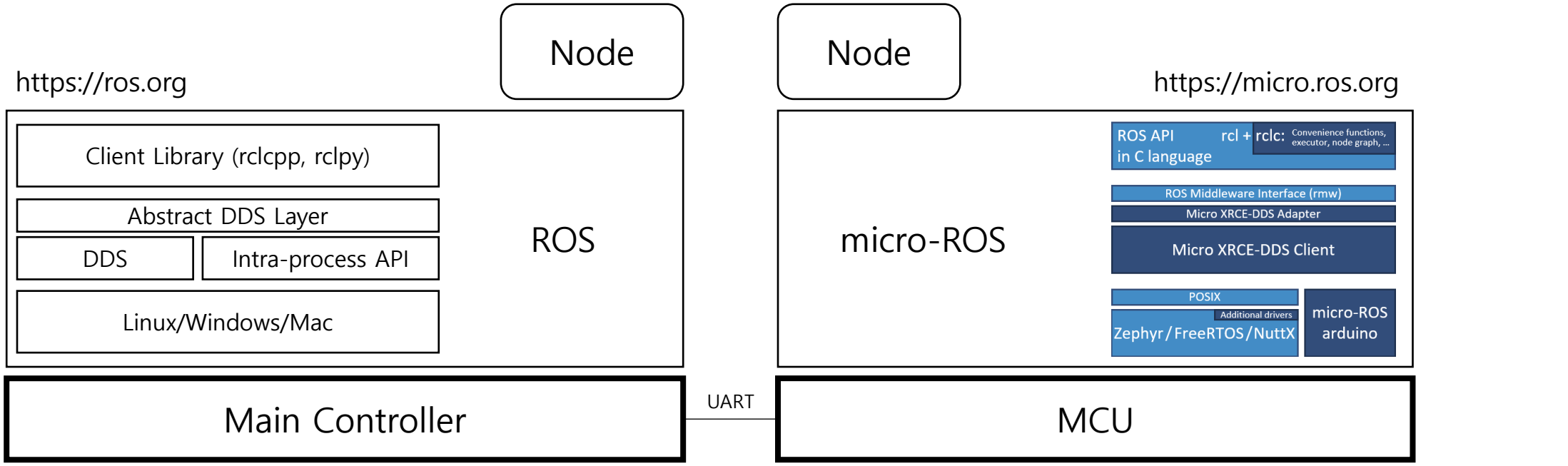

Caddy Robot
User
Tablet PC
LiDAR
GPS
Actuator
Camera
Speaker
LED
Button
...
Motor(L)
Motor(R)
Steering Motor
Braking Motor



시스템 정의 – HW


Buttons
GPIO
Tablet PC
USB
Speaker
USB
Camera
CSI
Lidar
UART
GPS
UART
Main Controller
UART
MCU
GPIO
IMU Sensor
UART
Wheel Motor(L)
UART
Wheel Motor(R)
UART
Steering Motor
UART
Brake Motor
GPIO
LEDs

시스템 정의 – ROS



OMG DDS(Data Distribution Service)

XRCE-DDS(eXtremely Resource Constrained Environments-DDS)

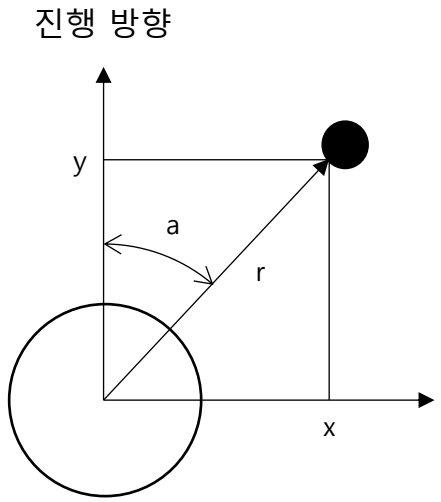
시스템 정의 – LiDAR



LSLIDAR M10

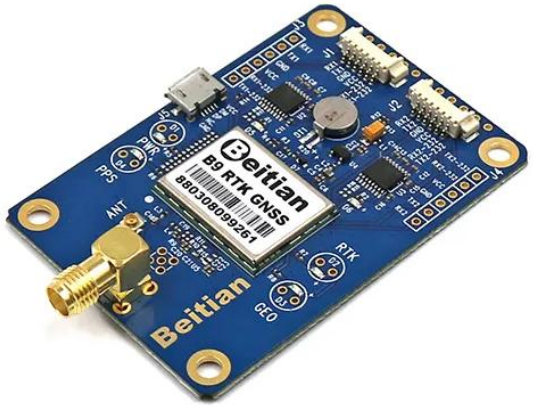
Model	M10
Wavelength	905nm
Laser Classification	Class 1 (Eye-safe)
Data Content	Distance, Angle
Measurement Range	25m@70% (10m@10%)
Ranging Accuracy	±3cm
FOV	360°
Rotation Rate	10Hz
Data Points Generated	10,000 pts/s
Angular Resolution	0.36°
Operating Voltage	5V
Operating Temperature	-20℃ ~ 60℃
Motor	Built-in Brushless Motor
Communication Interface	UART(Baud rate 460800)
Shock Test	500m/sec², last11ms
Vibration	5Hz-2000Hz, 3G rms
Weight	200g
Dimension (D·H)	Φ80*40mm

https://www.lslidar.com/product/m10-navigation-obstacle-avoidance-lidar/
https://github.com/Lslidar/Lslidar_ROS2_driver/tree/M10_V1.0



M10의 경우, 10Hz, 10000pts/s 이므로
한번에 1000개의 거리 데이터가 제공된다.
즉, 0.36도마다 측정한다.

시스템 정의 – GPS



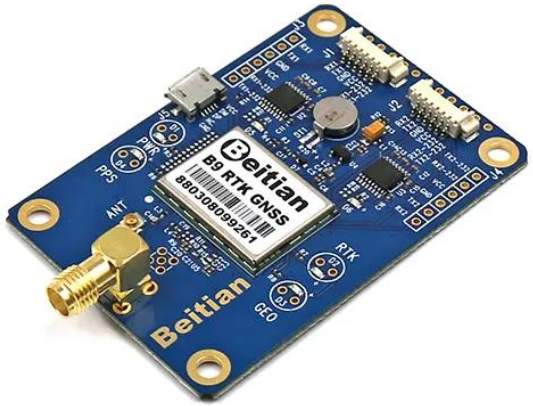
RTK GNSS 모듈 BT-F9PK2

GPS (Global Positioning System)
GNSS(Global Navigation Satellite System)

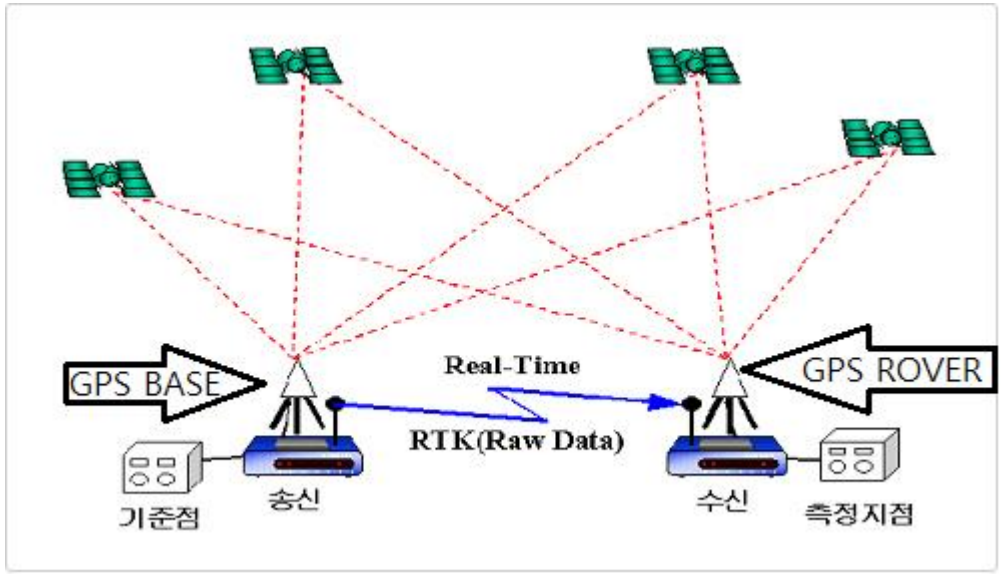
시스템명칭	운용국	시스템 완성 연도	위성 수(개)	비고
GPS	미국	1994년	32	시스템 유지에 최소 24개 위성 필요
글로나스(GLONASS)	러시아	1995년	24	시스템 붕괴로 중단, 2009년 재개 예정
갈릴레오(Galileo)	유럽연합(EU)	2013년	30	한국 참여
베이더우(北斗)	중국	2010년 이후	35	베이징 올림픽 기간 중 시범운용
준텐초(準千頂)	일본	2009년	3개이상	일본 내 우선 서비스
IRNSS	인도	2013년	7	인도 내 우선 서비스

글로벌 항법 위성 시스템(GNSS) 운용 및 개발 현황

시스템 정의 – GPS



RTK GNSS 모듈 BT-F9PK2



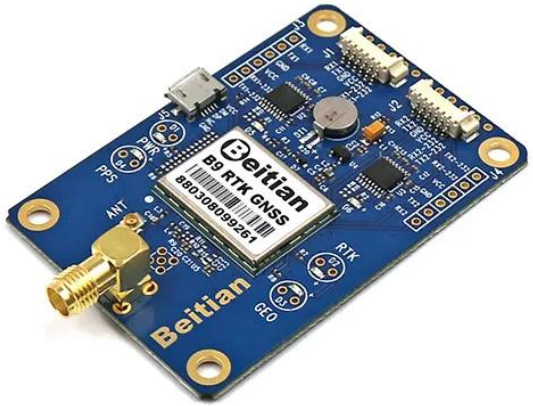
RTK (Real-Time Kinematic) Positioning

VRS (Virtual Reference Station) ← Network RTK

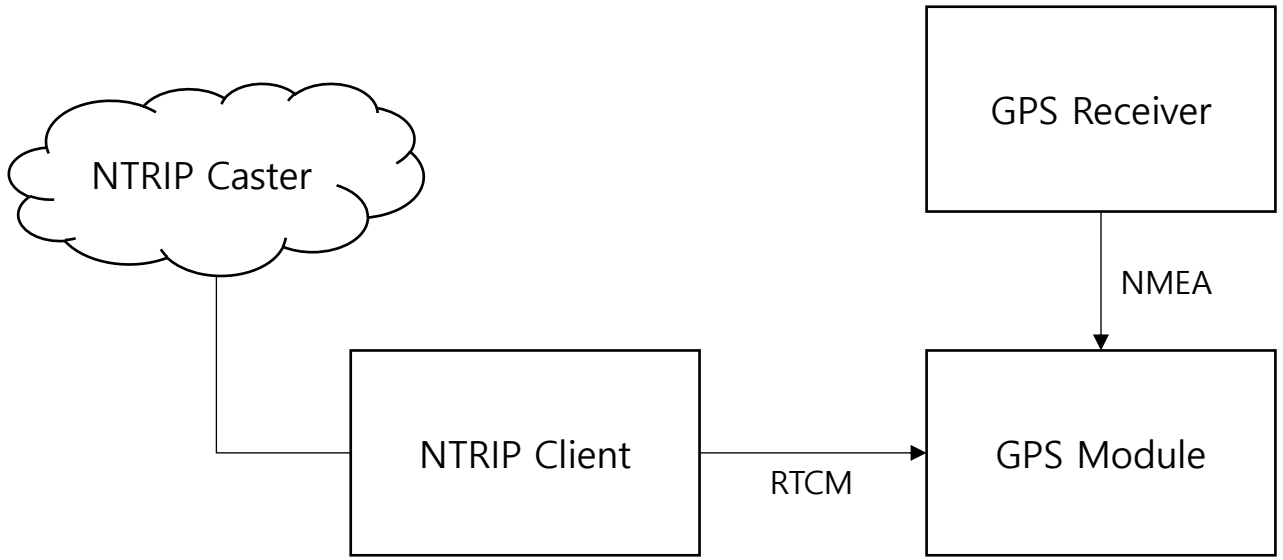
국토지리정보원(에서 제공하는 보정 신호를 이용함

시스템 정의 – GPS

https://github.com/leighleighleigh/ublox_dgnss



RTK GNSS 모듈 BT-F9PK2

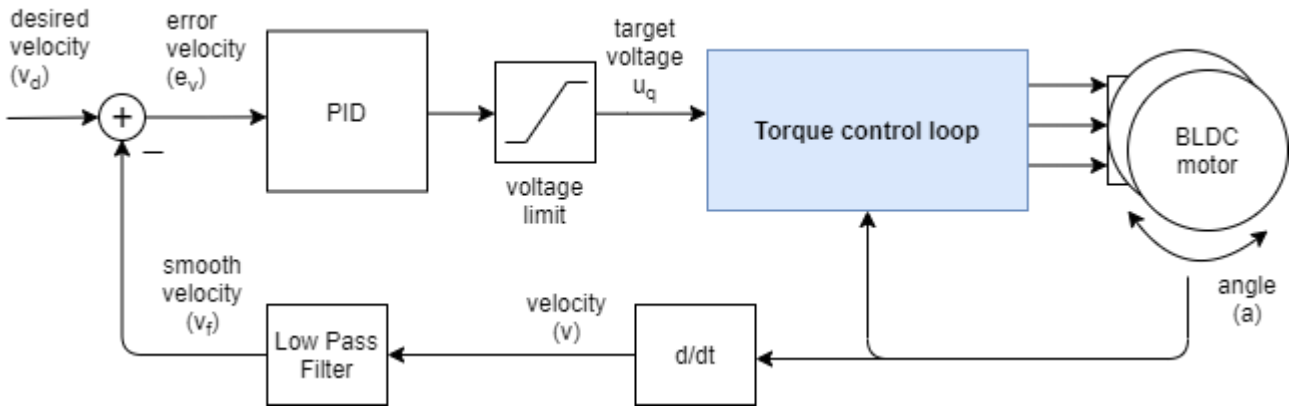


RTCM SC-104 (Radio Technical Commission for Maritime Services)
← GPS 보정 신호 전송 데이터 포맷
NTRIP (Network Transport of RTCM via Internet Protocol)
NMEA-0183 (National Marine Electronics Association) ← GPS 표준 프로토콜

시스템 정의 – BLDC Motor

https://en.wikipedia.org/wiki/Vector_control_(motor)
https://docs.simplefoc.com/

FOC(Field Oriented Control) for Brushless DC Moter




```
// velocity set point variable
float target_velocity = 2;    // 2 rad/s

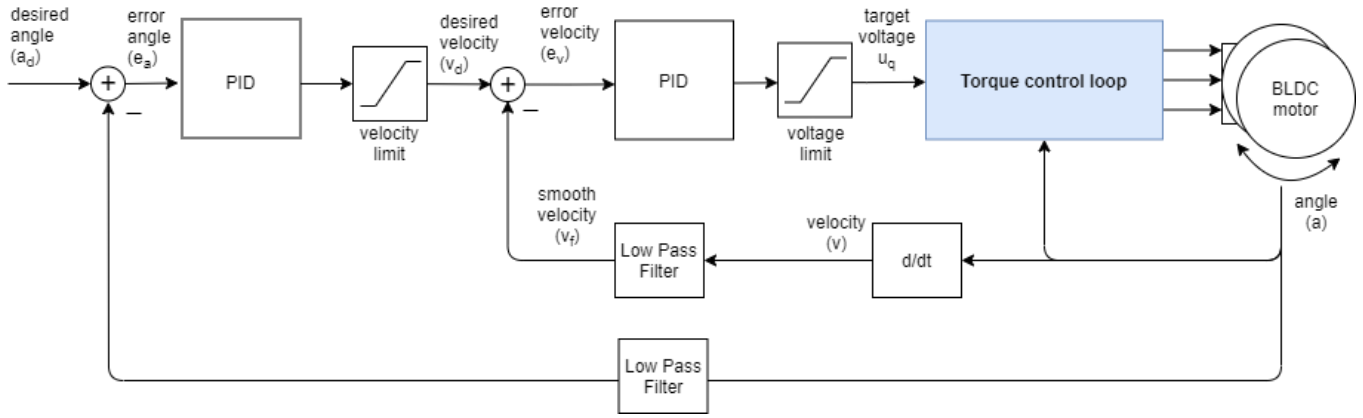
void loop() {
  // main FOC algorithm function
  motor.loopFOC();

  // Motion control function
  motor.move(target_velocity);
}
```

시스템 정의 – BLDC Motor

https://en.wikipedia.org/wiki/Vector_control_(motor)
https://docs.simplefoc.com/

FOC(Field Oriented Control) for Brushless DC Moter




```
// angle set point variable
float target_angle = 1;    // 1 rad

void loop() {
  // main FOC algorithm function
  motor.loopFOC();

  // Motion control function
  motor.move(target_angle);
}
```

https://github.com/chiprobotics/chip_bldc_driver

chip_bldc_driver::Command
int16 motor_command
[-1000, 1000]

<<node>>
bldc_driver

시스템 정의 – ROS Messages

https://wiki.ros.org/common_msgs
https://github.com/ros2/common_interfaces

ROS2 interfaces

sensor_msgs/msg/LaserScan.msg

```
std_msgs/msg/Header header
float32 angle_min
float32 angle_max
float32 angle_increment
float32 time_increment
float32 scan_time
float32 range_min
float32 range_max
float32[] ranges
float32[] intensities
```

sensor_msgs/msg/NavSatFix.msg

```
uint8 COVARIANCE_TYPE_UNKNOWN=0
uint8 COVARIANCE_TYPE_APPROXIMATED=1
uint8 COVARIANCE_TYPE_DIAGONAL_KNOWN=2
uint8 COVARIANCE_TYPE_KNOWN=3
std_msgs/msg/Header header
sensor_msgs/msg/NavSatStatus status
float64 latitude
float64 longitude
float64 altitude
float64[9] position_covariance
uint8 position_covariance_type
```

geometry_msgs/msg/Twist

```
geometry_msgs/msg/Vector3 linear
geometry_msgs/msg/Vector3 angular
```

geometry_msgs/msg/Pose

```
geometry_msgs/msg/Point position
geometry_msgs/msg/Quaternion orientation
```

std_msgs/msg/Float32

```
float32 data
```

ROS - Robot Operating System

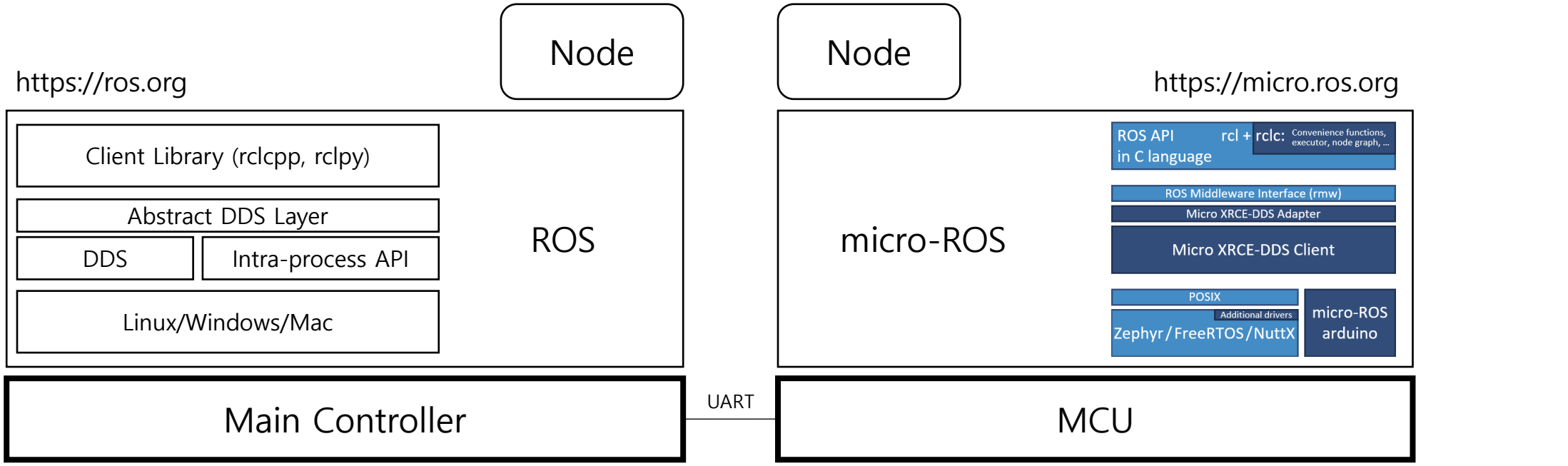
The Robot Operating System (ROS) is a set of software libraries and tools that help you build robot applications. From drivers to state-of-the-art algorithms, and with powerful developer tools, ROS has what you need for your next robotics project. And it's all open source.



https://ros.org

주식회사 보이는 소프트웨어 연구소
http://www.bosornd.com

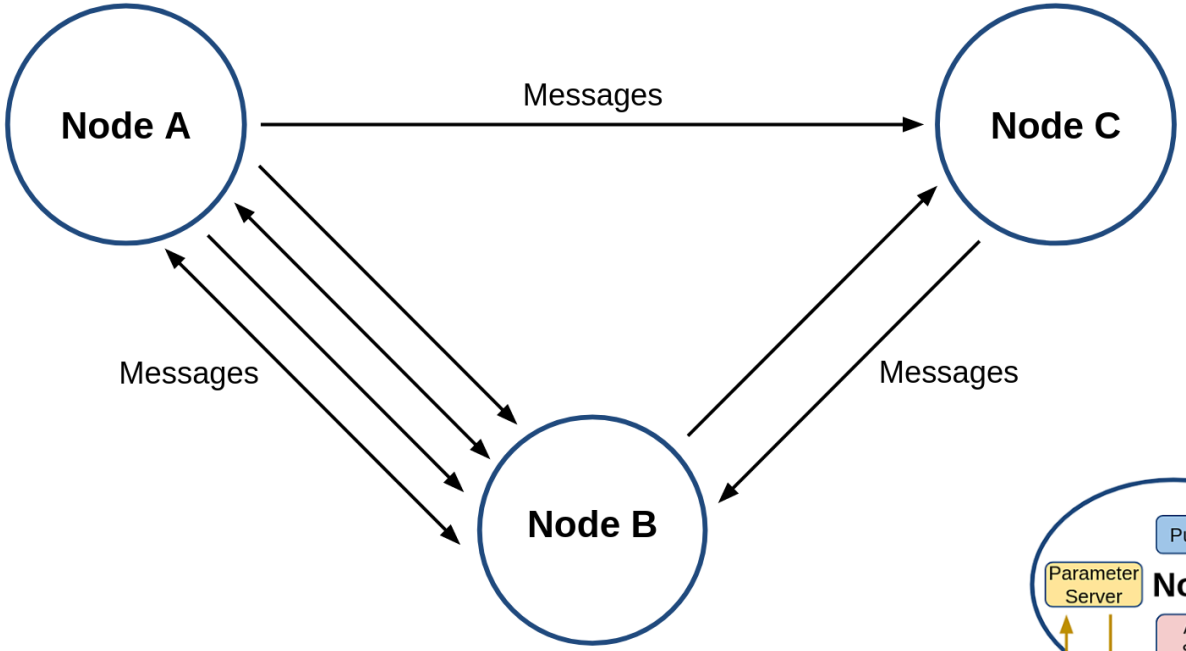
ROS Architecture

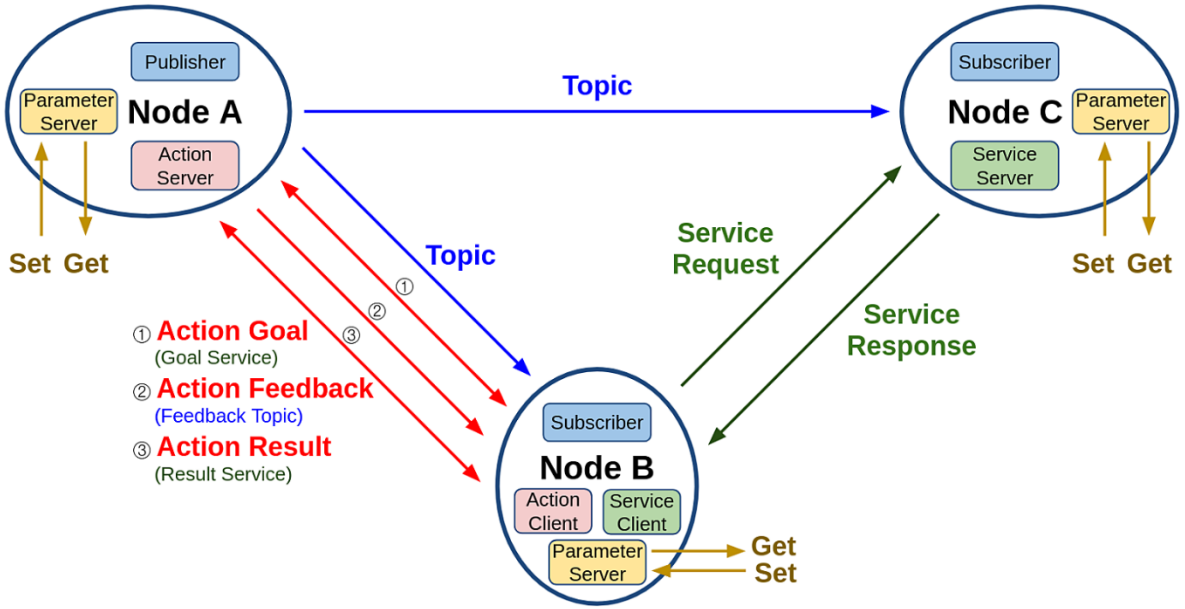


OMG DDS(Data Distribution Service)

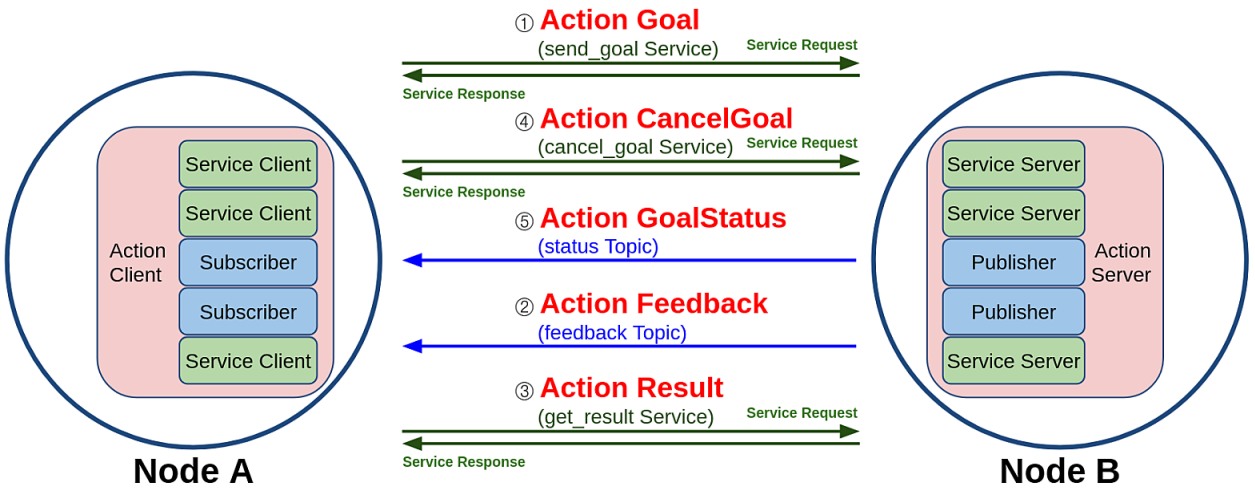
XRCE-DDS(eXtremely Resource Constrained Environments-DDS)

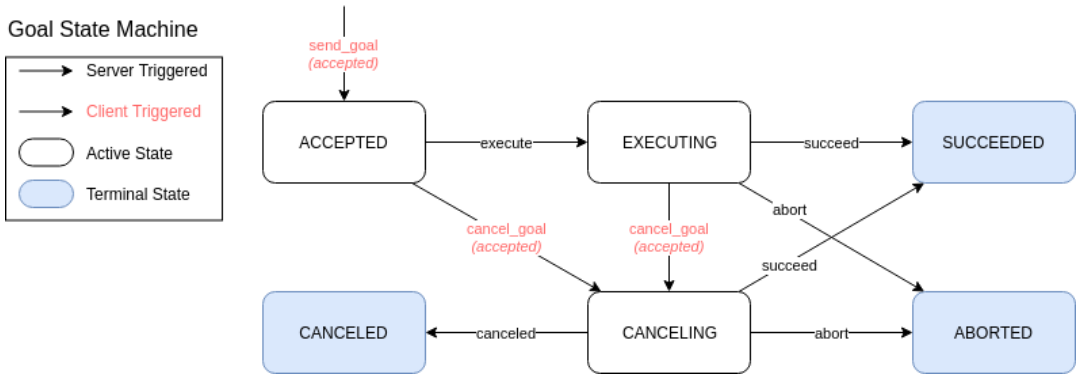
Node & message, topic, service, action

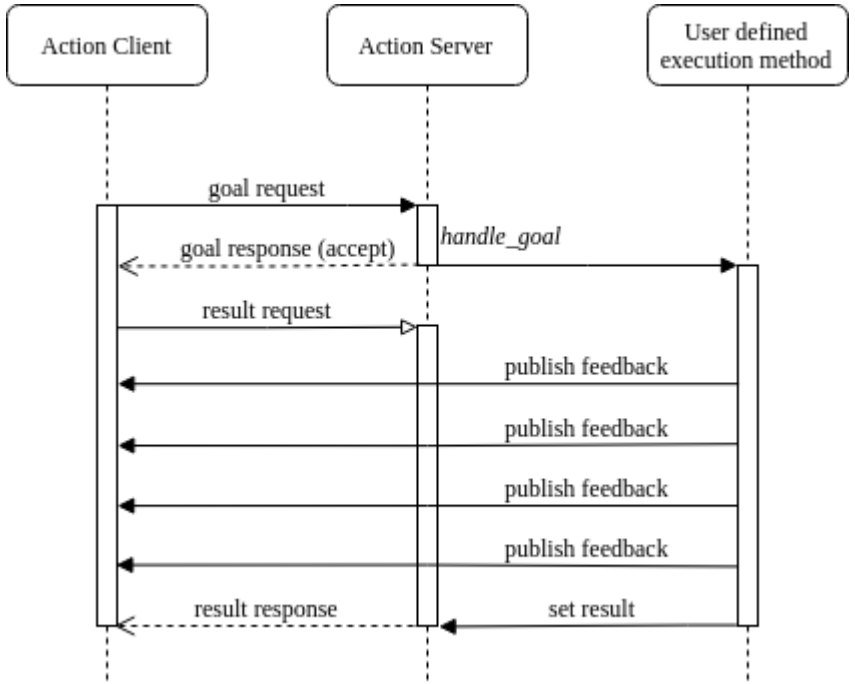




Node & message, topic, service, action





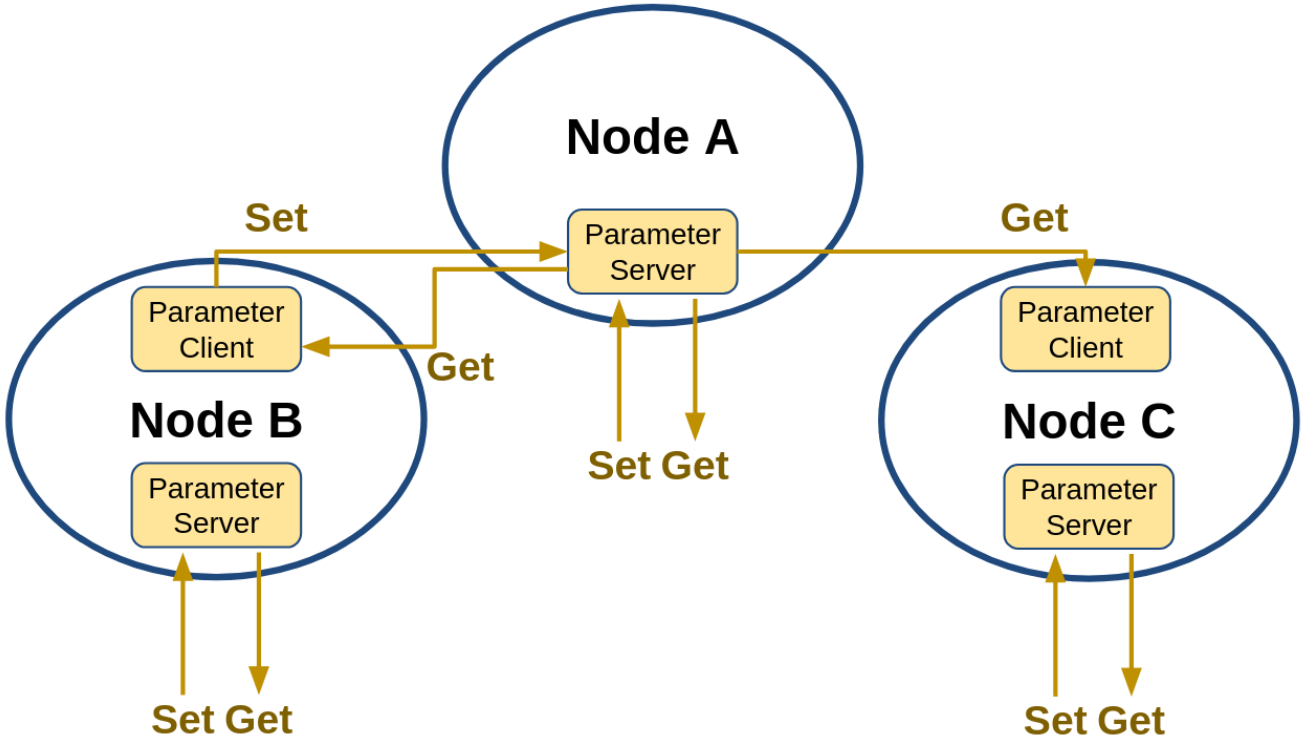


Node & message, topic, service, action

	토픽 (topic)	서비스 (service)	액션 (action)
연속성	연속성	일회성	복합 (토픽+서비스)
방향성	단방향	양방향	양방향
동기성	비동기	동기	동기 + 비동기
다자간 연결	1:1, 1:N, N:1, N:N (publisher:subscriber)	1:1 (server:client)	1:1 (server:client)
노드 역할	발행자 (publisher) 구독자 (subscriber)	서버 (server) 클라언트 (client)	서버 (server) 클라언트 (client)
동작 트리거	발행자	클라언트	클라언트
인터페이스	msg 인터페이스	srv 인터페이스	action 인터페이스
CLI 명령어	ros2 topic ros2 interface	ros2 service ros2 interface	ros2 action ros2 interface
사용 예	센서 데이터, 로봇 상태, 로봇 좌표, 로봇 속도 명령 등	LED 제어, 모터 토크 On/Off, IK/FK 계산, 이동 경로 계산 등	목적지로 이동, 물건 파지, 복합 태스크 등

	msg 인터페이스	srv 인터페이스	action 인터페이스
확장자	*.msg	*.srv	*.action
데이터	토픽 데이터 (data)	서비스 요청 (request) --- 서비스 응답 (response)	액션 목표 (goal) --- 액션 결과 (result) --- 액션 피드백 (feedback)
형식	fieldtype1 fieldname1 fieldtype2 fieldname2 fieldtype3 fieldname3	fieldtype1 fieldname1 fieldtype2 fieldname2 --- fieldtype3 fieldname3 fieldtype4 fieldname4	fieldtype1 fieldname1 fieldtype2 fieldname2 --- fieldtype3 fieldname3 fieldtype4 fieldname4 --- fieldtype5 fieldname5 fieldtype6 fieldname6
사용 예	[geometry_msgs/msg/Twist]	[turtlesim/srv/Spawn.srv]	[turtlesim/action/RotateAbsolute.action]
	Vector3 linear Vector3 angular	float32 x float32 y float32 theta string name --- string name	float32 theta --- float32 delta --- float32 remaining

Node & parameter

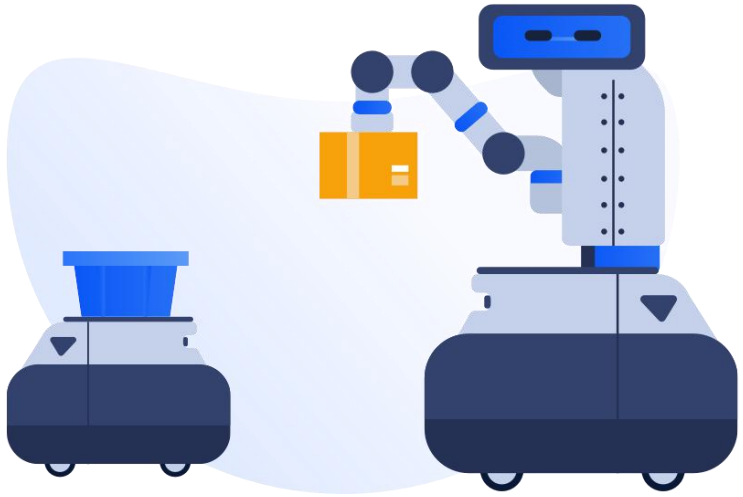


ROS 추천 패키지

https://cafe.naver.com/openrt/25422

MoveIt – Robotic Manipulation Platform

https://moveit.ros.org



Motion Planning
Generate high-degree of freedom trajectories through cluttered environments and avoid local minimums

Manipulation
Analyze and interact with your environment with grasp generation

Inverse Kinematics
Solve for joint positions for a given pose, even in over-actuated arms

Control
Execute time-parameterized joint trajectories to low level hardware controllers through common interfaces

3D Perception
Connect to depth sensors and point clouds with Octomaps

Collision Checking
Avoid obstacles using geometric primitives, meshes, or point cloud data

ROS 추천 패키지

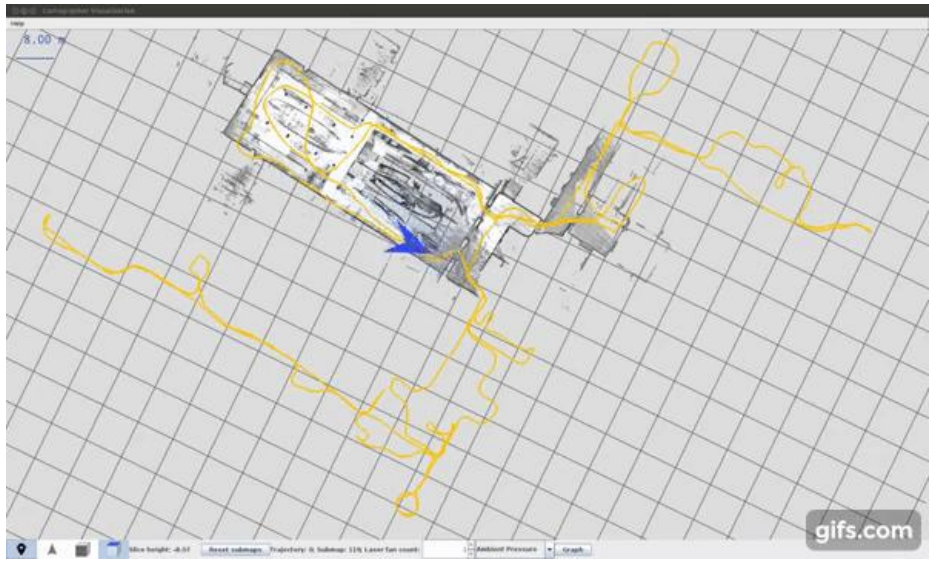
https://cafe.naver.com/openrt/25422

Cartographer

https://opensource.google/projects/cartographer

Google에서 개발하여 오픈소스로 공개한 SLAM 시스템이다.

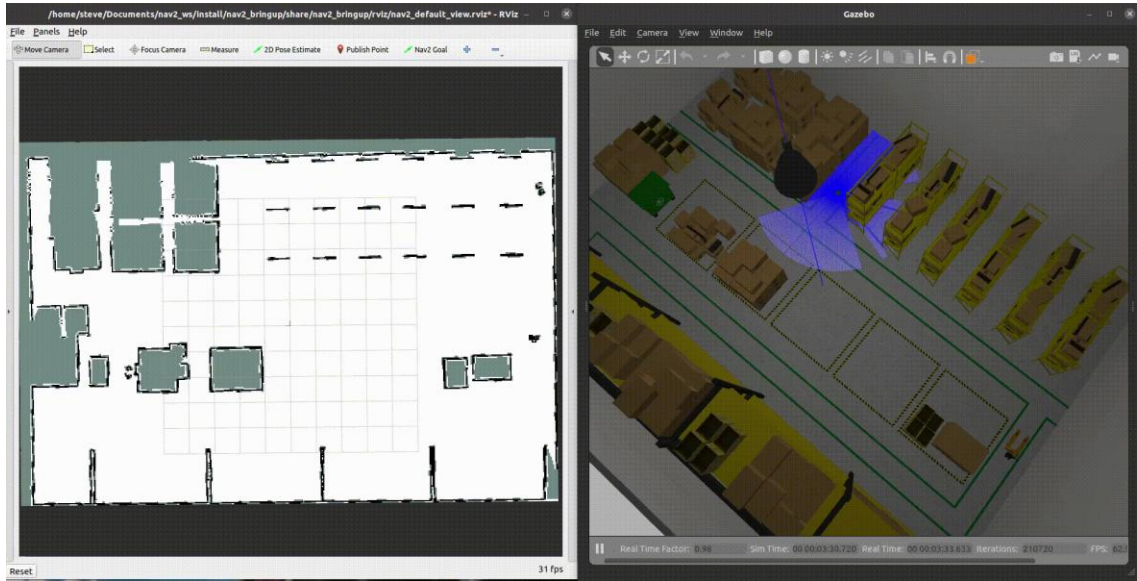
ROS 2에서 가장 많이 사용되고 있는 SLAM이다.



NAV2

https://navigation.ros.org/

모바일 로봇에 필요한 내비게이션 기능을 모아둔 패키지이다.



caddybot

https://github.com/bosornd/caddybot

1. Docker Desktop 설치 (https://www.docker.com/)
2. ROS2 humble vnc 컨테이너 생성

   C:₩> docker run -p 8080:80 -it --name ros2 tiryoh/ros2-desktop-vnc:humble

3. 빌드 및 실행

build

1. update packages

```
$ sudo apt update && sudo apt upgrade
```

2. git clone

```
$ git clone --recursive https://github.com/bosornd/caddybot.git
$ cd caddybot
```

3. install dependencies

```
$ rosdep install --from-paths src --ignore-src --rosdistro humble -y
```

```
$ pip install pygame
$ pip install pyserial sparkfun-ublox-gps utm
```

4. build

```
$ colcon build
$ source install/setup.bash
```

5. build micro_ros_agent for MCU

```
$ ros2 run micro_ros_setup create_agent_ws.sh
$ ros2 run micro_ros_setup build_agent.sh
```

launch

using terminal1(ROS2),

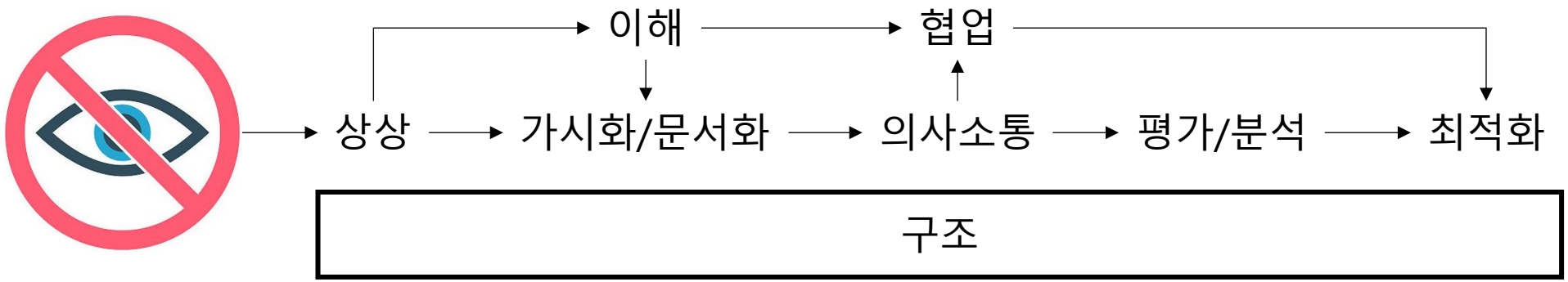
```
$ source install/setup.bash
$ ros2 launch caddybot bringup.py
```

구조도

cc BY NC ND

아키텍트의 역할

- 구조도를 그린다.



- 최적 설계/개발을 이끈다.

Software Architecture

The software architecture of a system is

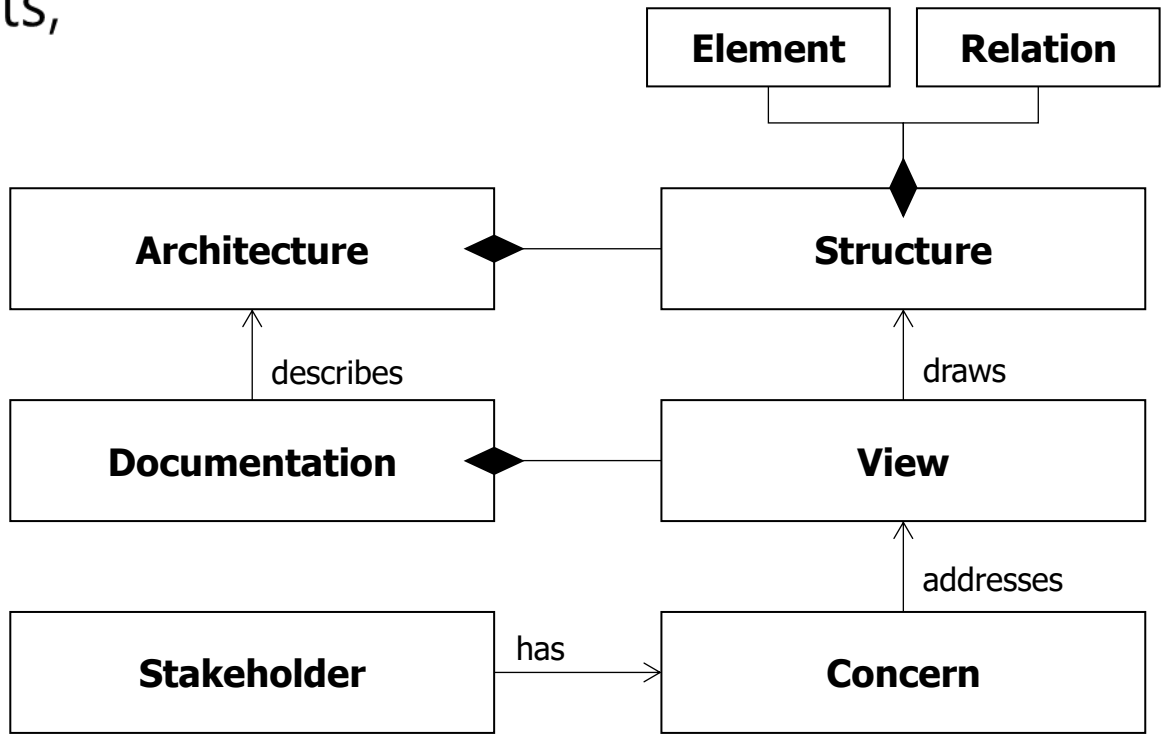
the set of structures

needed to reason about the system,

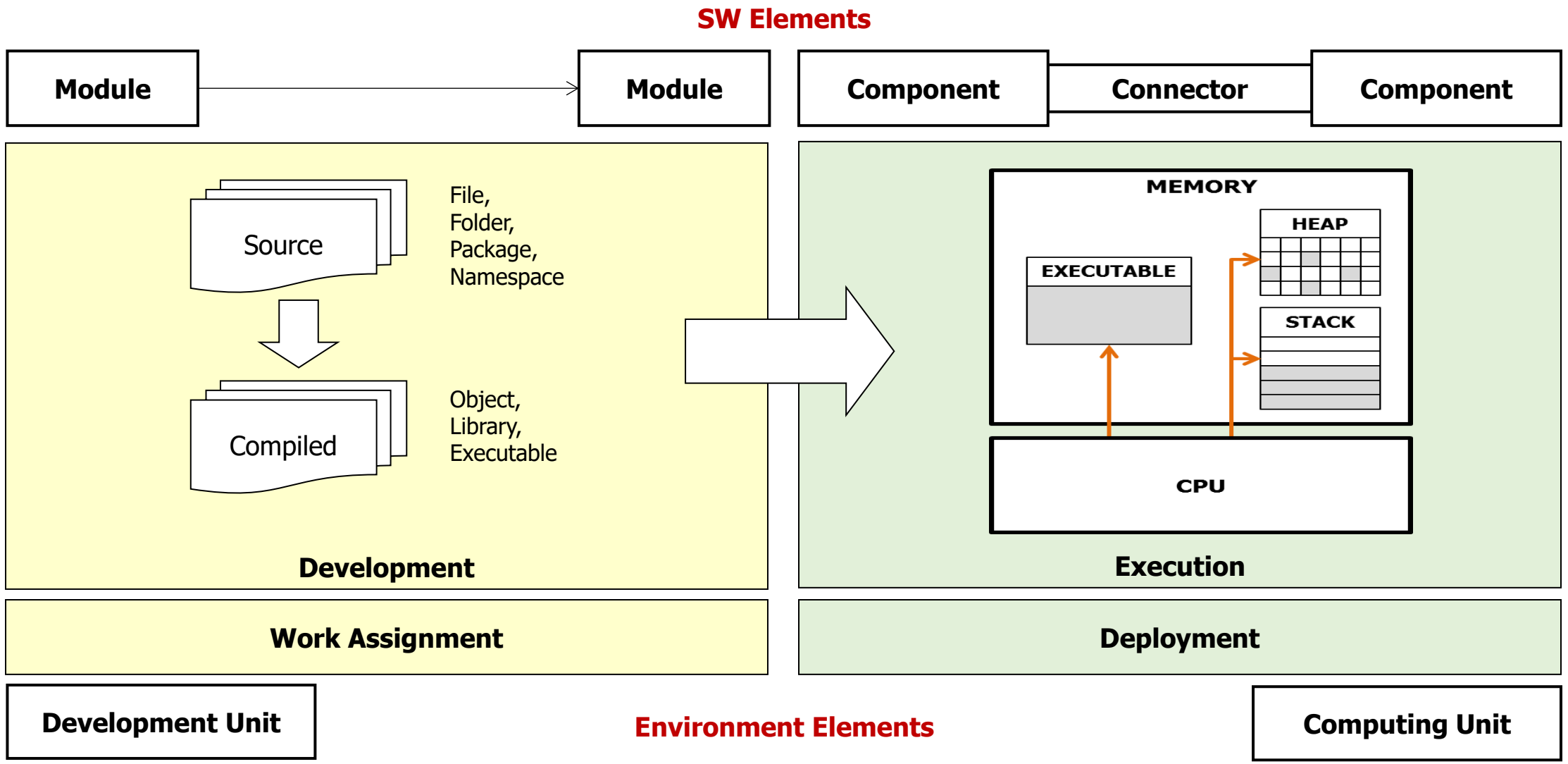
which comprise software elements,

relations among them,

and properties of both.



Kinds of Structures / Views



Module View

Elements	• **Modules**, which are implementation units of software.
Relations	• **Is part of**, which defines a part/whole relationship between the submodule the part-and the aggregate module the whole. • **Depends on**, which defines a dependency relationship between two modules. Specific module views elaborate what dependency is meant. • **Is a**, which defines a generalization/specialization relationship between a more specific module and a more general module.
Constraints	• Different module views may impose specific topological constraints, such as limitations on the visibility between modules.
Usage	• **Blueprint for construction of the code** • **Change-impact analysis** • Planning incremental development • Requirements traceability analysis • Communicating the functionality of a system and the structure of its code base • **Supporting the definition of work assignments, implementation schedules, and budget information** • Showing the structure of information that the system needs to manage

Layered Style

Overview	• The layered style puts together layers (groupings of modules that offer a cohesive set of services) in a **unidirectional allowed-to-use relation** with each other.
Elements	• **Layer**. The description of a layer should define what modules the layer contains.
Relations	• **Allowed to use**, which is a specialization of the generic **depends-on** relation. The design should define the layer usage rules (for example, "A layer is allowed to use any lower layer.") and any allowable exceptions.
Constraints	• Every piece of software is allocated to exactly one layer. • There are at least two layers (typically three or more). • The allowed-to-use relations should not be circular (that is, a lower layer cannot use a layer above).
Usage	• Promoting modifiability and portability • Managing complexity and facilitating the communication of the code structure to developers • Promoting reuse • Achieving separation of concerns

Layered Style - Example

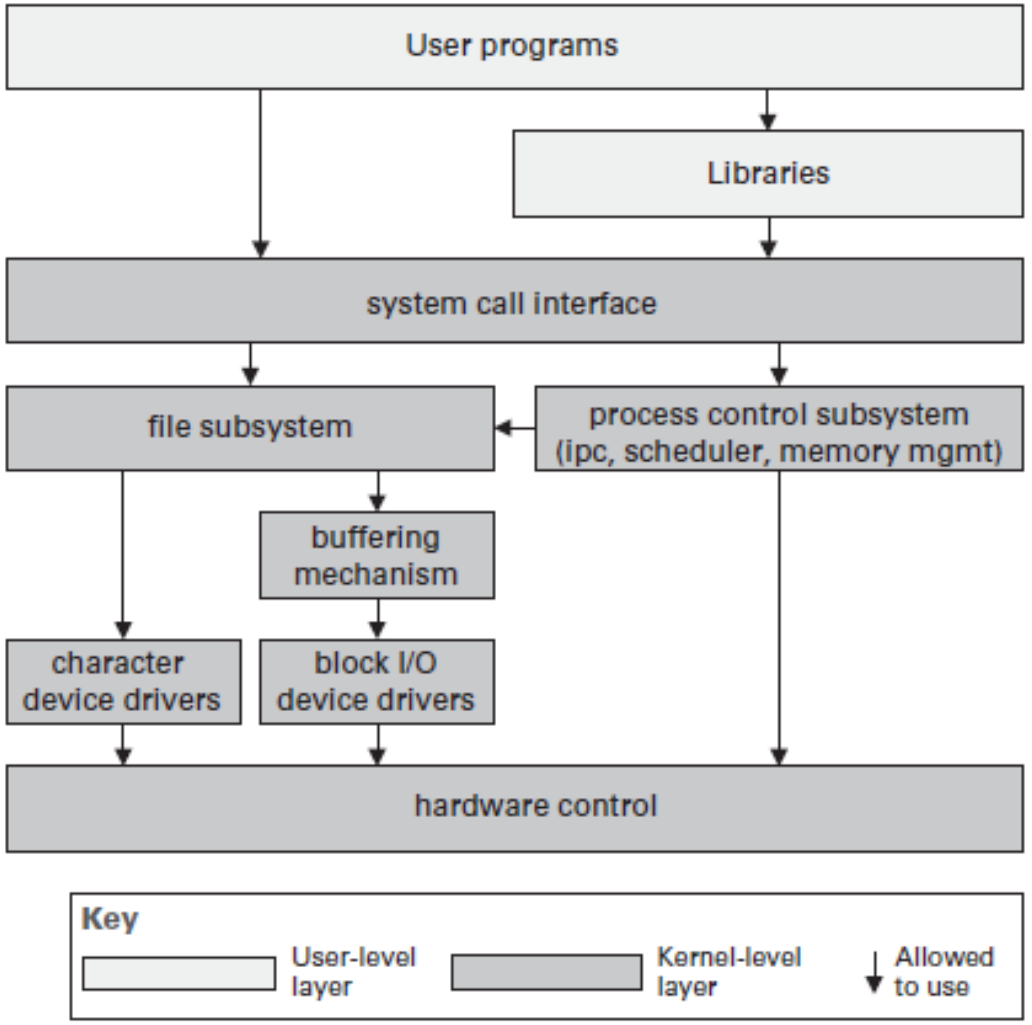


Figure 2.26
The primary presentation of a layered view of the UNIX System V operating system implementation (adapted from Bach 1986)

Work Assignment View

Overview	• The work assignment view describes the mapping of the software architecture to the teams in the development organization.
Elements	• **Software element**: a module. Properties include the required skill set and available capacity (effort, time) needed. • **Environmental element**: an organizational unit, such as a person, a team, a department, a subcontractor, and so on. Properties include the provided skill set and the capacity in terms of labor and calendar time available.
Relations	• **Allocated-to**. A software element is allocated to an organizational unit.
Constraints	• In general, the allocation is unrestricted; in practice, it is usually restricted so that one module is allocated to one organizational unit.
Usage	• For reasoning about distributed development and allocation of work to teams.

Work Assignment View - Example

ECS Element (Module)		
Segment	**Subsystem**	**Organizational Unit**
Science Data Processing Segment (SDPS)	Client	Science team
	Interoperability	Prime contractor team 1
	Ingest	Prime contractor team 2
	Data Management	Data team
	Data Processing	Data team
	Data Server	Data team
	Planning	Orbital vehicle team
Flight Operations Segment (FOS)	Planning and Scheduling	Orbital vehicle team
	Data Management	Database team
	User Interface	User interface team
. . .	. . .	. . .

Figure 5.6
Work assignment view using a tabular notation. The left two columns echo the system's module decomposition structure.

Component & Connector View

Elements	• **Components**. Principal processing units and data stores. A component has a set of ports through which it interacts with other components. • **Connectors**. Pathways of interaction between components. Connectors have a set of roles (interfaces) that indicate how components may use a connector in interactions.
Relations	• **Attachments**. Component ports are associated with connector roles to yield a graph of components and connectors. • **Interface delegation**. In some situations, component ports are associated with one or more ports in an "internal sub-architecture. The case is similar for the roles of a connector.
Constraints	• Components can only be attached to connectors, not directly to other components. • Connectors can only be attached to components, not directly to other connectors. • Attachments can only be made between compatible ports and roles. • Interface delegation can only be defined between two compatible ports (or two compatible roles). • Connectors cannot appear in isolation; a connector must be attached to a component.
Usage	• **Show how the system works.** • Guide development by specifying structure and behavior of runtime elements. • **Help reason about runtime system quality attributes, such as performance and availability.**

Client-Server Style

Elements	• **Client,** which is a component that invokes services of a server component. • **Server,** which is a component that provides services to client components. • **Request/reply connector,** which is used by a client to invoke services on a server. Request/reply connectors have two roles: a request role and a reply role. Connector properties may include whether the calls are local or remote, and whether data is encrypted.
Relations	• The **attachment relation** associates client service-request ports with the request role of the connector and server service-reply ports with the reply role of the connector.
Constraints	• Clients are connected to servers through request/reply connectors. • Server components can be clients to other servers. • Components may be arranged in tiers.
Usage	• Promoting modifiability and reuse by factoring out common services • Improving scalability and availability in case server replication is in place • Analyzing dependability, security, and throughput

Client-Server Style - Example

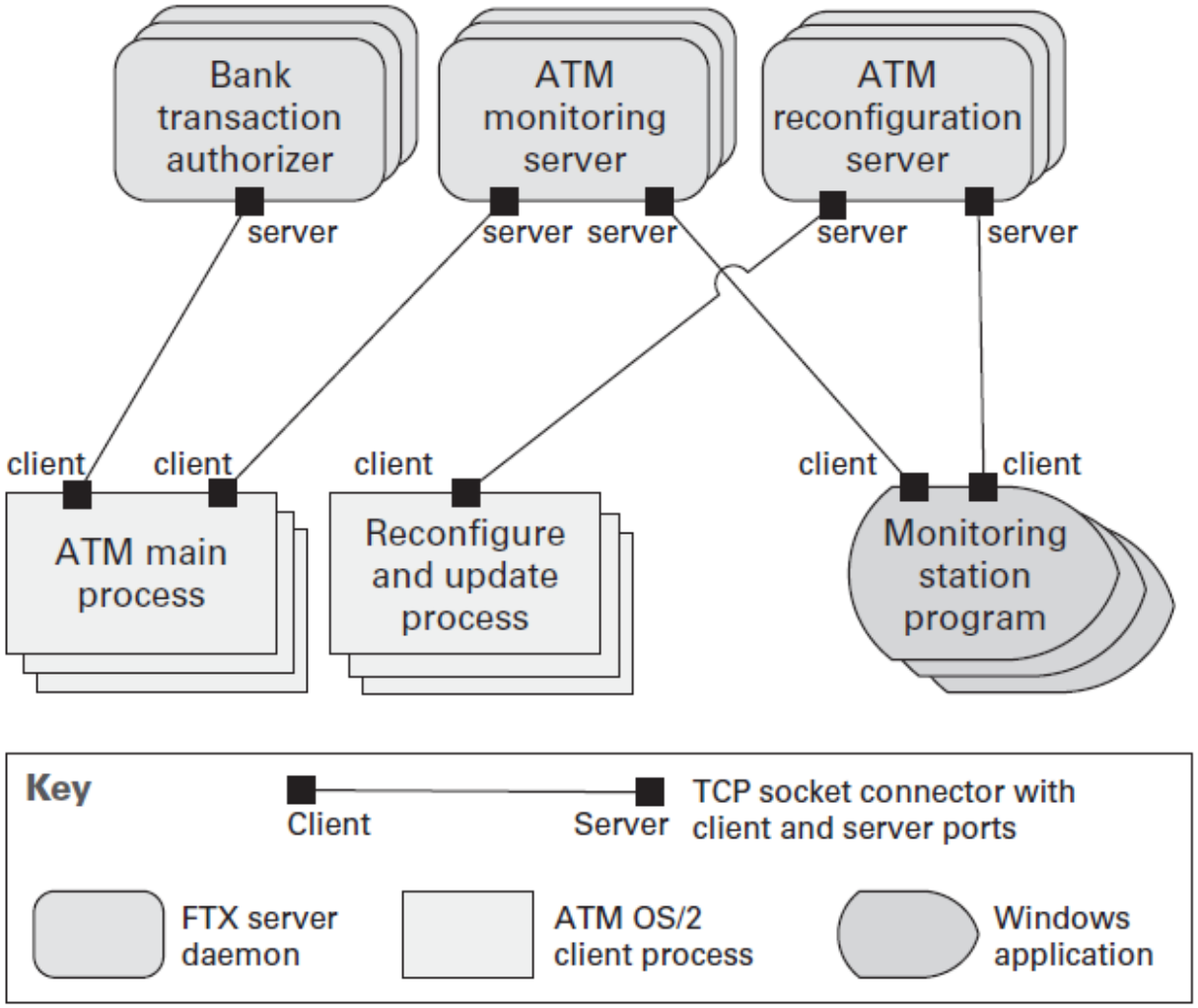
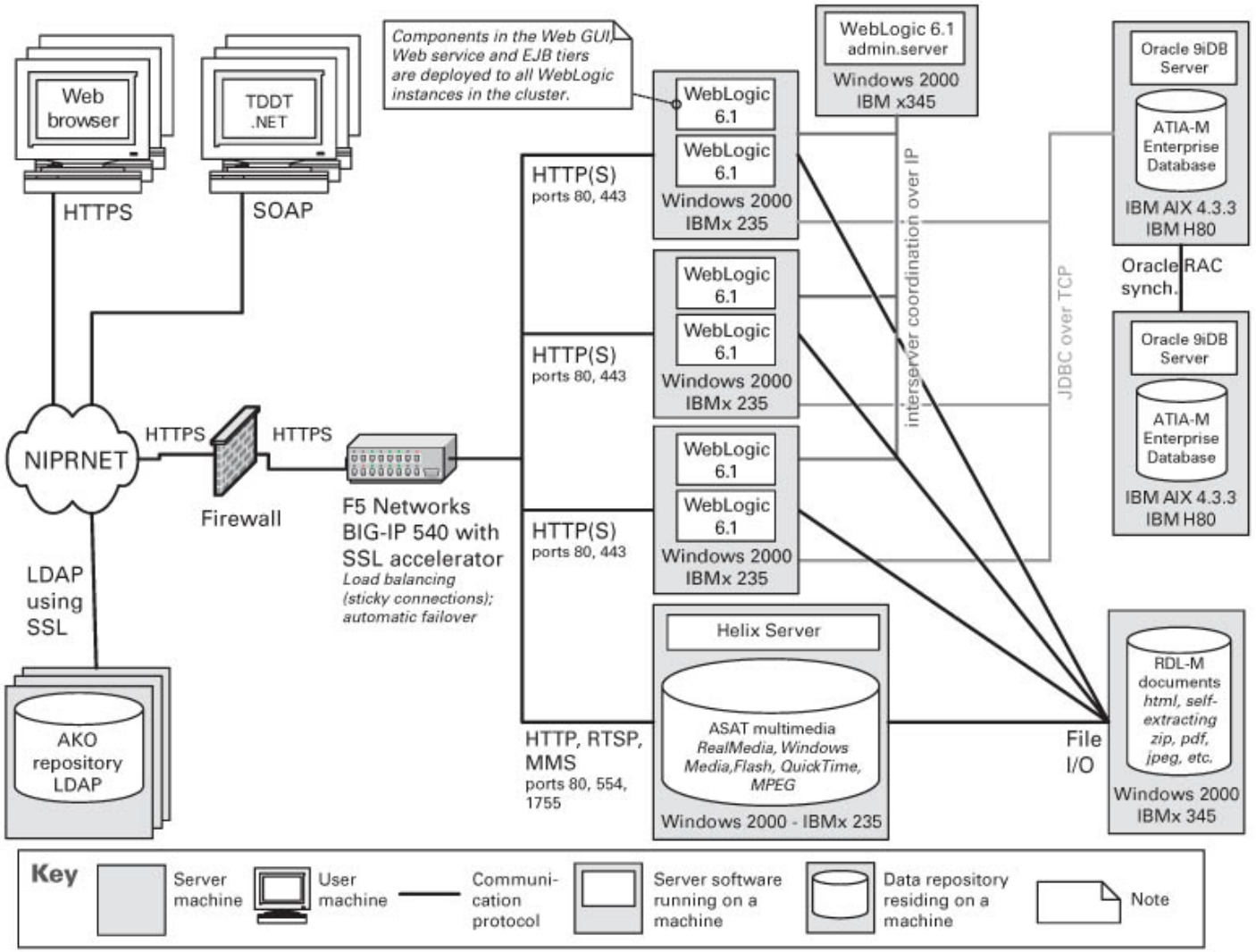


Figure 4.3
Client-server architecture of an ATM banking system. The `ATM main process` sends requests to `Bank transaction authorizer` corresponding to user operations (such as deposit, withdrawal). It also sends messages to `ATM monitoring server` informing the overall status of the ATM (devices, sensors, and supplies). The `Reconfigure and update process` component sends requests to `ATM reconfiguration server` to find out if a reconfiguration command was issued for that particular ATM. Reconfiguration of an ATM (for example, enabling or disabling a menu option) and data updates are issued by bank personnel using the `Monitoring station program`. `Monitoring station program` also sends peri-

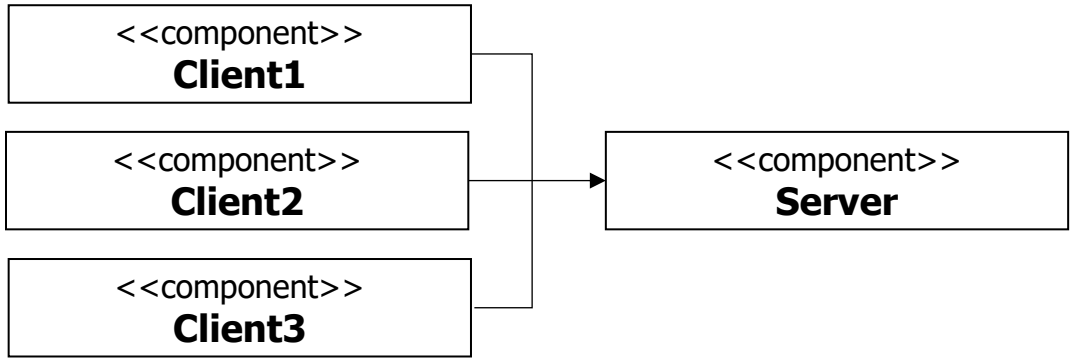
Deployment View

Overview	• The deployment view describes the mapping of components and connectors in the software architecture to the hardware of the computing platform.
Elements	• **Software element**: elements from a C&C view. Useful properties to document include the significant features required from hardware, such as processing, memory, capacity requirements, and fault tolerance. • **Environmental element**: hardware of the computing platform - processor, memory, disk, network(such as router, bandwidth, firewall, bridge), and so on. Useful properties of an environmental element are the significant hardware aspects that influence the allocation decision.
Relations	• **Allocated-to**. Physical units on which the software elements reside during execution. Properties include whether the allocation can change at execution time or not. • **Migrates-to, copy-migrates-to, and/or execution-migrates-to** if the allocation is dynamic. Properties include the trigger that causes the migration.
Constraints	• The allocation topology is unrestricted. However, the required properties of the software must be satisfied by the provided properties of the hardware.
Usage	• For reasoning about performance, availability, security and safety.

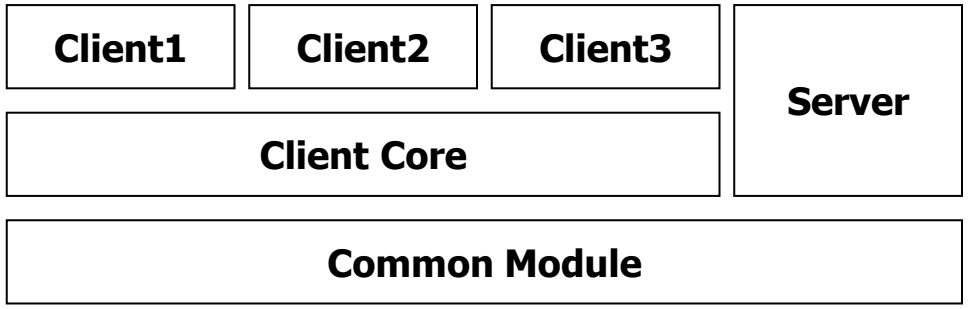
Deployment View - Example



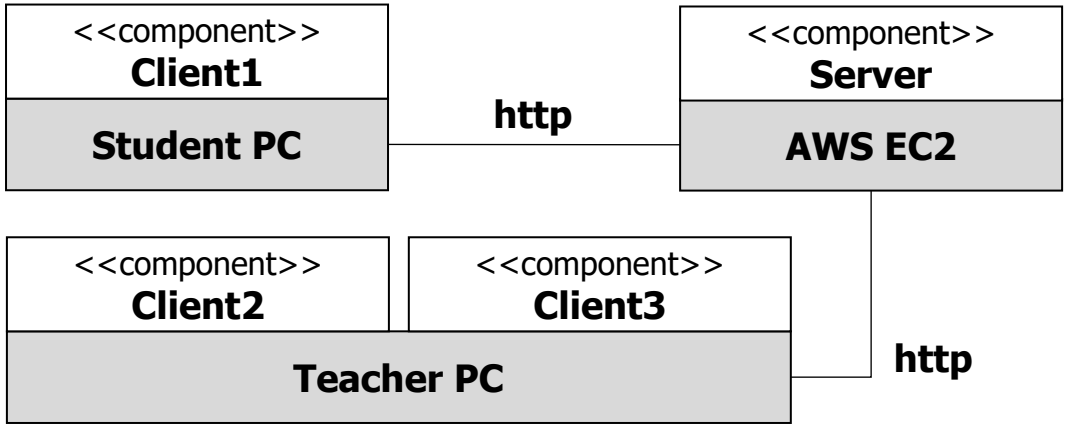
Kinds of Structures / Views



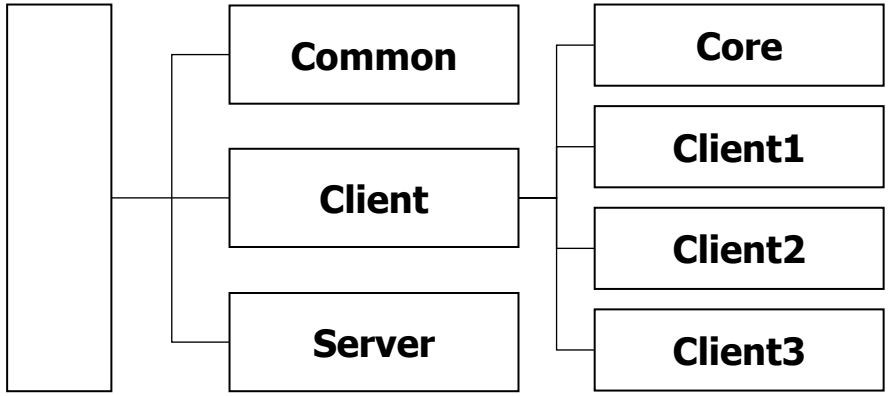
[Component & Connector View]



[Module View]

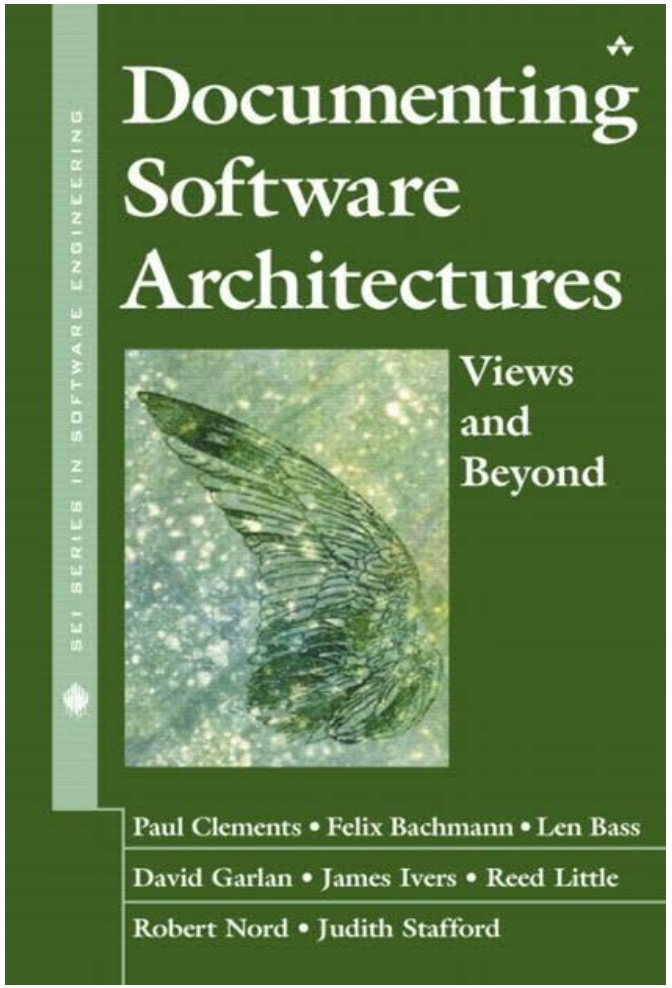


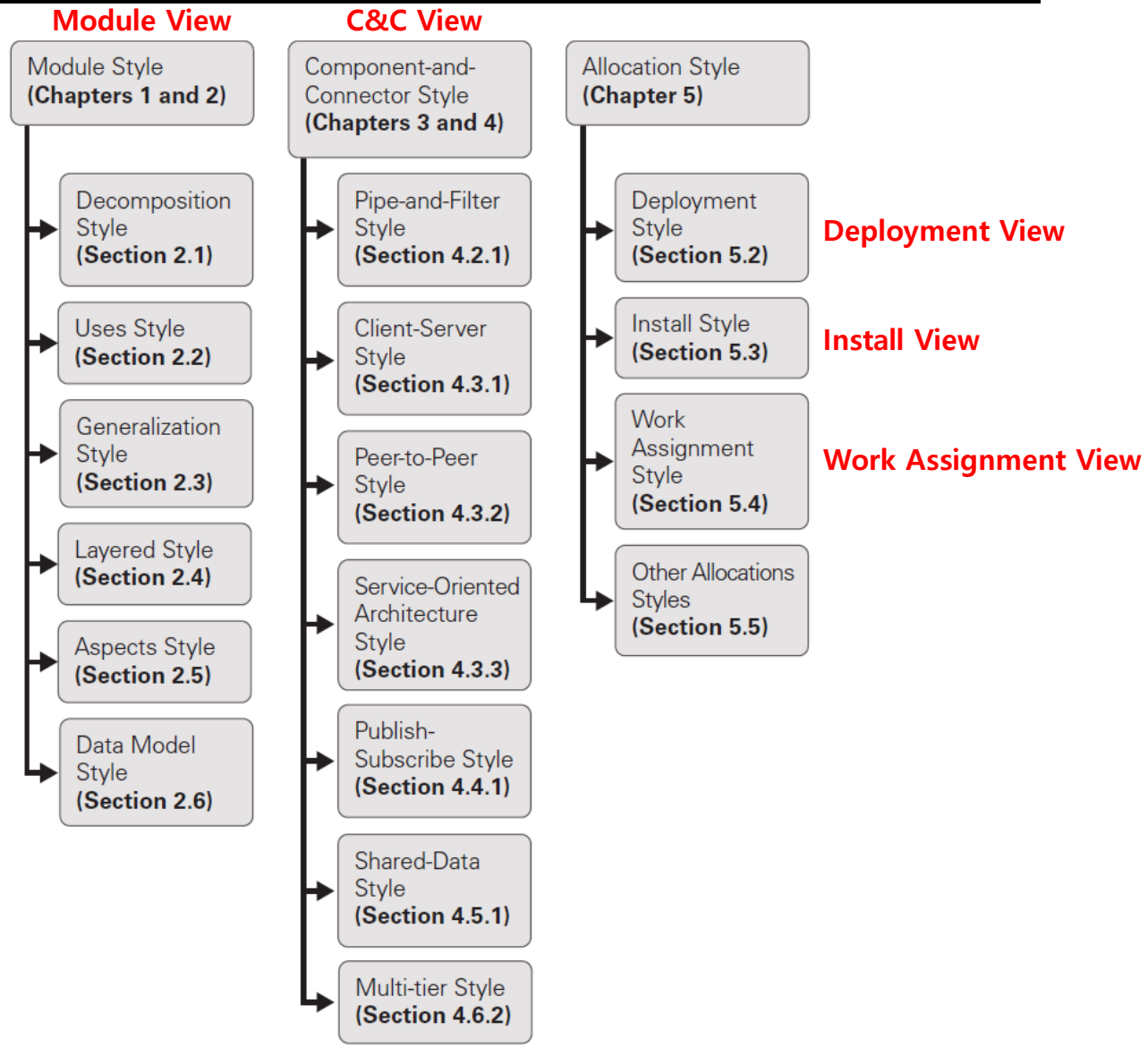
[Deployment View]



[Work Assignment View]

SW Architecture - Views and Beyond





Example 1.

https://github.com/bosornd/example-module-design

- The program starts in TU::main().
  - Objects c, m2, b, m1, a and u are created.
  - m2 is connected to c, and b is connected to m2.
  - m1 is connected to b, and a is connected to m1.
  - Thread objects m1 and m2 are started.

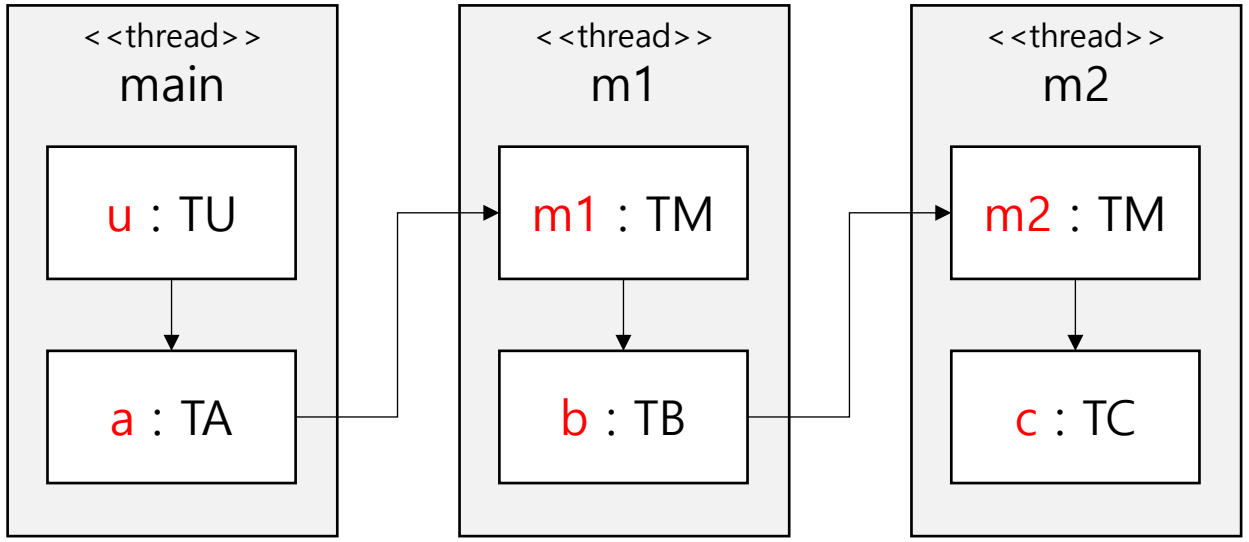
  - u’s process is requested.
  - u gets a number from the console and requests a's process.
  - a prints the result and requests m1 to do the next process.
  - m1 requests b's process. b prints the result and requests m2 to do the next process.
  - m2 requests c's process. c prints the result.

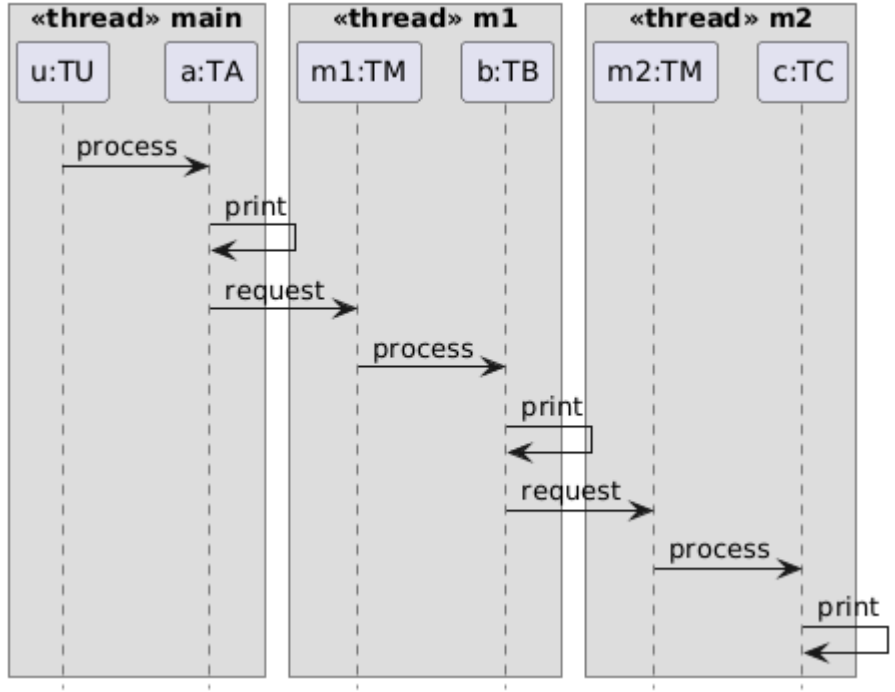
  - The main thread waits until m1 and m2 are finished.

Example 1.

https://github.com/bosornd/example-module-design

- The program starts in TU::main().
  - Objects c, m2, b, m1, a and u are created.
  - m2 is connected to c, and b is connected to m2.
  - m1 is connected to b, and a is connected to m1.
  - Thread objects m1 and m2 are started.



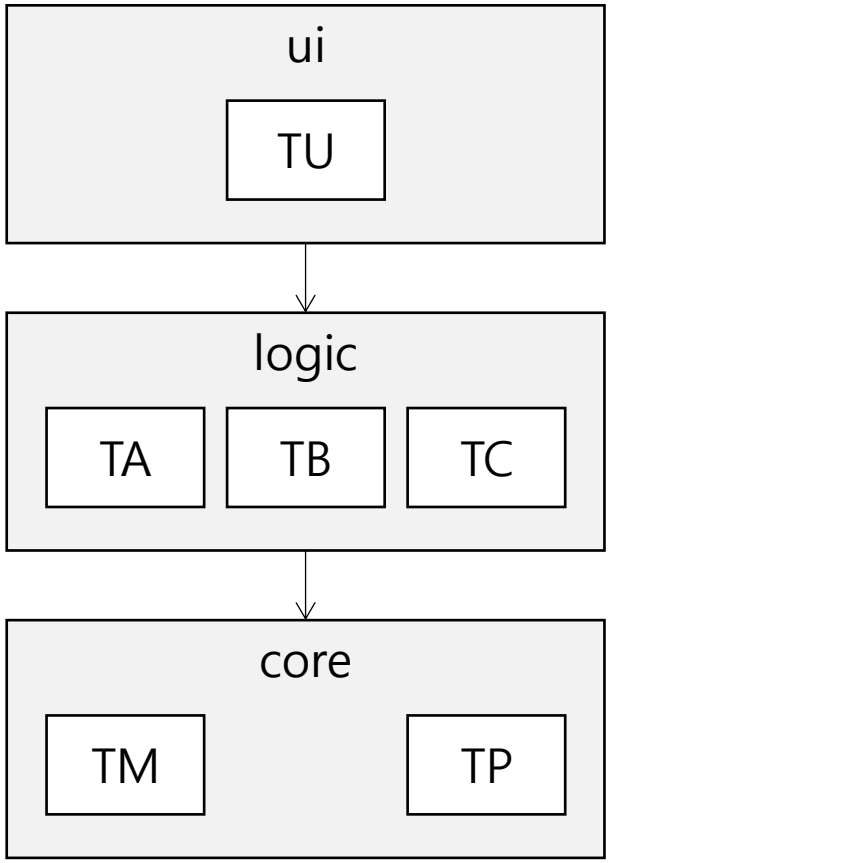


connector (function call)

Example 1.

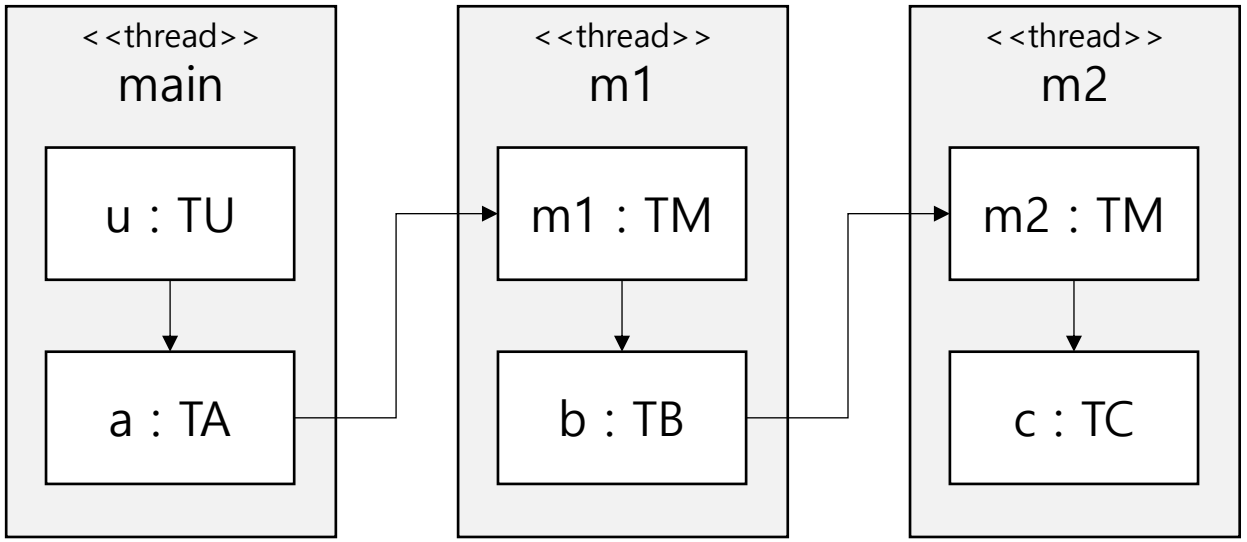
https://github.com/bosornd/example-module-design

- System consists of 3 packages - ui, logic, core.
  - The ui package contains TU classes.
  - The logic package contains TA, TB and TC classes.
  - The core package contains TM and TP classes.
  - TU uses (depends on) TA, TB, TC and TM.
  - TA, TB and TC extends (depends on) TP.
  - TA and TB uses (depends on) TM.
  - TM uses (depends on) TP.

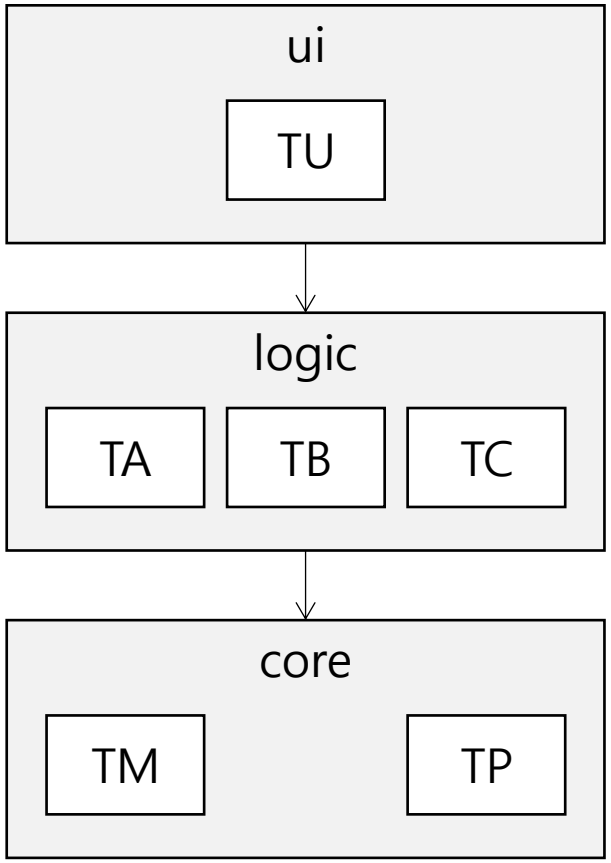


Example 1.

https://github.com/bosornd/example-module-design



Multi-threaded Architecture

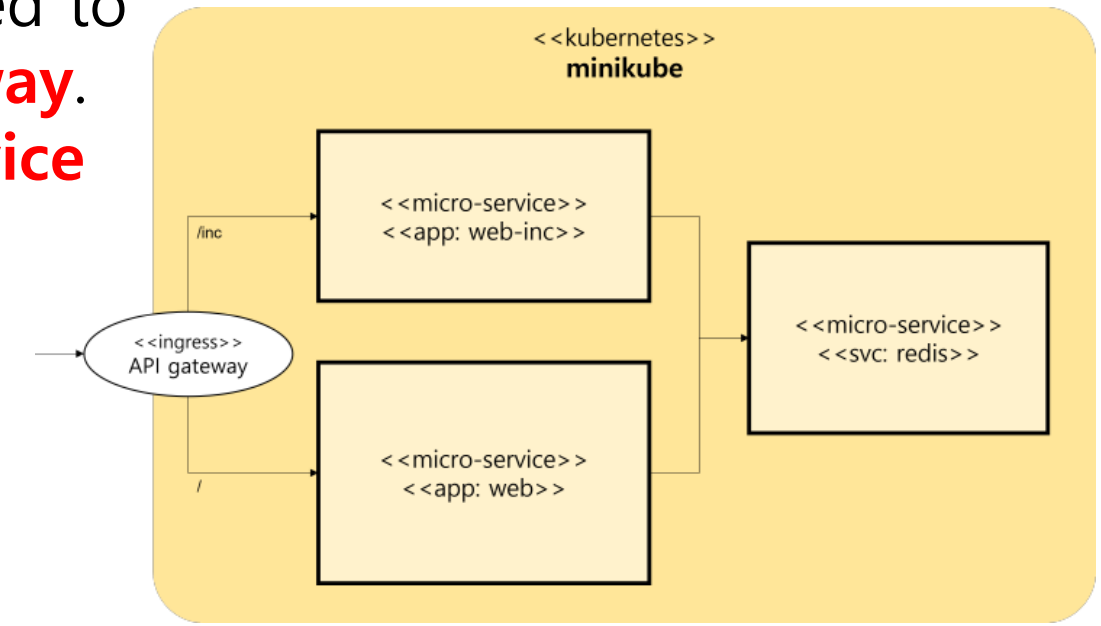


Layered Architecture

Example 2.

https://github.com/bosornd/k8s-example
https://k8s.bosornd.com/kubernetes/kubernetes-and-beyond

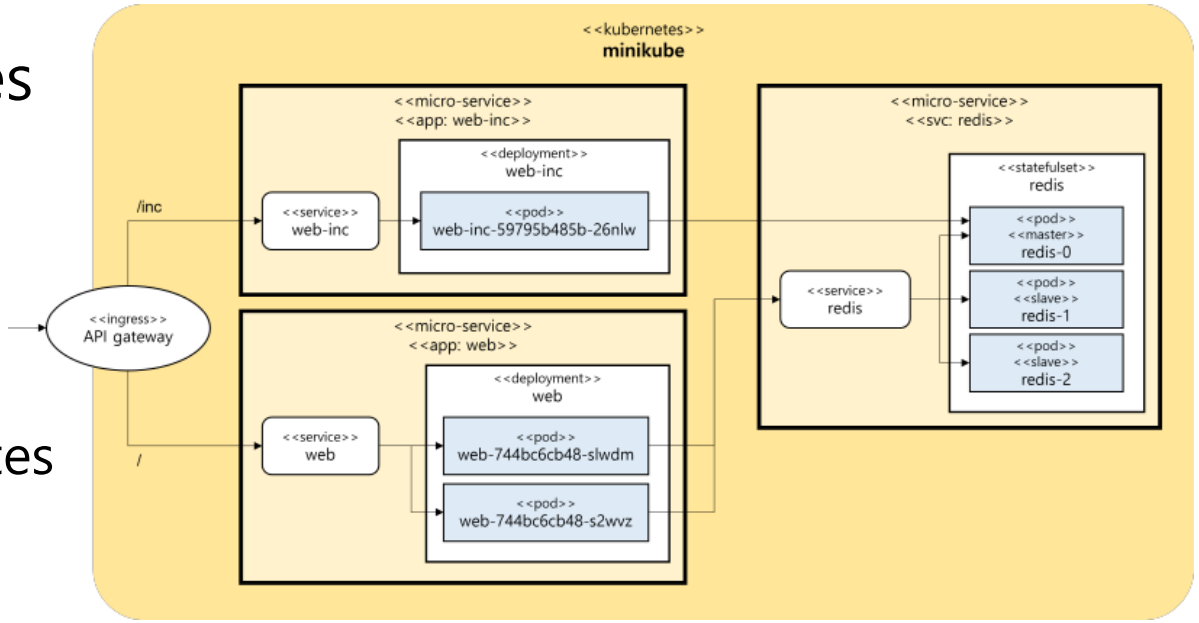
- The system consists of 3 microservices – web, web-inc and redis.
- Get requests (/) are forwarded to **web service** through **API gateway**. **web service** gets the current count value stored in **redis service** and forwards it.
- Increase requests (/inc) are forwarded to **web-inc service** through **API gateway**. **web-inc service** requests **redis service** to increase the current count value.



Example 2.

https://github.com/bosornd/k8s-example
https://k8s.bosornd.com/kubernetes/kubernetes-and-beyond

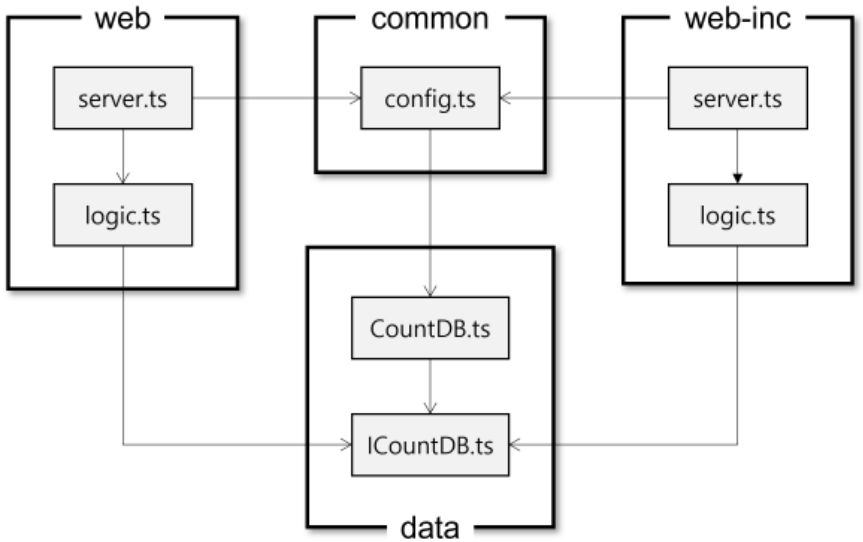
- web microservice consists of 2 pods managed by the Kubernetes deployment controller. Kubernetes service web distributes the request across 2 pods as load balancer.
- web-inc microservice is almost same as web microservice.
- redis microservice consists of 3 pods managed by the Kubernetes statefulset controller. redis-0 pod acts as a master and the other pods act as slaves.
  - Only redis-0 handles update(increase) request. redis-0 propagates the updates to the other slaves.

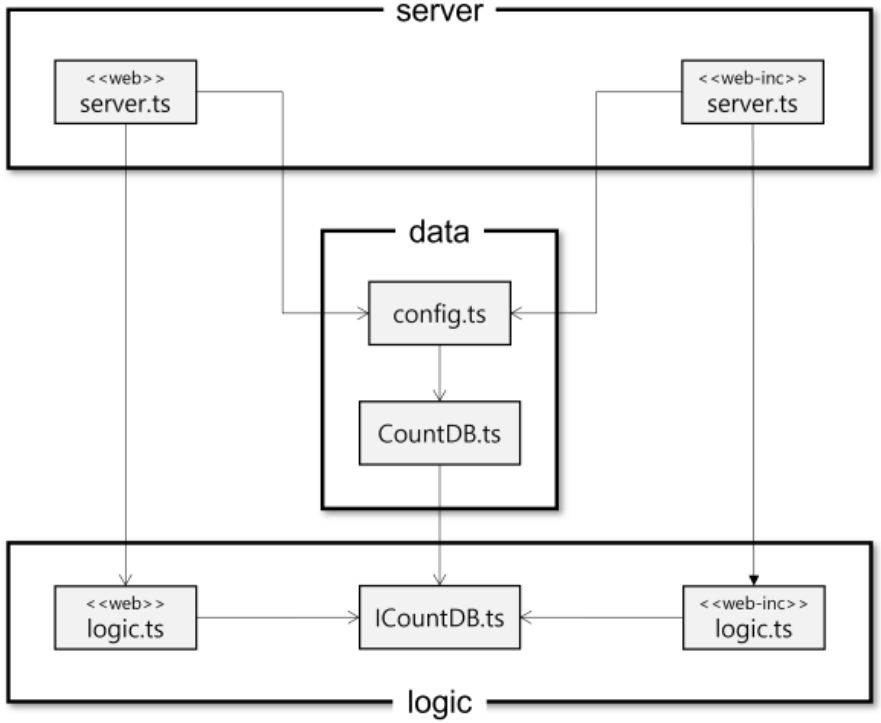


Example 2.

https://github.com/bosornd/k8s-example
https://k8s.bosornd.com/kubernetes/kubernetes-and-beyond

- System consists of 4 packages which are divided into folders.
  - ICountDB.ts in data package provides interface to access redis DB.
  - logic.ts in web and web-inc package uses (depends on) ICountDB.ts.
  - config.ts provides CountDB configuration using dependency injection framework.
  - server.ts configures CountDB using config.ts.





Role of Architect

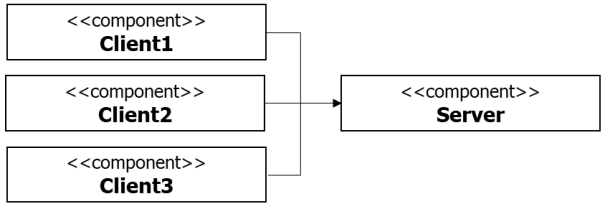
- Visualize S/W architecture



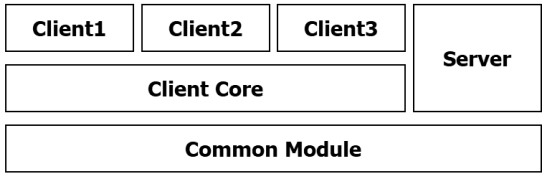
Understand → Collaborate

Imagine → Visualize/Document → Communicate → Evaluate/Analyze → Optimize

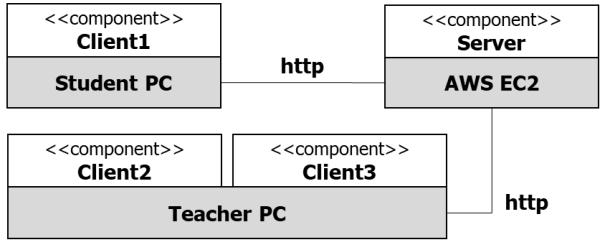
Architecture



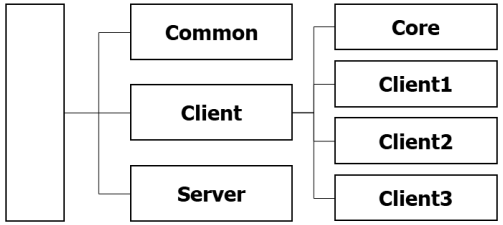
[Component & Connector View]



[Module View]



[Deployment View]



[Work Assignment View]

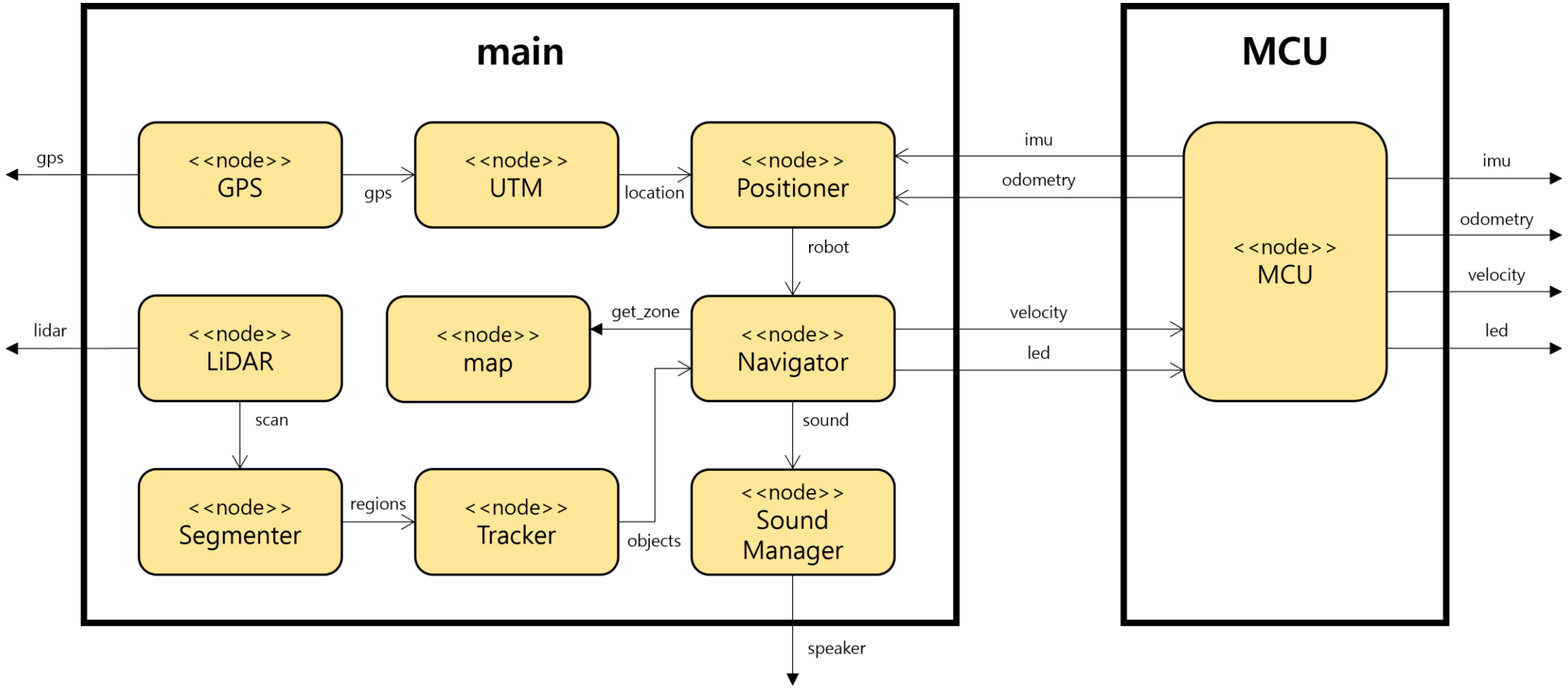
[Install View]

[Data View]

실습1. 구조 명세(동작)

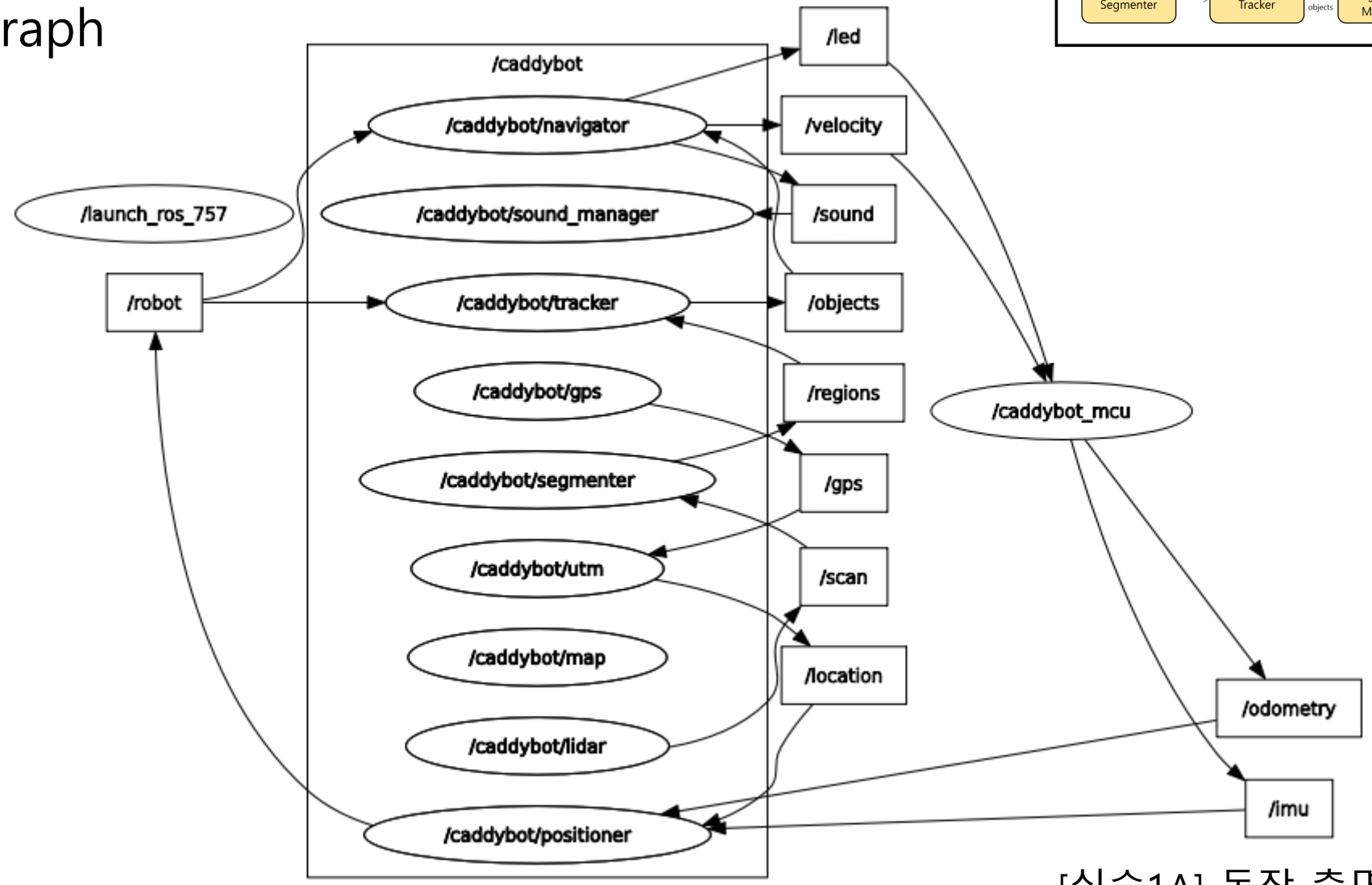
cc BY NC ND

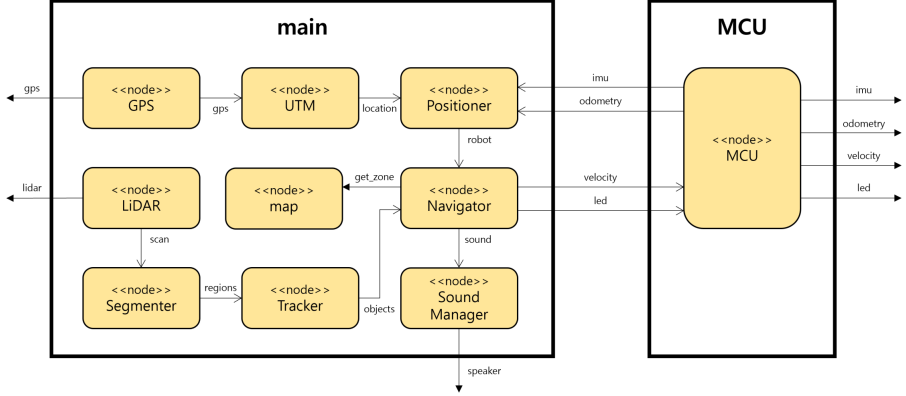
동작 측면의 구조


main
MCU
gps
<<node>>
GPS
gps
<<node>>
UTM
location
<<node>>
Positioner
robot
imu
odometry
<<node>>
MCU
imu
odometry
velocity
led
lidar
<<node>>
LiDAR
<<node>>
map
get_zone
<<node>>
Navigator
velocity
led
scan
sound
<<node>>
Segmenter
regions
<<node>>
Tracker
objects
<<node>>
Sound
Manager
speaker

동작 측면의 구조

- rqt_graph

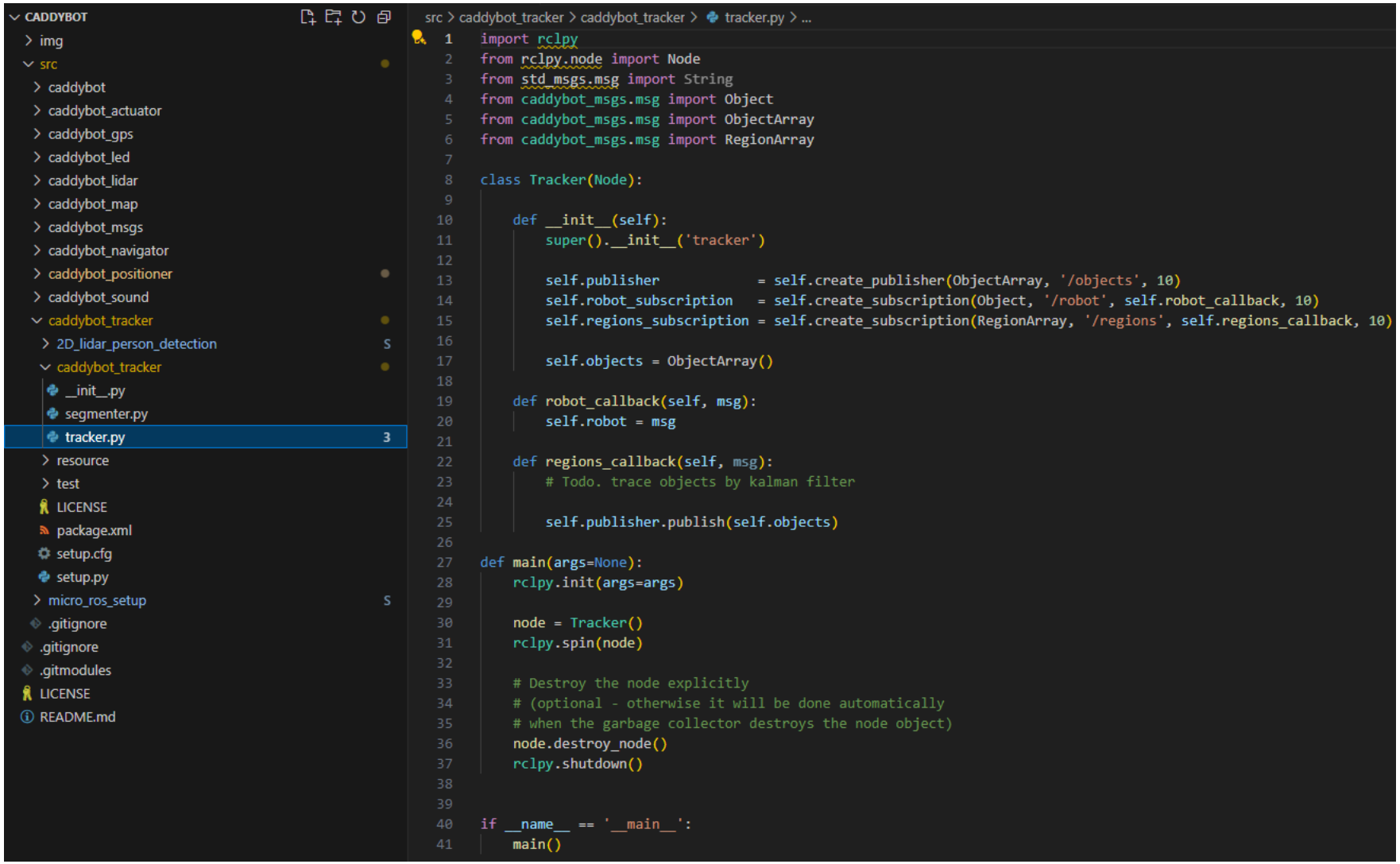




[실습1A] 동작 측면의 구조를 그려보자.

caddybot

https://github.com/bosornd/caddybot




```python
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from caddybot_msgs.msg import Object
from caddybot_msgs.msg import ObjectArray
from caddybot_msgs.msg import RegionArray

class Tracker(Node):

    def __init__(self):
        super().__init__('tracker')

        self.publisher           = self.create_publisher(ObjectArray, '/objects', 10)
        self.robot_subscription  = self.create_subscription(Object, '/robot', self.robot_callback, 10)
        self.regions_subscription = self.create_subscription(RegionArray, '/regions', self.regions_callback, 10)

        self.objects = ObjectArray()

    def robot_callback(self, msg):
        self.robot = msg

    def regions_callback(self, msg):
        # Todo. trace objects by kalman filter

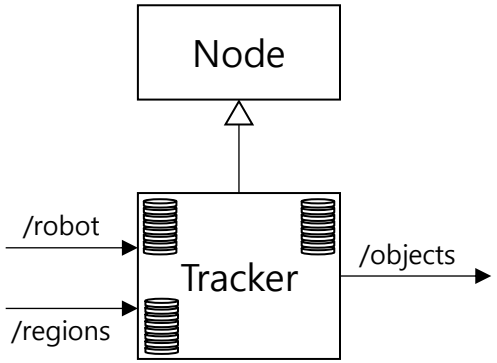
        self.publisher.publish(self.objects)

def main(args=None):
    rclpy.init(args=args)

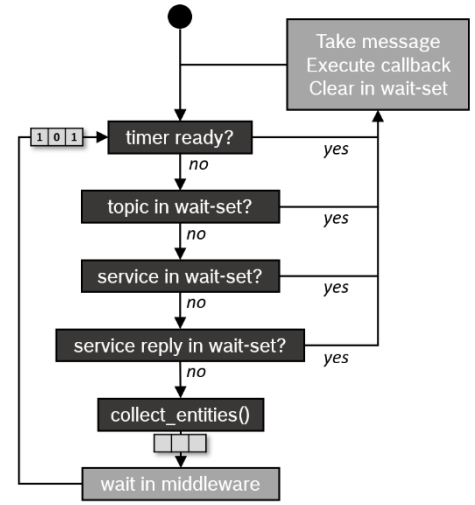
    node = Tracker()
    rclpy.spin(node)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

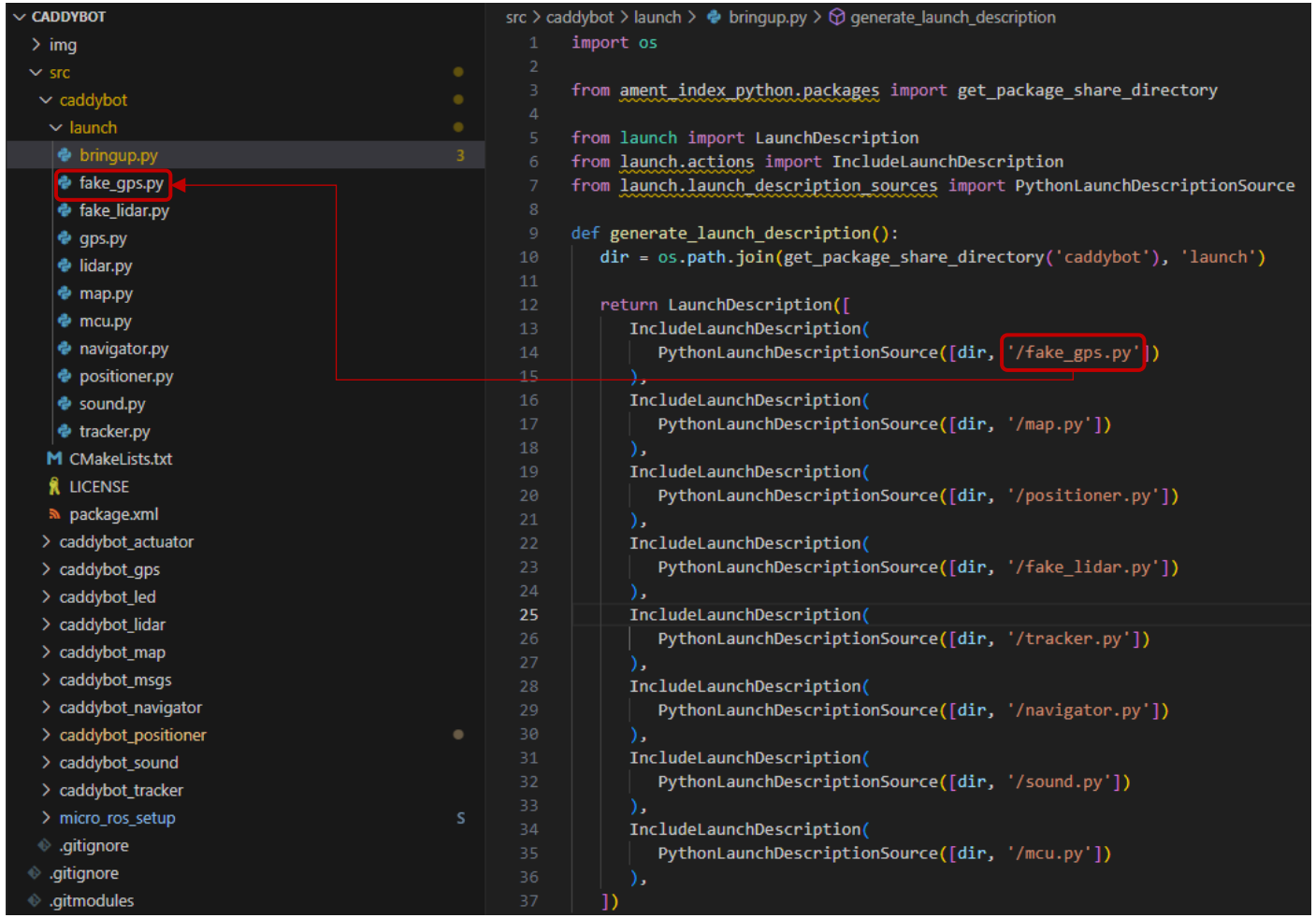


Executer



caddybot

https://github.com/bosornd/caddybot



fake_gps.py

```python
from launch import LaunchDescription
from launch_ros.actions import LifecycleNode

def generate_launch_description():
    return LaunchDescription([
        LifecycleNode(
            package='caddybot_gps',
            namespace='caddybot',
            executable='fake_gps',
            name='gps',
        ),
        LifecycleNode(
            package='caddybot_gps',
            namespace='caddybot',
            executable='utm',
            name='utm',
        ),
    ])
```

배치 구조

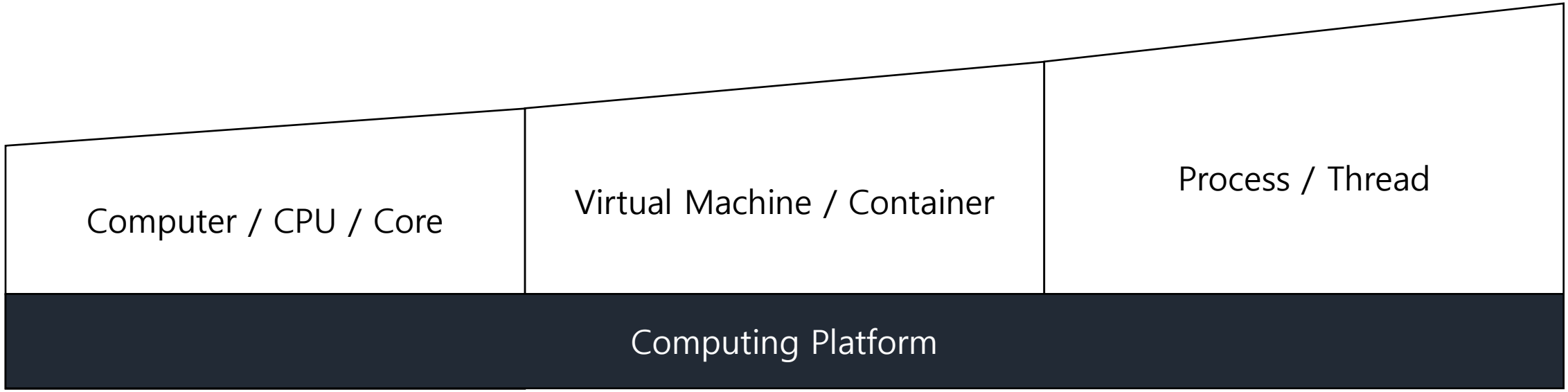
cc BY NC ND

Deployment View

Overview	• The deployment view describes the mapping of components and connectors in the software architecture to the hardware of the computing platform.
Elements	• **Software element**: elements from a C&C view. Useful properties to document include the significant features required from hardware, such as processing, memory, capacity requirements, and fault tolerance. • **Environmental element**: hardware of the computing platform - processor, memory, disk, network(such as router, bandwidth, firewall, bridge), and so on. Useful properties of an environmental element are the significant hardware aspects that influence the allocation decision.
Relations	• **Allocated-to**. Physical units on which the software elements reside during execution. Properties include whether the allocation can change at execution time or not. • **Migrates-to, copy-migrates-to, and/or execution-migrates-to** if the allocation is dynamic. Properties include the trigger that causes the migration.
Constraints	• The allocation topology is unrestricted. However, the required properties of the software must be satisfied by the provided properties of the hardware.
Usage	• For reasoning about performance, availability, security and safety.

Deployment Unit

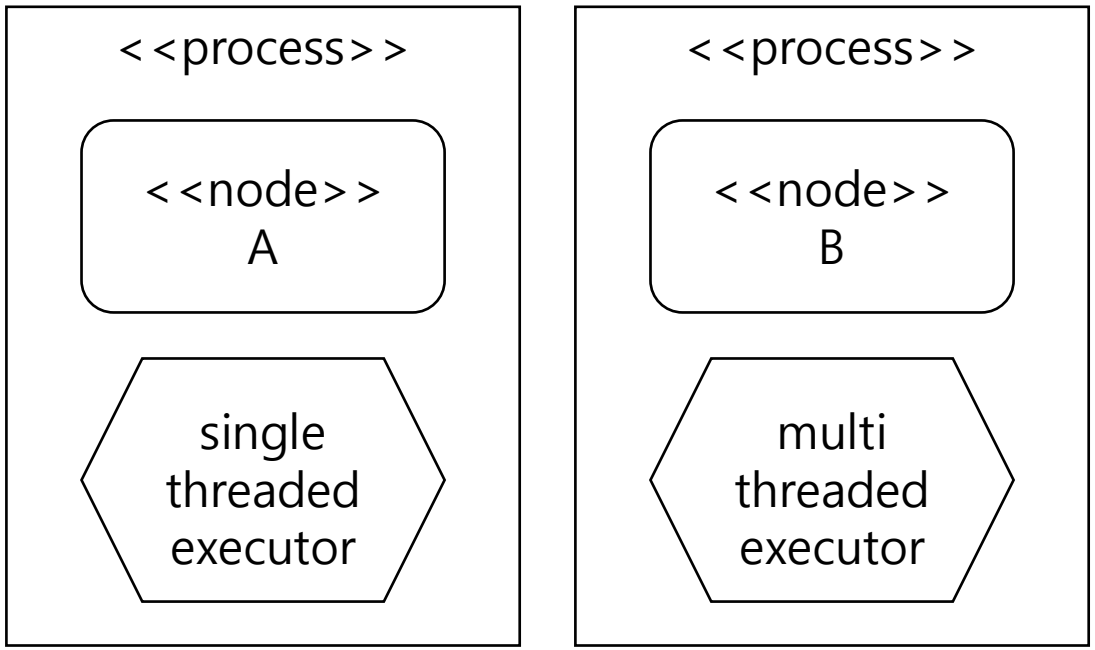
- Physical unit on which the software elements reside during execution.
- Computing unit that executes the software elements concurrently.

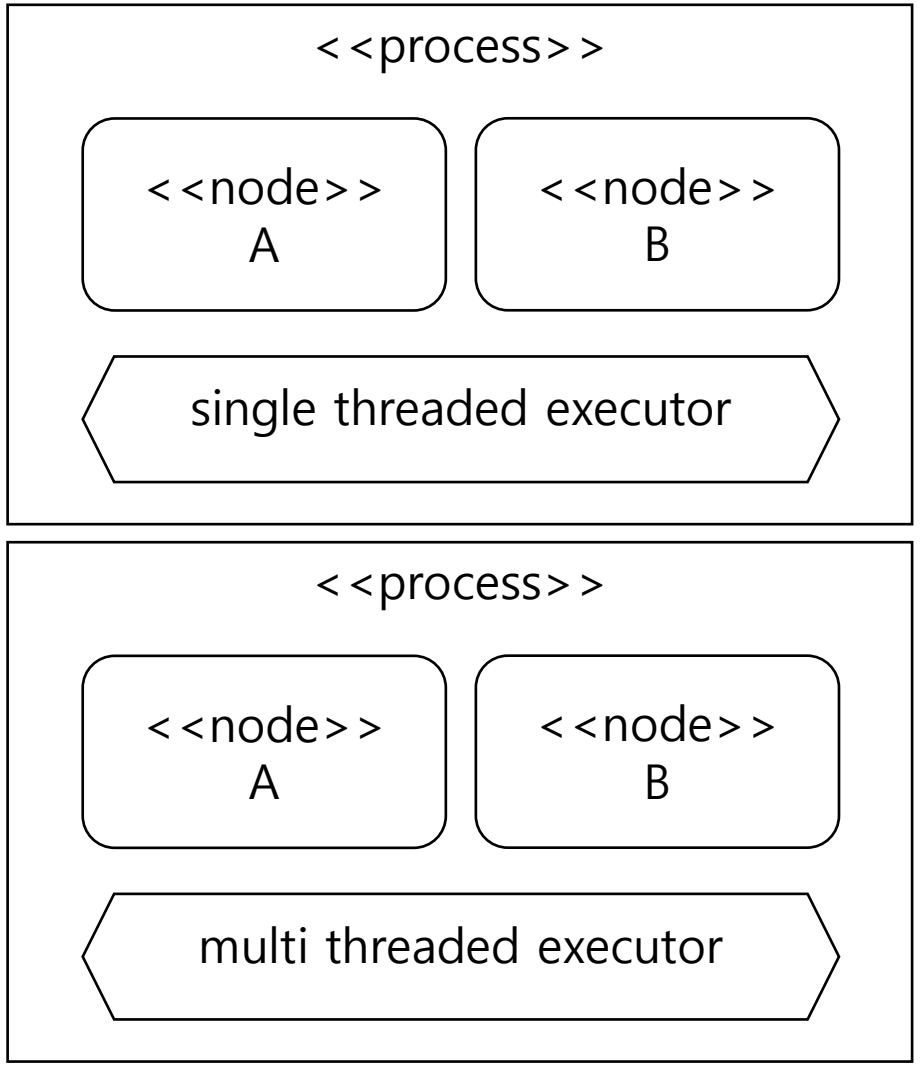


Deployment Unit – ROS

https://docs.ros.org/en/humble/Concepts/Intermediate/About-Executors.html

- single node vs. multi node process
- single threaded vs. multi threaded executor





Deployment Unit – ROS

https://github.com/bosornd/ros2_examples

[실습1B] 배치 구조를 그려보자.

- single_node_process
- multi_node_single_threaded_process
- multi_node_multi_threaded_process

Deployment Unit – ROS

https://github.com/bosornd/ros2_examples

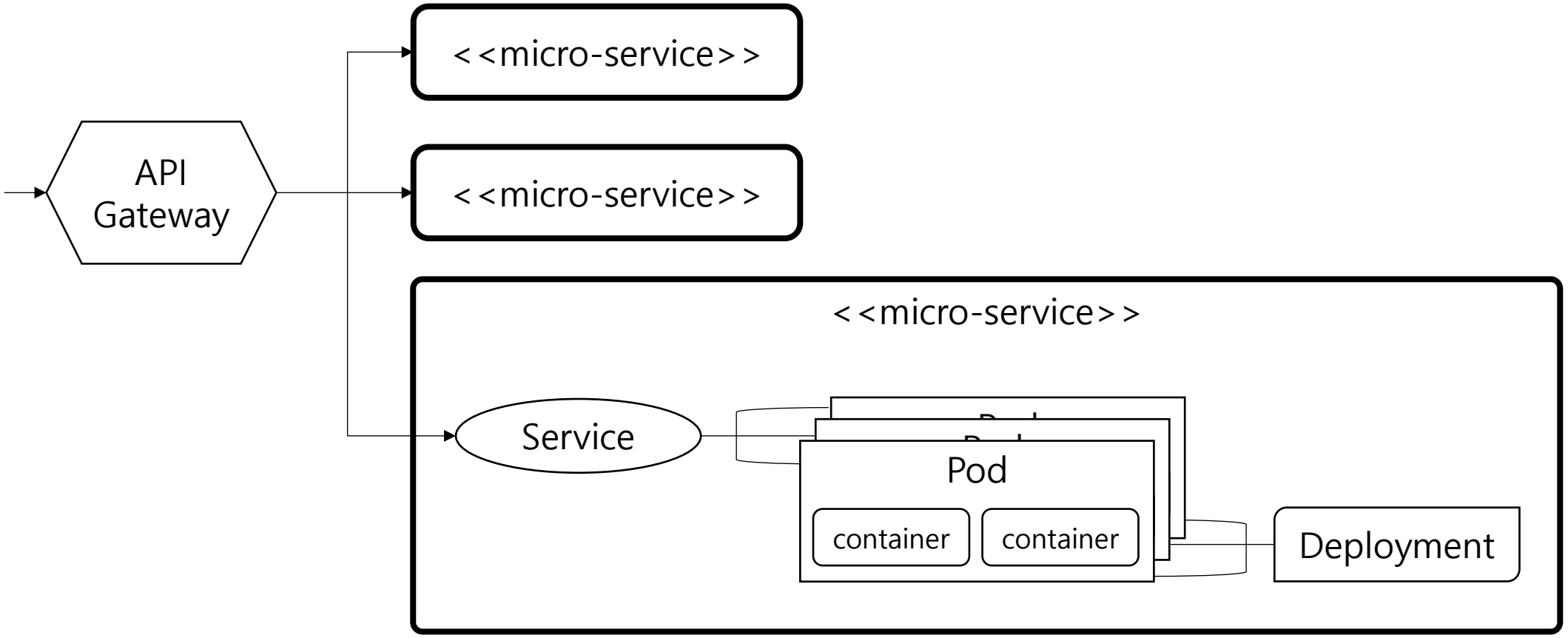
[실습1B] 배치 구조를 그려보자.

- single_node_process
- multi_node_single_threaded_process
- multi_node_multi_threaded_process

[실습1C] 실행 결과에 대해 분석해 보자.

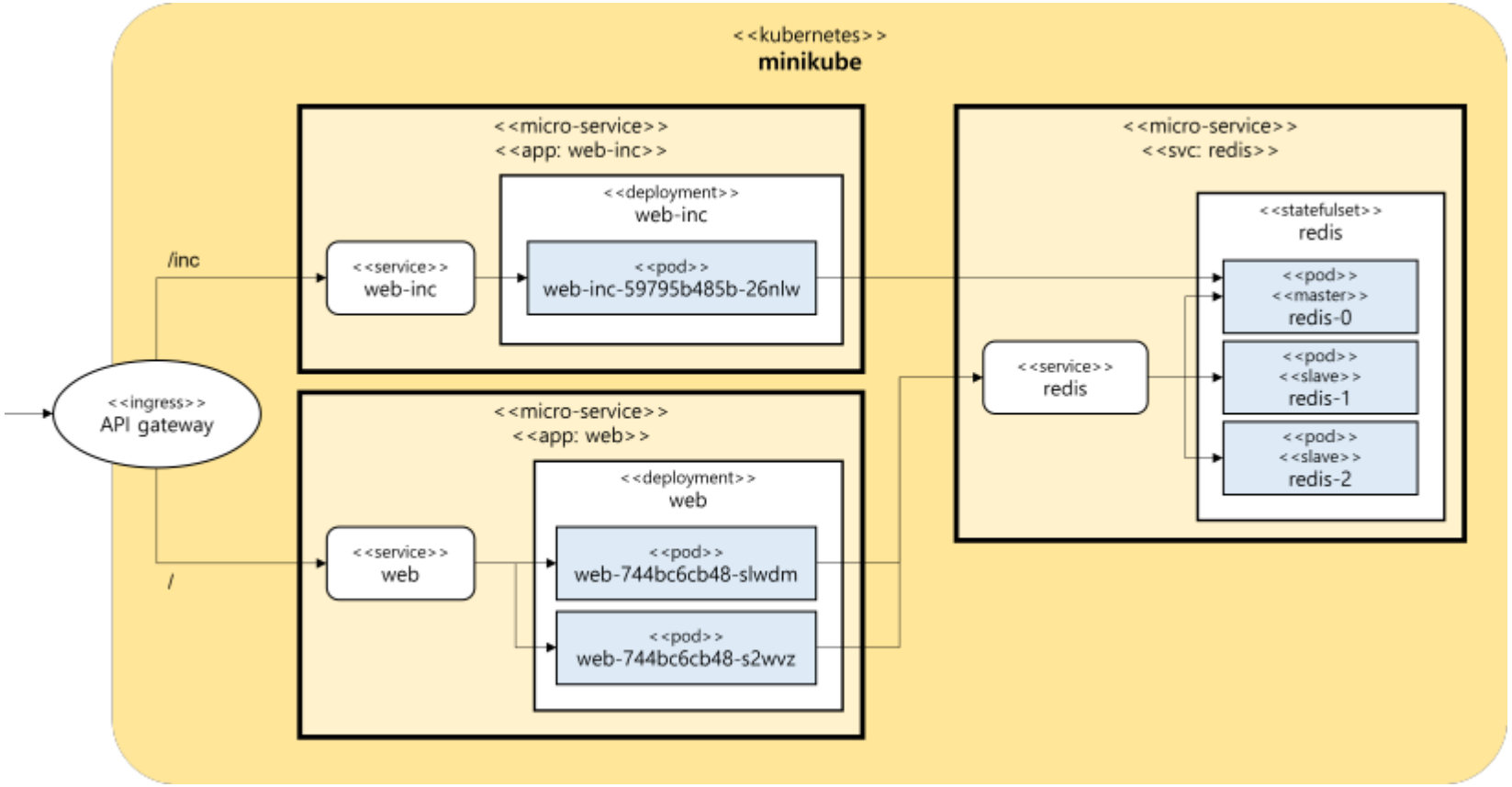
(multi_node_multi_threaded_process가 느린 이유는?)

Deployment Unit – Kubernetes / MSA



Deployment Unit – Kubernetes / MSA

https://github.com/bosornd/k8s-example



실습2. 구조 명세(개발)

cc BY NC ND

Module View

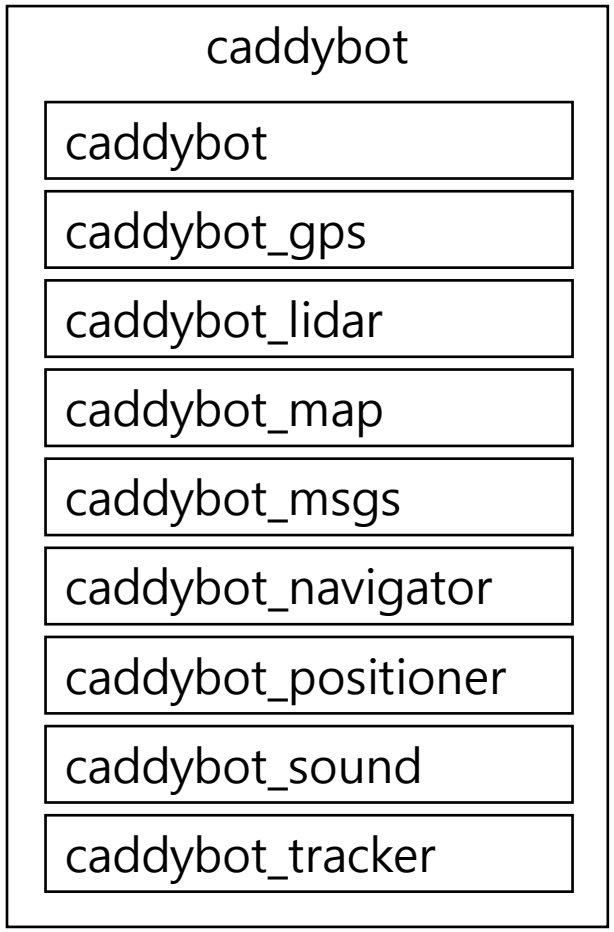
Elements	• **Modules**, which are implementation units of software.
Relations	• **Is part of**, which defines a part/whole relationship between the submodule the part-and the aggregate module the whole. • **Depends on**, which defines a dependency relationship between two modules. Specific module views elaborate what dependency is meant. • **Is a**, which defines a generalization/specialization relationship between a more specific module and a more general module.
Constraints	• Different module views may impose specific topological constraints, such as limitations on the visibility between modules.
Usage	• **Blueprint for construction of the code** • **Change-impact analysis** • Planning incremental development • Requirements traceability analysis • Communicating the functionality of a system and the structure of its code base • **Supporting the definition of work assignments, implementation schedules, and budget information** • Showing the structure of information that the system needs to manage

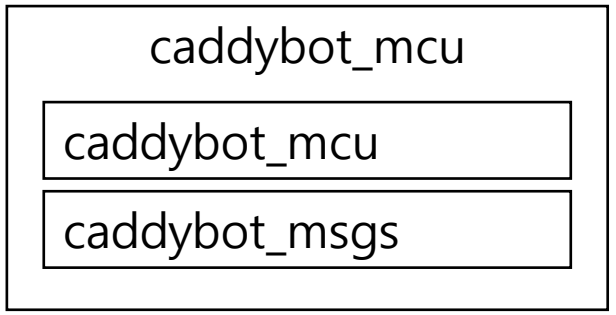
Work Assignment View

Overview	• The work assignment view describes the mapping of the software architecture to the teams in the development organization.
Elements	• **Software element**: a module. Properties include the required skill set and available capacity (effort, time) needed. • **Environmental element**: an organizational unit, such as a person, a team, a department, a subcontractor, and so on. Properties include the provided skill set and the capacity in terms of labor and calendar time available.
Relations	• **Allocated-to**. A software element is allocated to an organizational unit.
Constraints	• In general, the allocation is unrestricted; in practice, it is usually restricted so that one module is allocated to one organizational unit.
Usage	• For reasoning about distributed development and allocation of work to teams.

caddybot Packages

- ros2 pkg list | grep caddybot





[실습2A] 모듈 뷰를 그려 보자.

모듈 간의 의존성

Source	Kind	Target	Description
type	extends	type	the source type extends/implements the target type
type	contains	type	the source type contains a nested class whose type is the target type
type	contains	field	the source type contains the target field
method	returns	type	the source method declares a return type based on the target type
method	has param	type	the source method declares a parameter based on the target type
method	throws	type	the source method declares the target type in its throws clause
method	calls	method	the source method code invokes the target method
method	accesses	field	the source method code accesses the target field
field	is of type	type	the source field's type is based on the target type
any	references	type	any relation not covered by one of the others, e.g. • generic type parameters/bounds • parameter/return types of called methods • local variable types • types declared in catch blocks • types used in an `instanceof` operator

모듈 간의 의존성

```
public class classA implements interfaceA {
    public void sayHello(){
        System.out.println("hello!!!₩n");
    }

    classB b;
    public classB getB(){
        return b;
    }
}
```

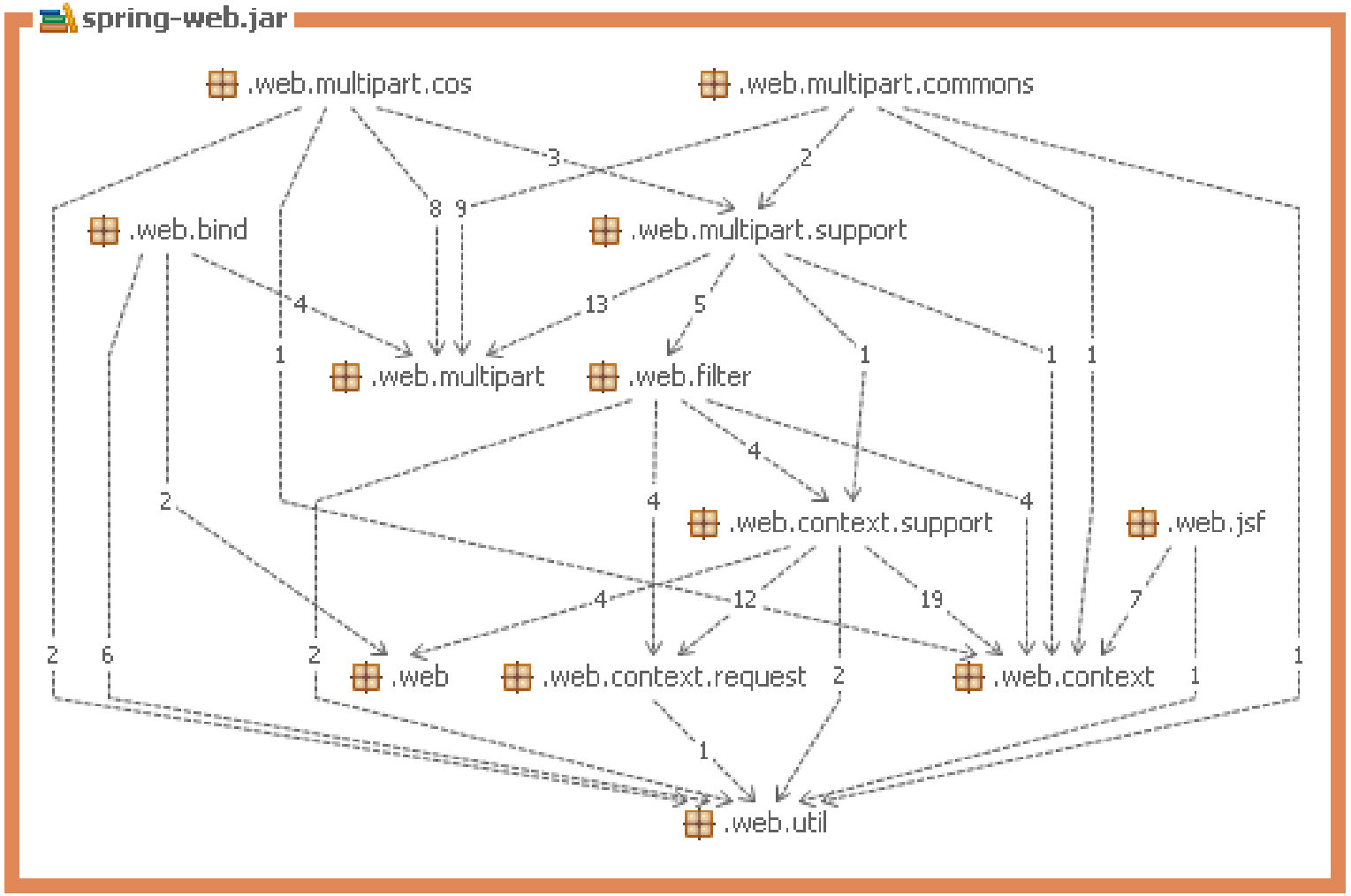
```
public interface interfaceA {
    void sayHello();
}
```

```
public class classB {
    ...
}
```

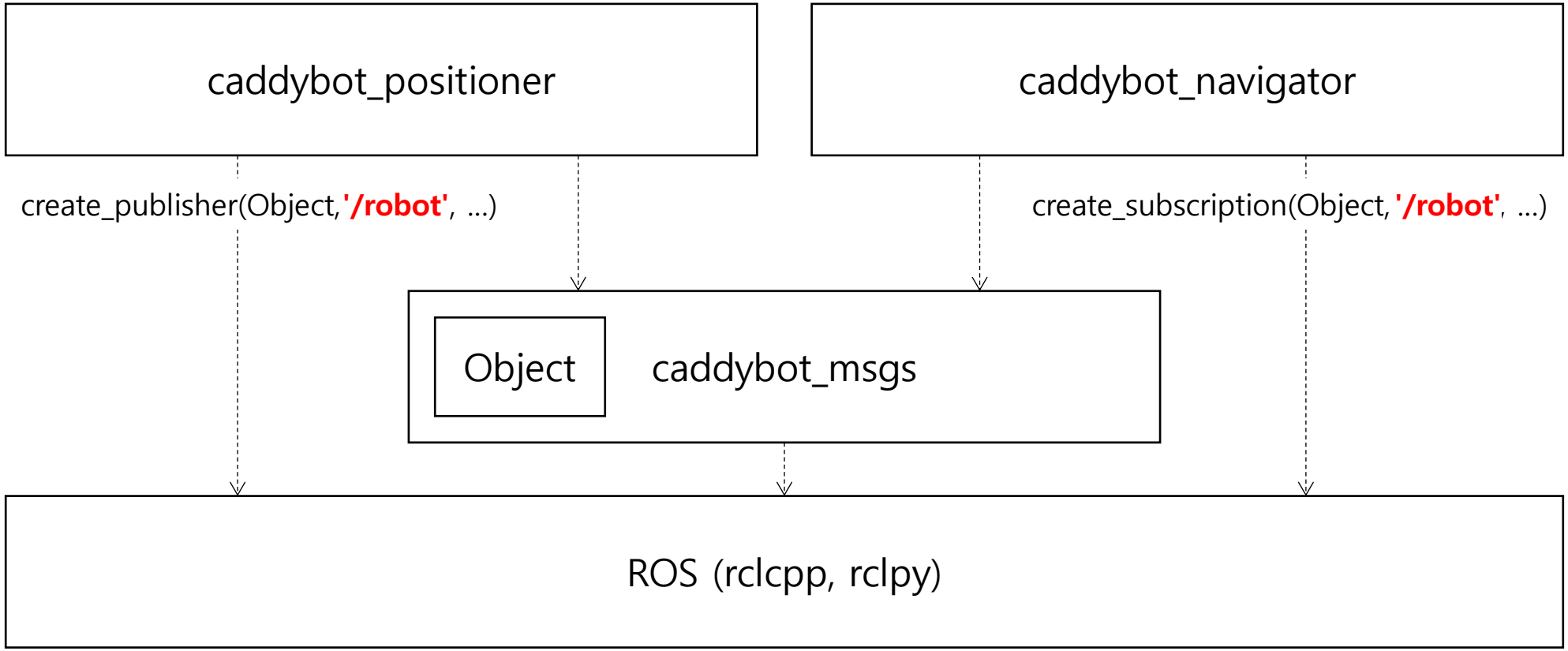
Source	Relationship	Target
classA	implements	interfaceA
classA	contains	classB
classA.b	is of type	classB
classA.getB()	returns	classB



모듈 간의 의존성


spring-web.jar
.web.multipart.cos
.web.multipart.commons
.web.bind
.web.multipart.support
.web.multipart
.web.filter
.web.context.support
.web.jsf
.web
.web.context.request
.web.context
.web.util

모듈 간의 의존성 – ROS



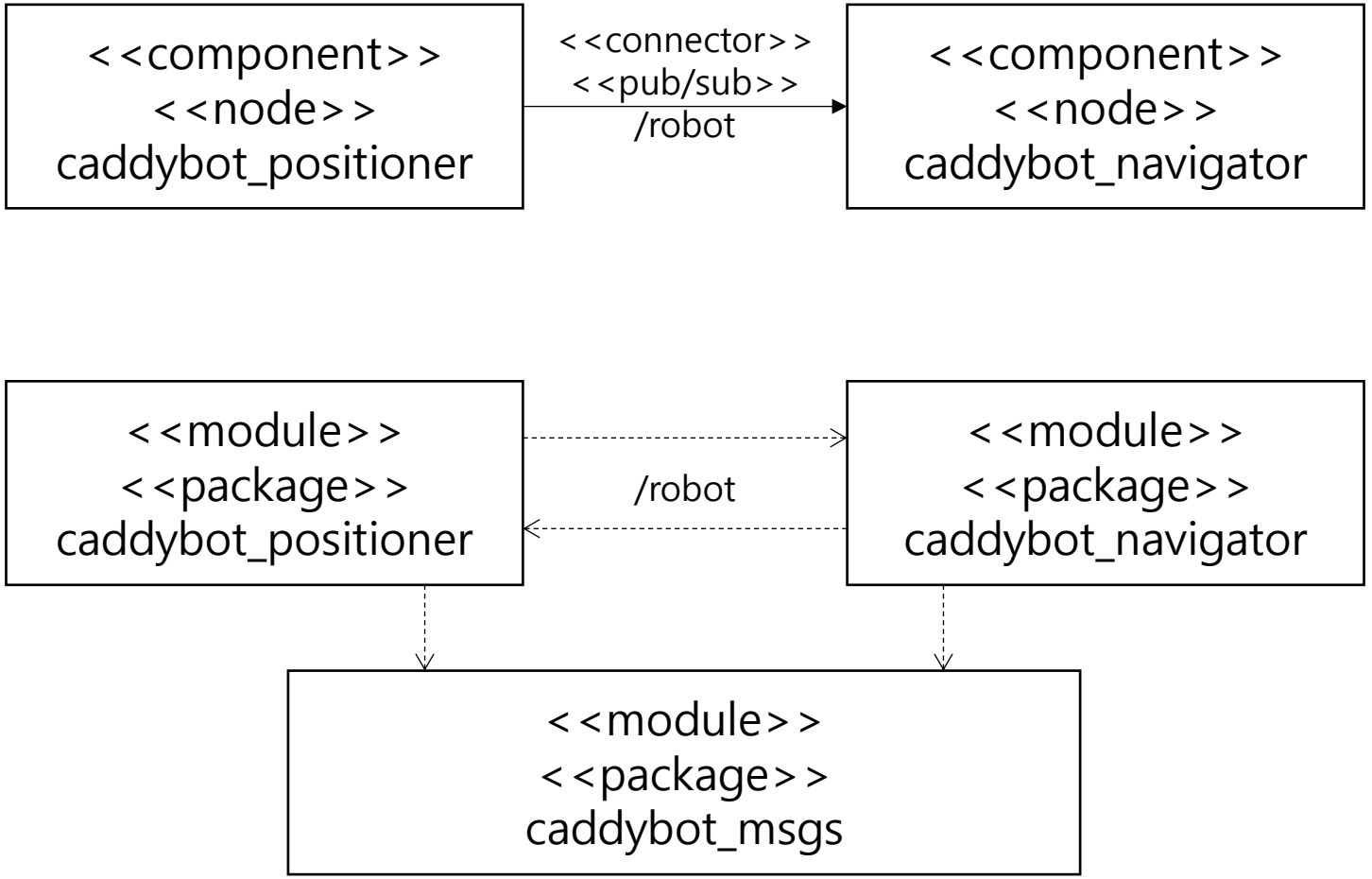
[실습2B] 모듈 뷰를 갱신해 보자.

모듈 간의 의존성 – ROS

https://github.com/bosornd/ros2_examples

[실습2C] 모듈 구조를 그려보자.

컴포넌트 vs 모듈

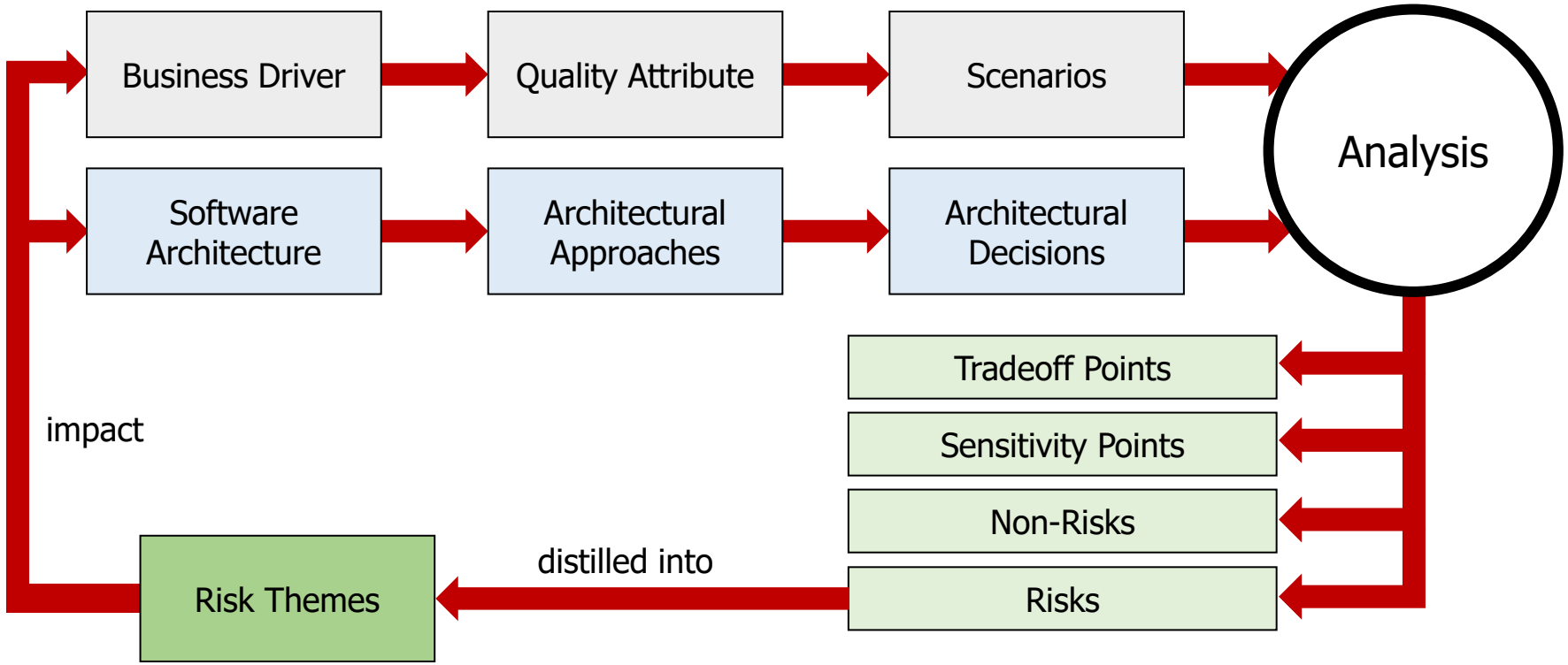

<<component>>
<<node>>
caddybot_positioner
<<connector>>
<<pub/sub>>
/robot
<<component>>
<<node>>
caddybot_navigator
<<module>>
<<package>>
caddybot_positioner
/robot
<<module>>
<<package>>
caddybot_navigator
<<module>>
<<package>>
caddybot_msgs

구조 평가

cc BY NC ND

구조 평가

- 설계된 구조가 요구되는 목적(품질)에 부합하는 가를 확인한다.
  - 구조 = **구조적 의사 결정**의 집합
  - 구조적 의사 결정이 품질에 미치는 영향으로 평가한다.



구조 평가(예)

시나리오 번호: M1.1	**시나리오:** (M1.1) 지구과학 데이터 타입 갱신과 위도/경도 박스 지원을 위해 과학 데이터 서버를 5A 버전에서 5B 버전으로 업그레이드할 때, 다른 서브시스템이나 검색, 조회, 주문 기능에 영향을 주지 않아야 한다.
속성	유지보수성
환경	정규 유지보수 작업 시
자극	과학 데이터 서버를 5A 버전에서 5B 버전으로 업그레이드 요청
반응	과학 데이터 서버 업그레이드(5A → 5B) 1. 시스템을 정지한다. 2. 과학 데이터 서버를 백업한다. 3. 과학 데이터 서버를 업그레이드(패치) 한다. 4. 과학 데이터 서버의 동작을 점검한다. 5. 시스템을 재시동 한다.
측정	[데이터 서버 업그레이드 소요 시간]

구조적 의사결정	위험요소	민감점	절충점	비위험요소
AD01. 단일 데이터 서버 운영	R1	S1	T1	
AD02. 표준 데이터 인터페이스 사용				NR1
⋮				

구조 평가(예)

구조적 의사결정	위험요소	민감점	절충점	비위험요소
AD01. 단일 데이터 서버 운영	R1	S1	T1	
AD02. 표준 데이터 인터페이스 사용				NR1
⋮				

R1. 단일 데이터 서버를 사용하므로, 데이터 서버에 영향을 미치는 변경은
전체 서비스의 비가동시간을 초래한다(반응5 – 반응1).

S1. 데이터 서버의 검증 시간(반응4)이 비가동시간(R1)에 영향을 준다.

T1. 단일 데이터 서버를 사용하므로, 유지보수성에는 위험요소가 있으나(R1) 구축비용은 저렴하다.

NR1. 표준 데이터 인터페이스를 사용하므로, 데이터 서버에 영향을 미치는 변경에 대해서
검증 시간을 단축시킬 수 있으며, 다른 서브시스템의 영향을 최소화 할 수 있다.

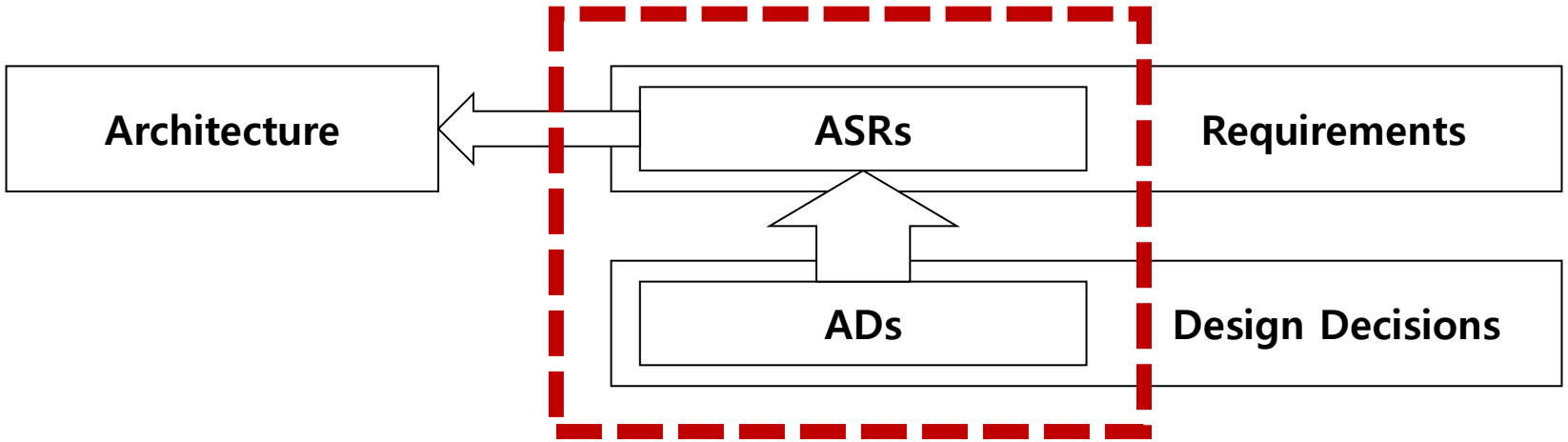
구조적 의사결정

Architectural decisions (ADs) are design decisions
that address architecturally significant requirements;
they are perceived as hard to make and/or costly to change.

https://en.wikipedia.org/wiki/Architectural_decision

Architecturally significant requirements (ASRs) are those requirements
that have a measurable effect on a software system's architecture.

https://en.wikipedia.org/wiki/Architecturally_significant_requirements

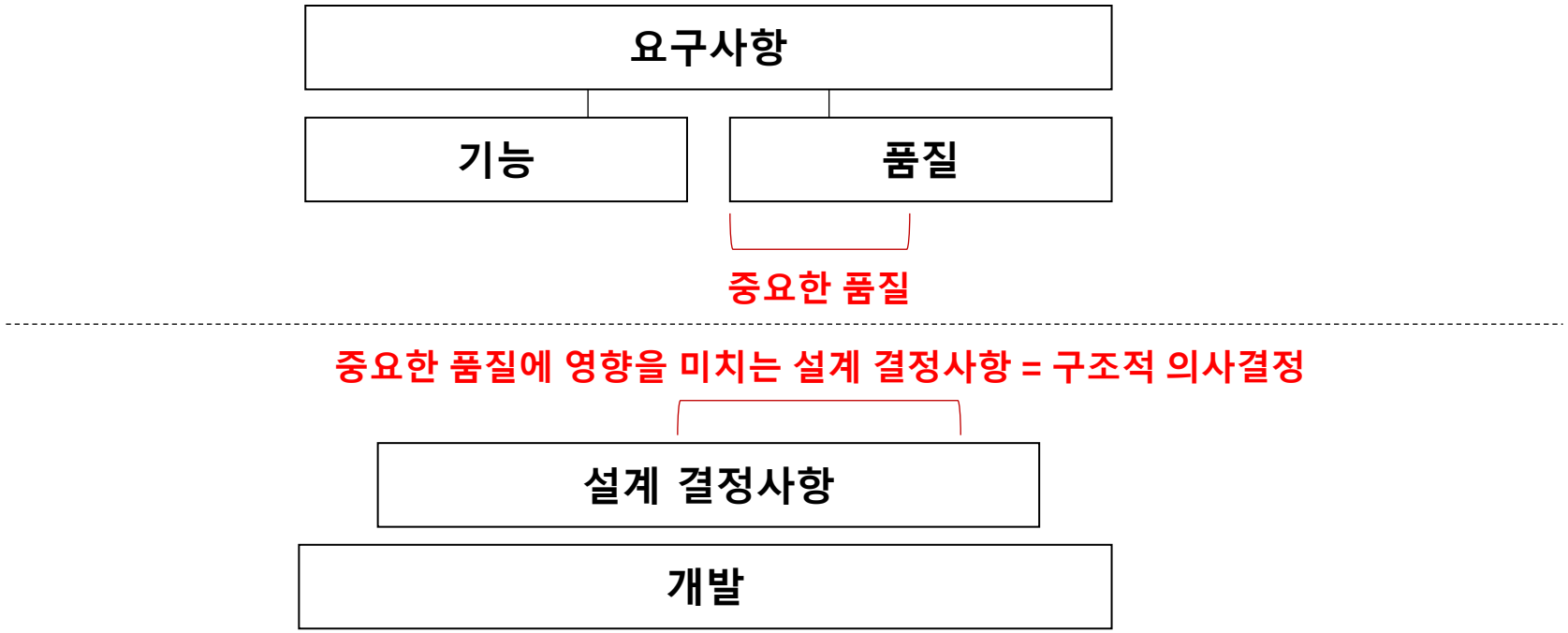


구조적 의사결정

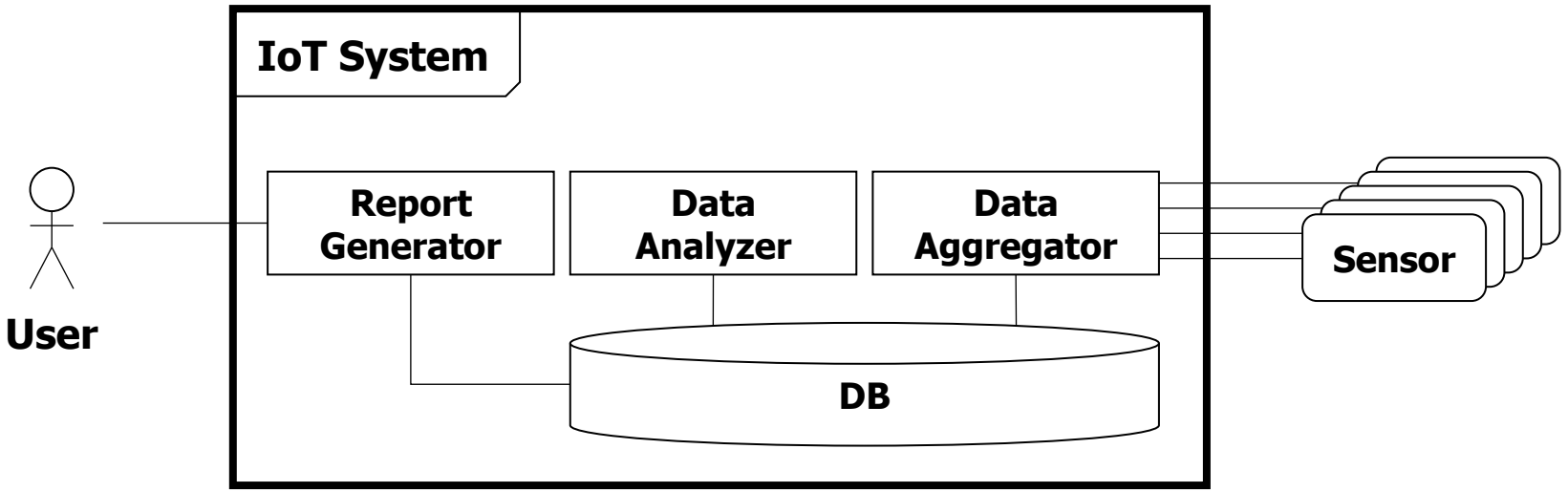
구조적 의사결정은 중요한 품질에 영향을 미치는 설계 결정사항이다.

품질 요구사항과 구조적 의사결정의 관계를 관리하는 것이 필요하다.

- 품질 요구사항이 변한다. 최적화된 설계 결정사항의 채택이 필요하다.
- Tradeoff로 RISK가 있는 품질 요구사항에 대한 완화가 필요하다.



구조 = 구조적 의사결정의 집합



	Sampling Rate	
	lower	higher
Quality 1. real-time nature of information	-	+
Quality 2. response time to user request	+	-

tradeoffs

구조 평가

- 설계된 구조가 요구되는 목적(품질)에 부합하는 가를 확인한다.
  - 구조 = **구조적 의사 결정**의 집합
  - 구조적 의사 결정이 품질에 미치는 영향으로 평가한다.

$$\boldsymbol{Architecture} = \{\boldsymbol{AD_1}, \boldsymbol{AD_2}, \dots, \boldsymbol{AD_N}\}$$

$$\boldsymbol{Quality} = \{\boldsymbol{Q_1}, \boldsymbol{Q_2}, \dots, \boldsymbol{Q_M}\}$$

$$\boldsymbol{Benefit}(\boldsymbol{Q_i}, \boldsymbol{Architecture}) = \sum_{\boldsymbol{j=1}}^{\boldsymbol{N}} \boldsymbol{Benefit}(\boldsymbol{Q_i}, \boldsymbol{AD_j})$$

구조적 의사결정이 품질에 미치는 영향으로 품질의 위험도를 측정한다.

$$\boldsymbol{Evaluation}(\boldsymbol{Architecture}) = \sum_{\boldsymbol{i=1}}^{\boldsymbol{M}} \boldsymbol{Benefit}(\boldsymbol{Q_i}, \boldsymbol{ARCH})$$

구조 설계

- 시스템이 요구되는 목적(품질)에 잘 부합하도록 설계한다.
  - 품질을 좋게 하는 구조적 의사 결정을 채택한다.
  - 채택된 **구조적 의사 결정의 집합 ➔** 구조

$$\boldsymbol{Quality} = \{\boldsymbol{Q_1}, \boldsymbol{Q_2}, \ldots, \boldsymbol{Q_M}\}$$

$$\textbf{Candidate AD} = \{\boldsymbol{CAD_1}, \boldsymbol{CAD_2}, \ldots, \boldsymbol{CAD_N}\}$$

후보 구조: 채택 가능한 구조적 의사결정

$$\boldsymbol{Evaluation}(\boldsymbol{CAD_j}) = \sum_{\boldsymbol{i=1}}^{\boldsymbol{M}} \boldsymbol{Benefit}(\boldsymbol{Q_i}, \boldsymbol{CAD_j})$$

후보 구조가 품질에 미치는 영향으로 후보 구조의 채택 여부를 결정한다.

$$\boldsymbol{ARCH} = \{\boldsymbol{SAD_1}, \boldsymbol{SAD_2}, \ldots, \boldsymbol{SAD_L}\}$$

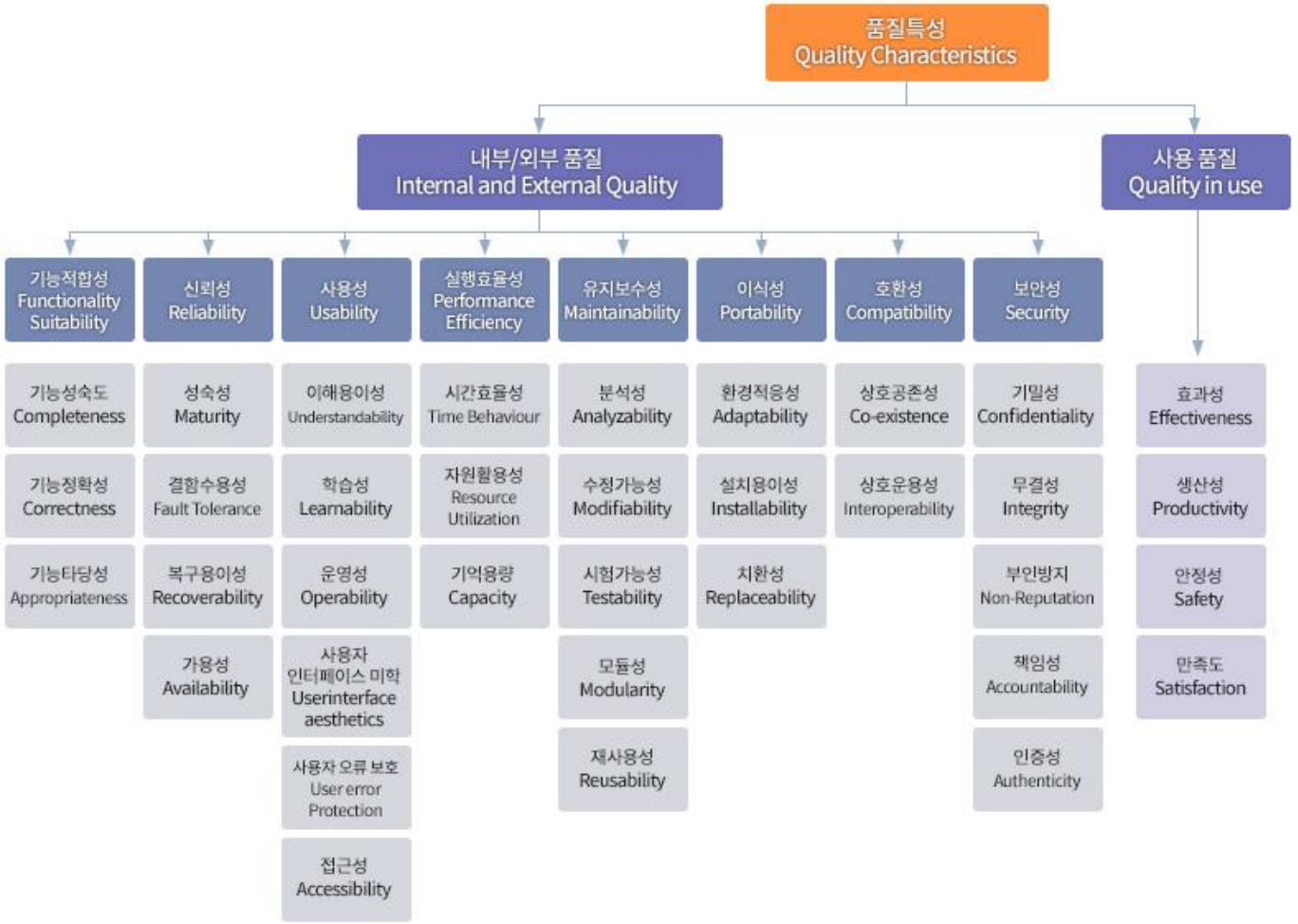
선택된 후보 구조로 구조를 설계한다.

품질 요구사항

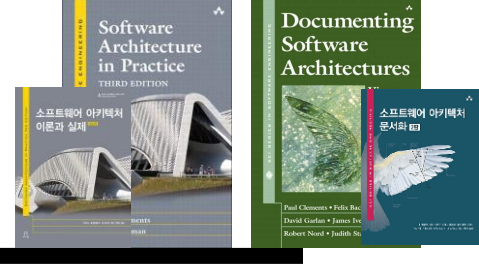
cc BY NC ND

품질 요구사항의 분류

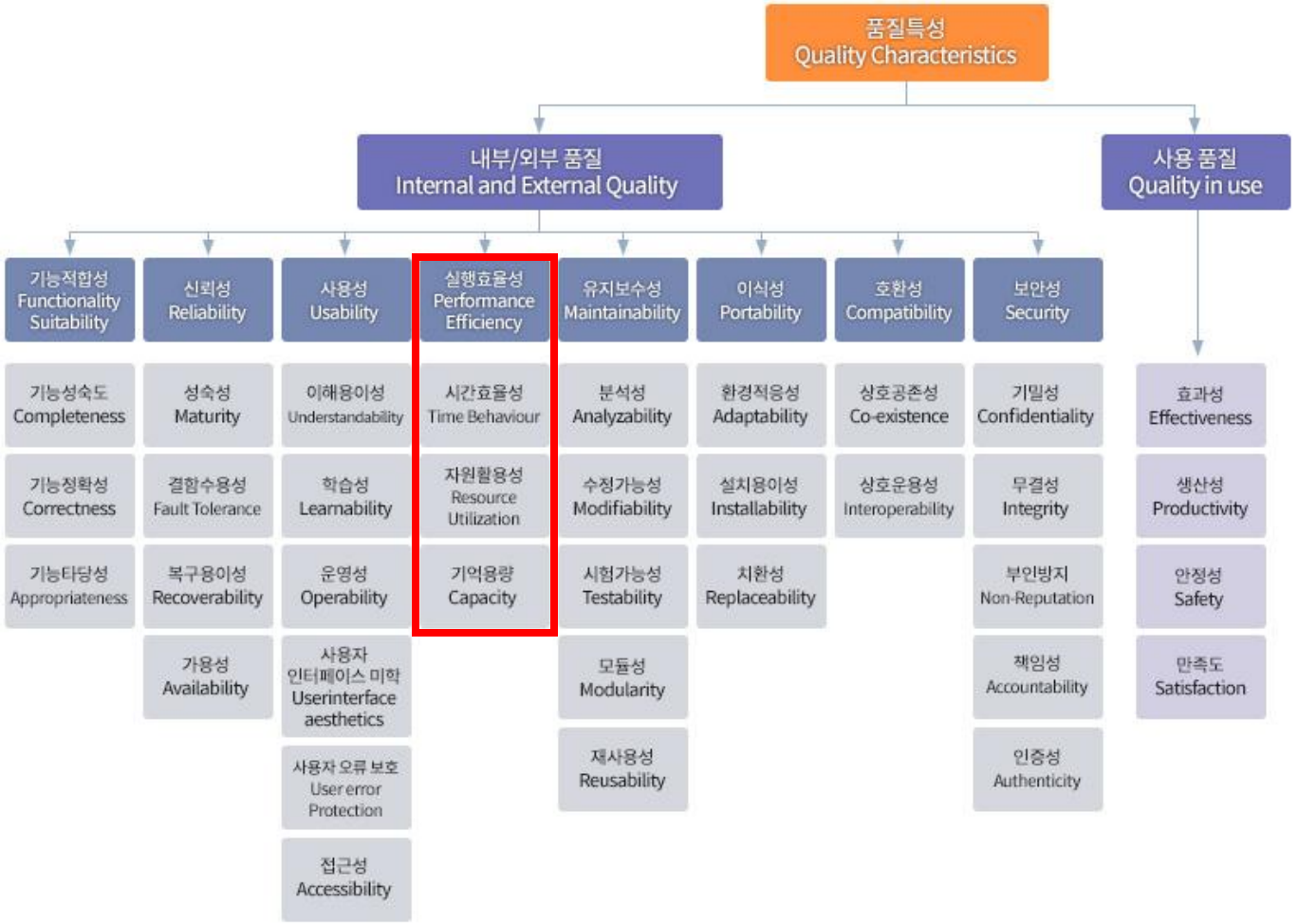
ISO/IEC 25010 - 2011



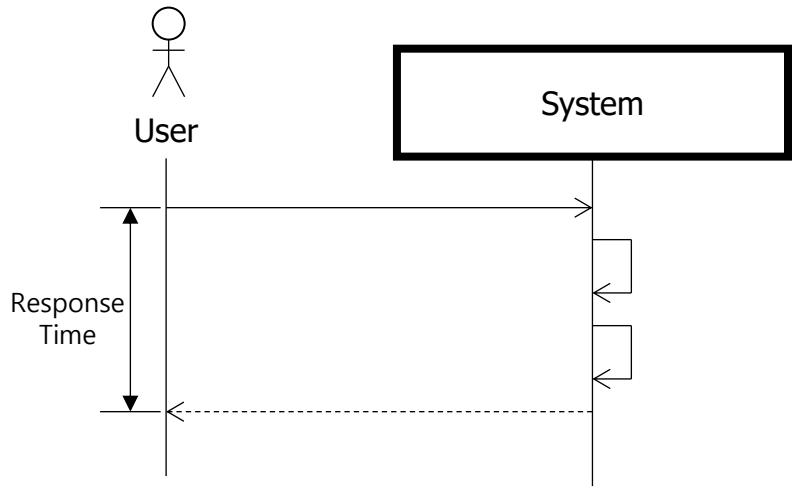
품질 요구사항의 분류



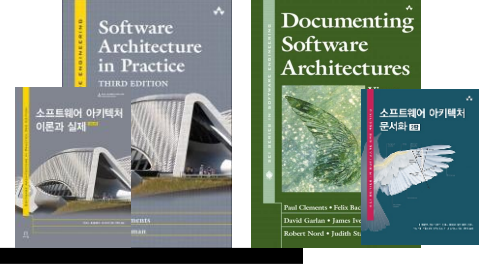
ISO/IEC 25010 - 2011



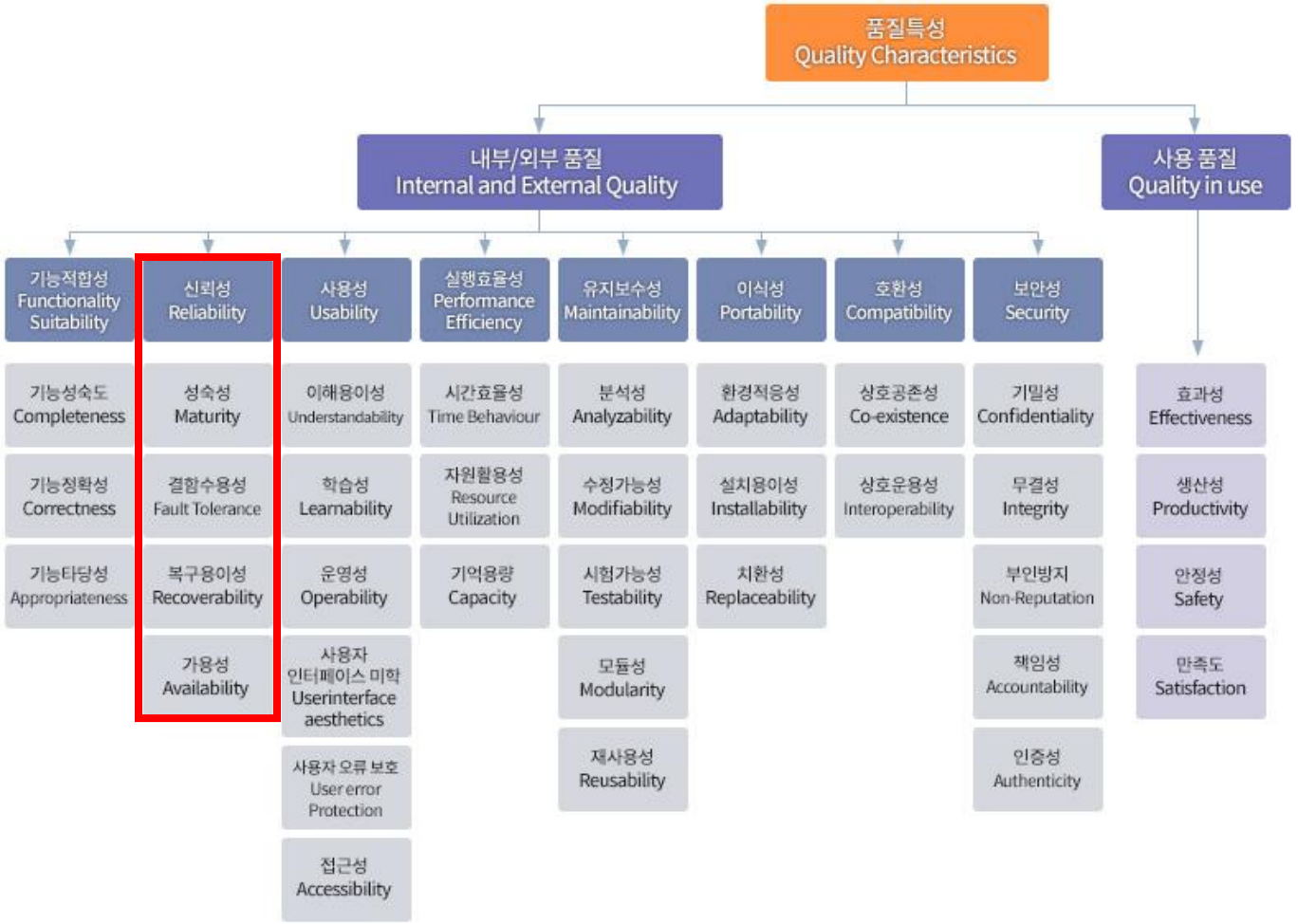
Performance

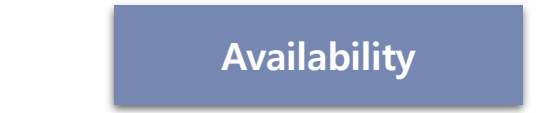


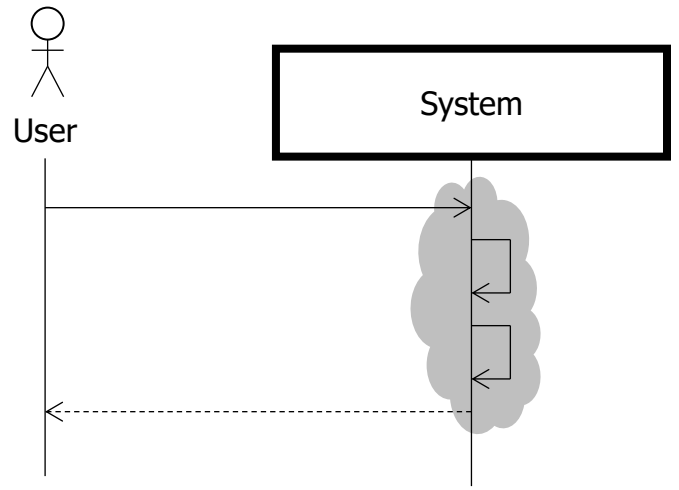
품질 요구사항의 분류



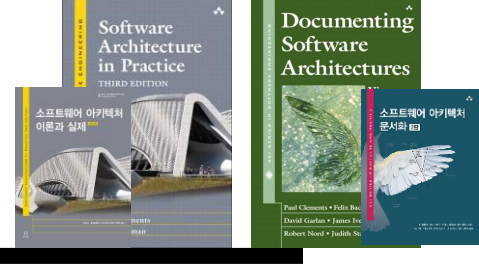
ISO/IEC 25010 - 2011



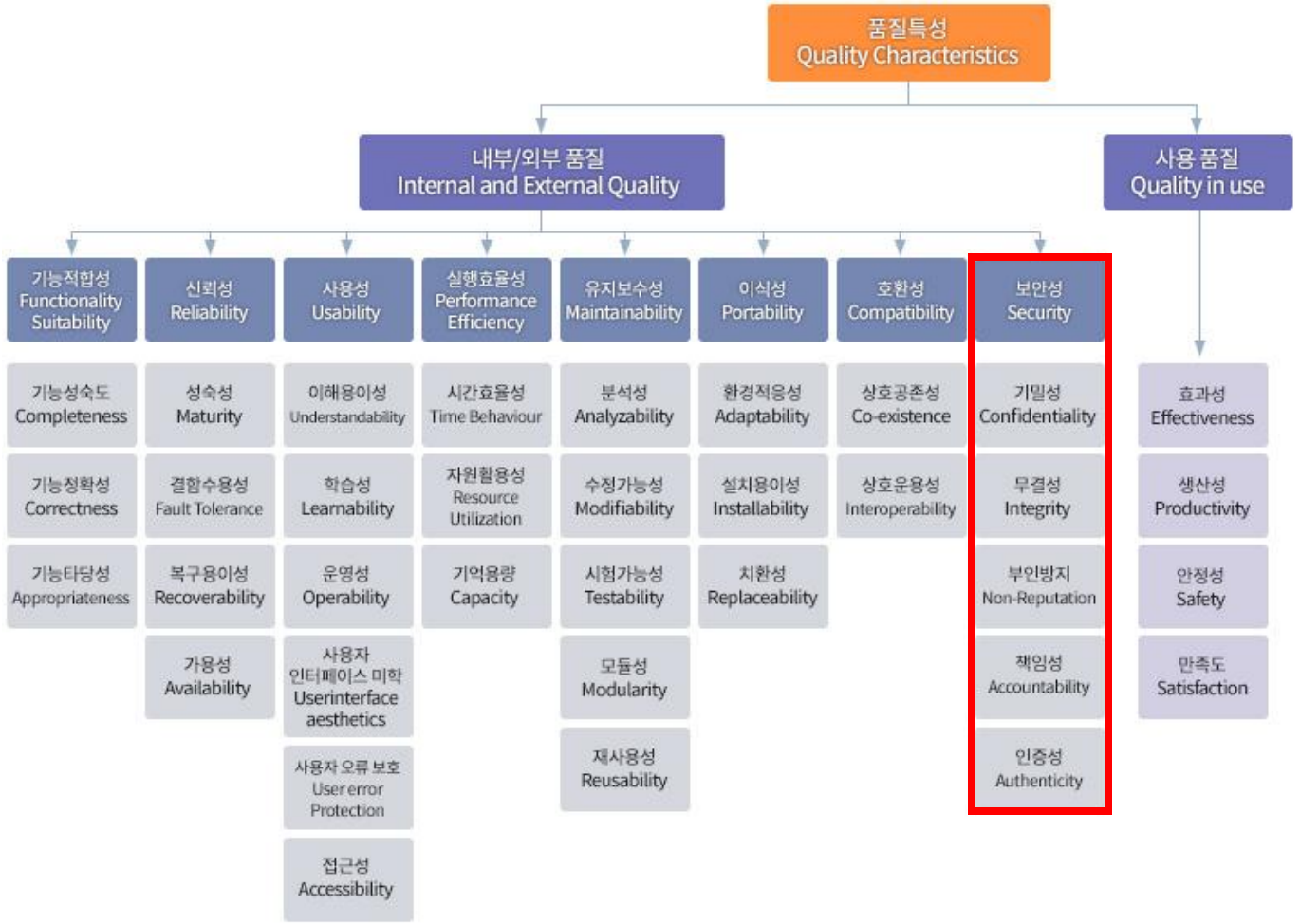




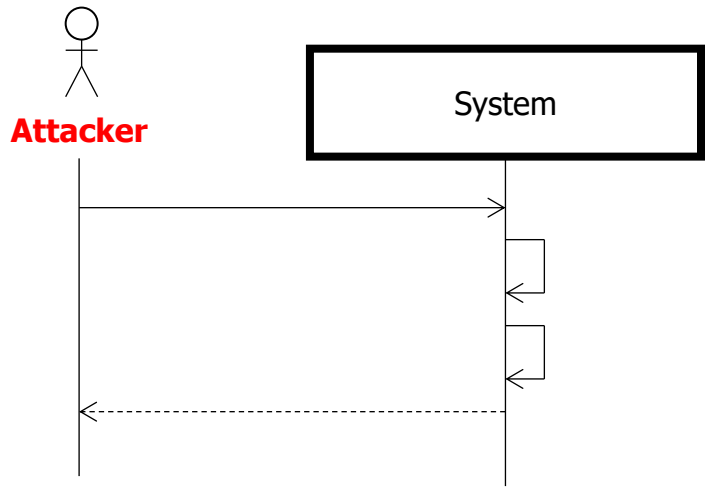
품질 요구사항의 분류



ISO/IEC 25010 - 2011



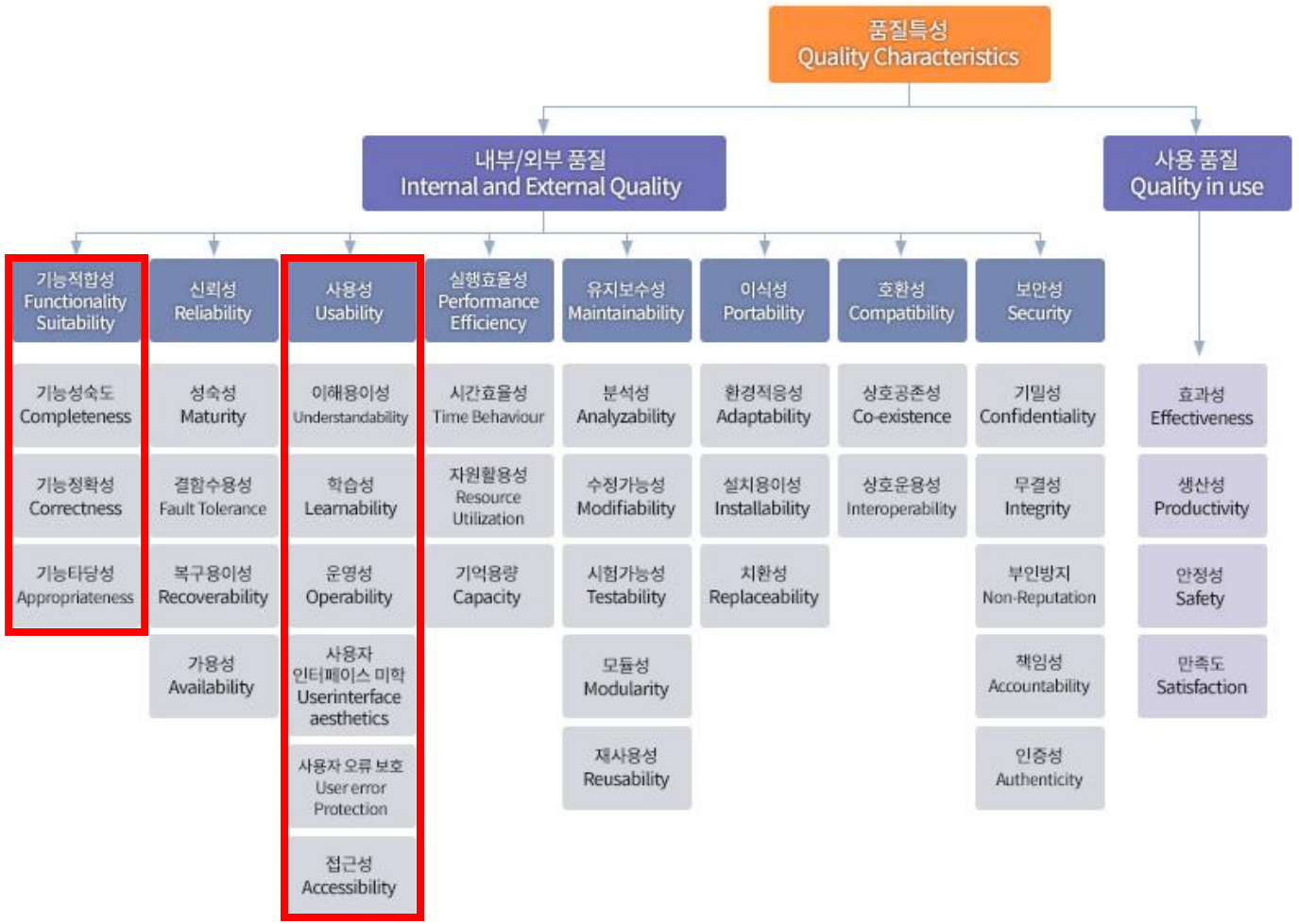




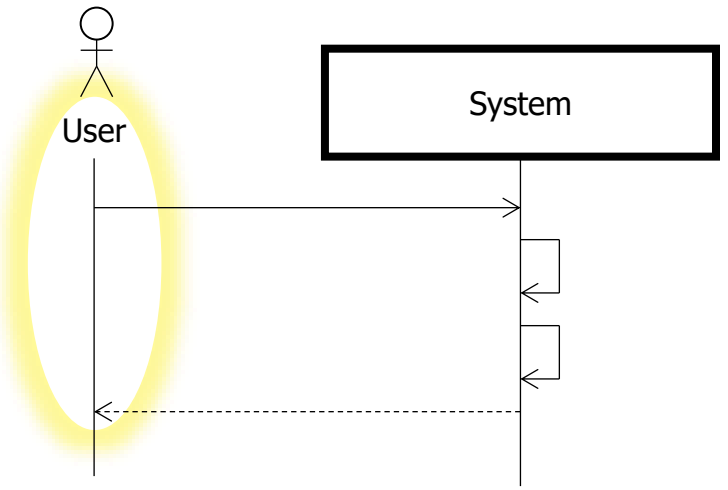
품질 요구사항의 분류



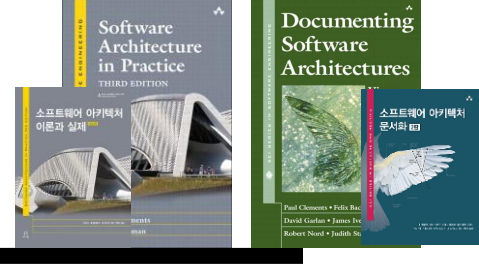
ISO/IEC 25010 - 2011



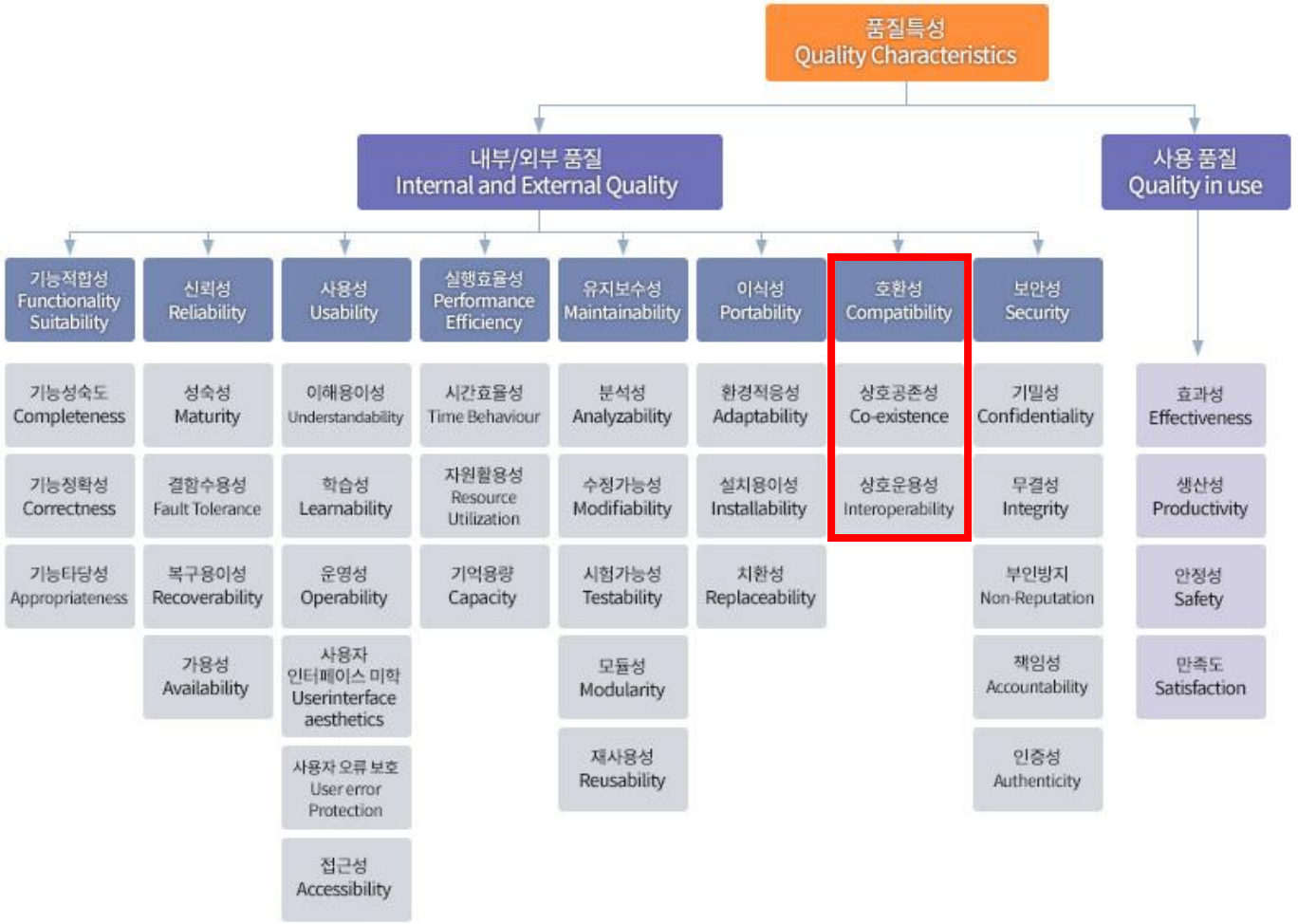


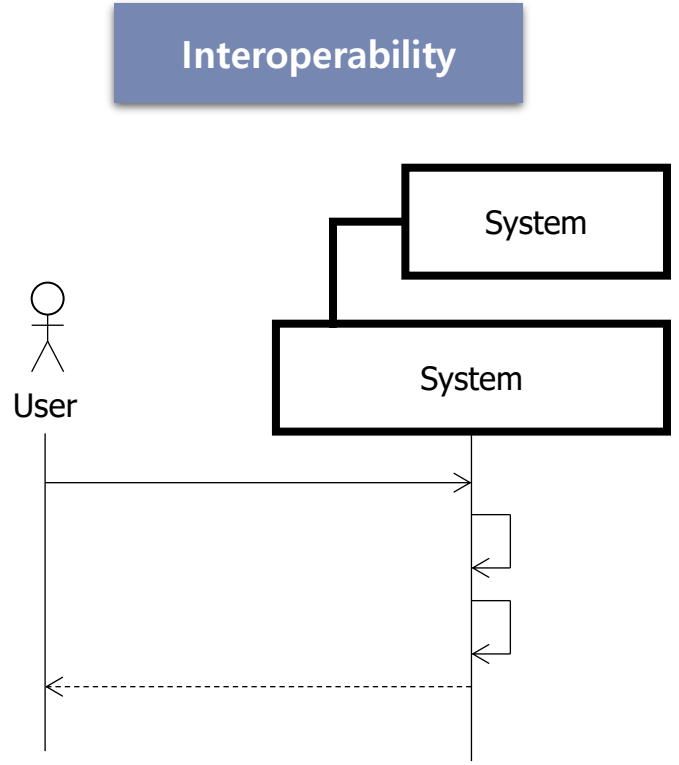


품질 요구사항의 분류

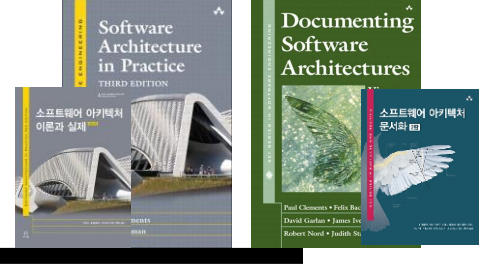


ISO/IEC 25010 - 2011

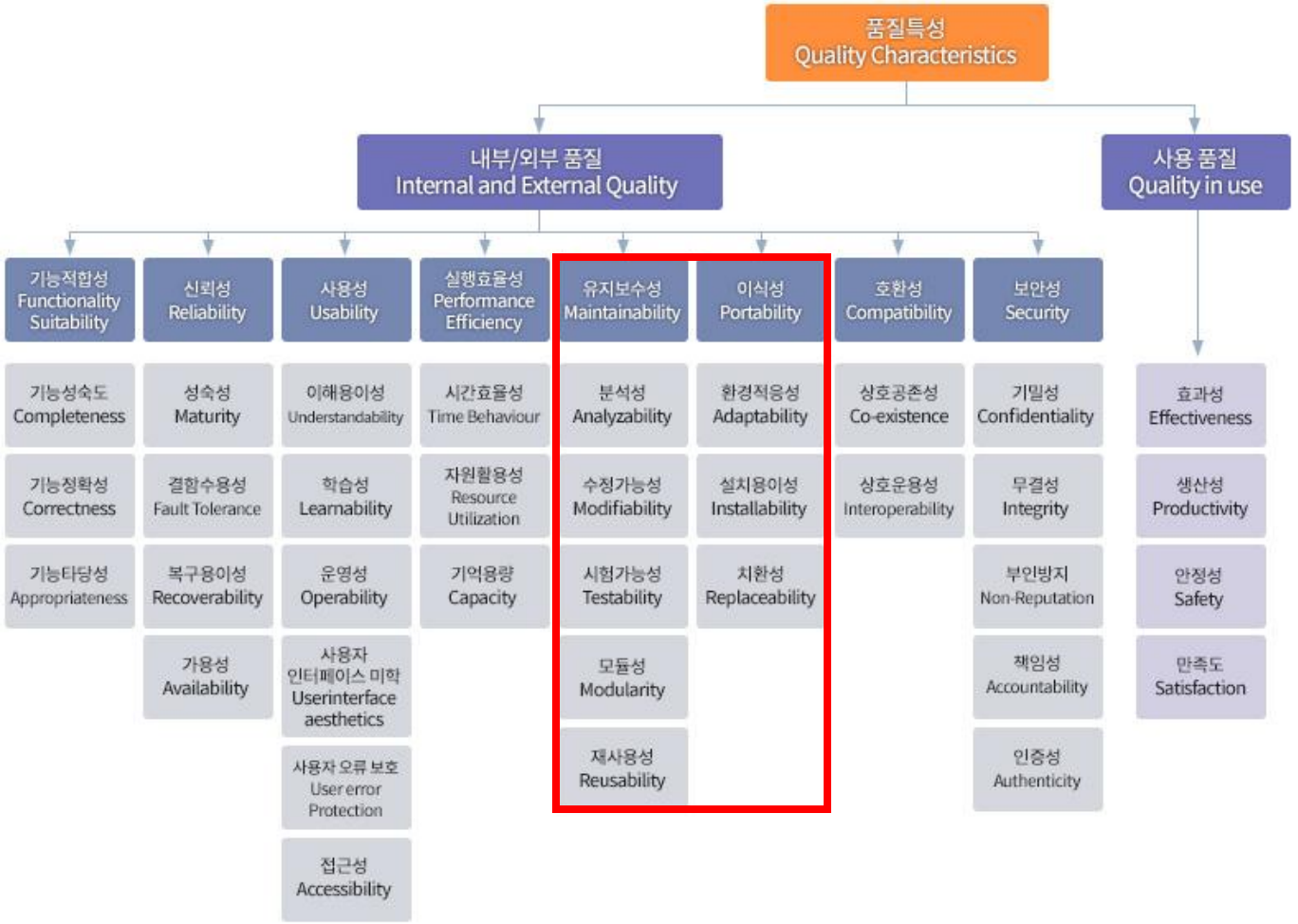




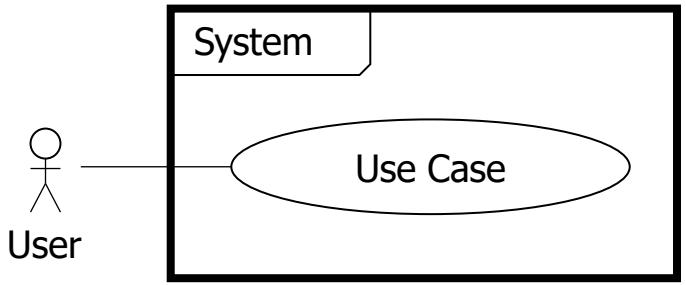
품질 요구사항의 분류



ISO/IEC 25010 - 2011



Modifiability



품질 시나리오

테스트 시나리오 + **측정**

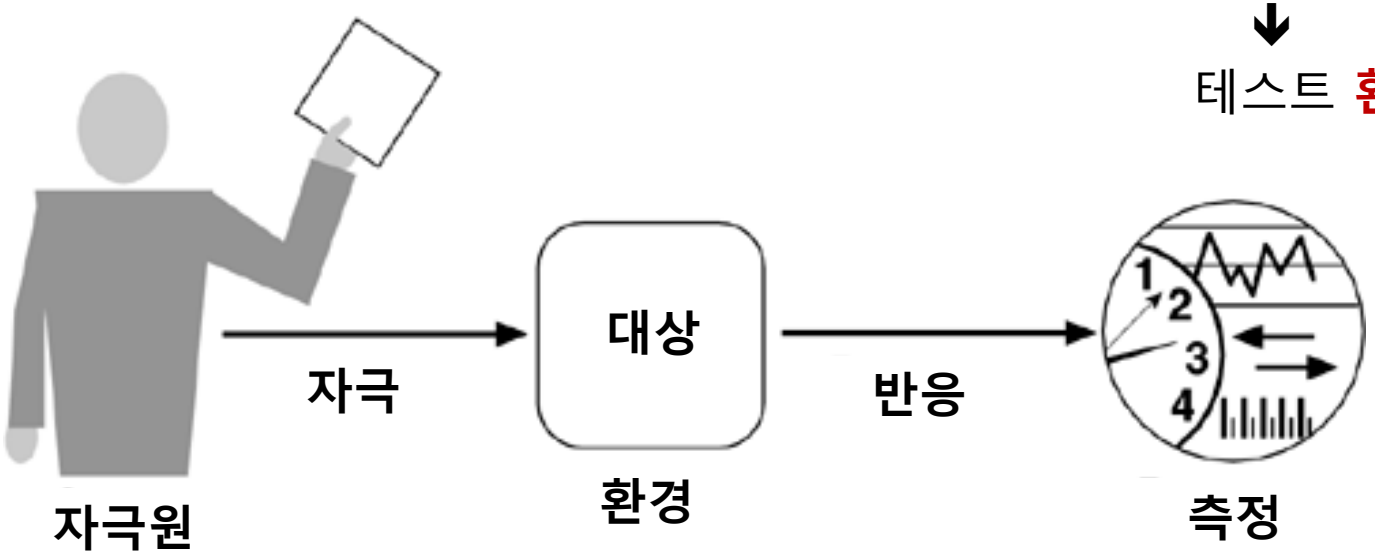
↓

테스트 **환경** + 동작

↓

자극 → 자극의 주체(**자극원**) + **자극**

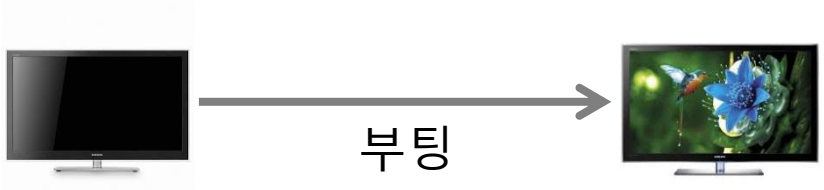
반응 → 반응의 주체(**대상**) + **반응**



주어진 환경에, 자극원의 자극에 대한,

대상의 반응을 측정한다.

품질 시나리오



부팅 : TV가 꺼진 상태(환경)에서 사용자(자극원)가 리모컨의 전원 버튼을 누르면(자극), TV(대상/시스템)가 현재 수신 중인 방송을 보여 주고 사용자의 입력을 기다리는 상태로 전환(반응)한다.

부팅 시간: 부팅에 걸리는 시간(측정).

부팅 시간 10초 이내이어야 한다, 부팅 시간이 짧을 수록 좋다.

품질 요구사항

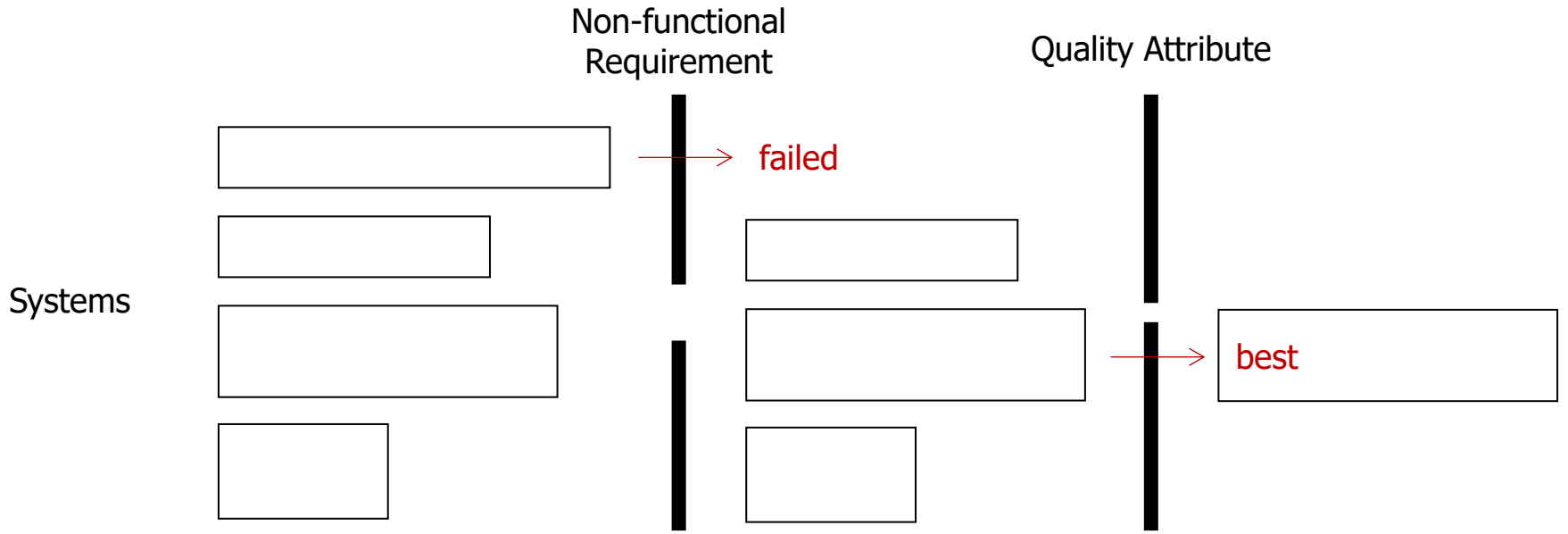
품질 요구사항의 유형

Non-functional Requirement

Quality criteria to accept the system. The accepted is enough.

Quality Attribute

Quality criteria to evaluate the system. The more is the better.

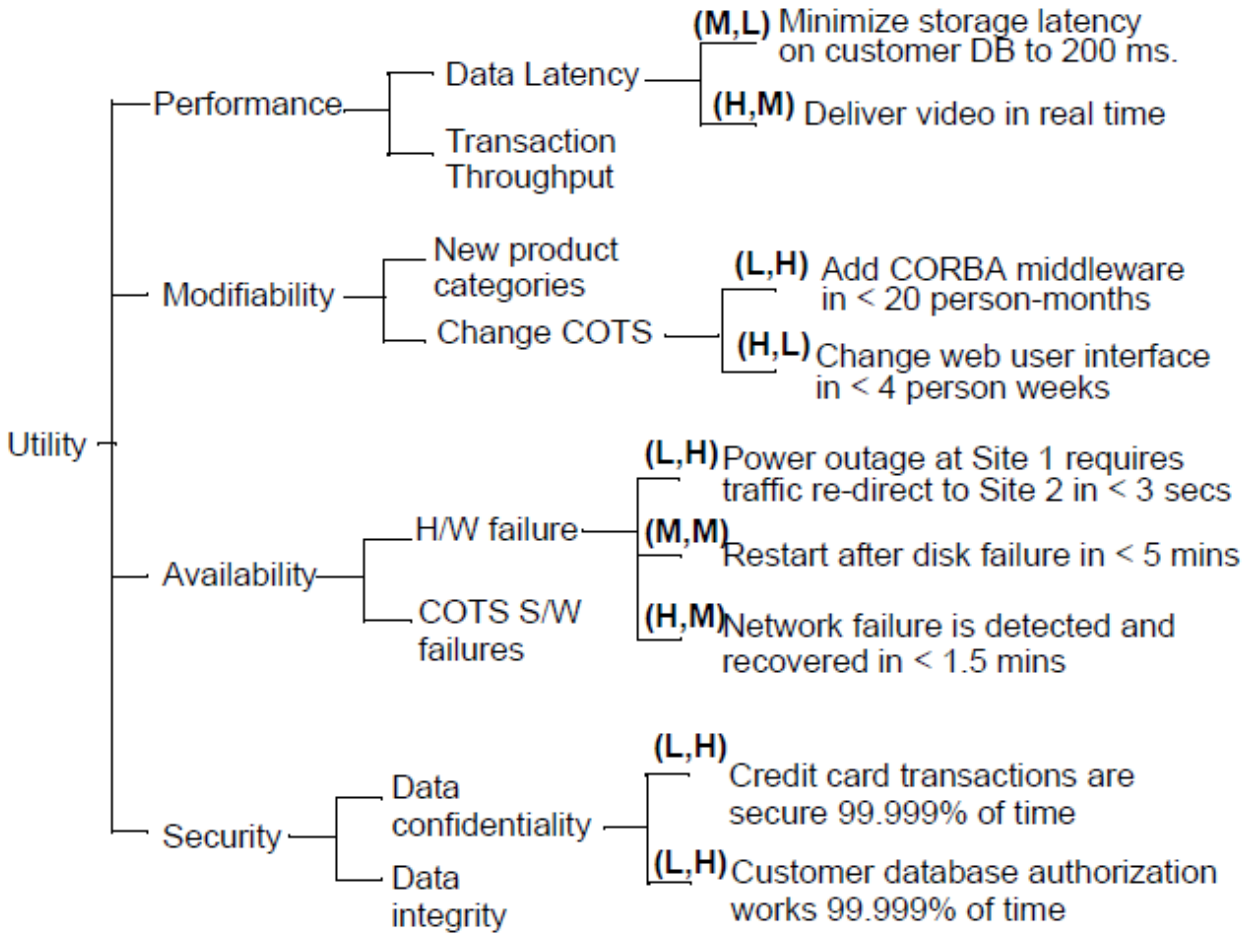


실습3. 품질 요구사항

CC BY NC ND

품질 시나리오의 생성

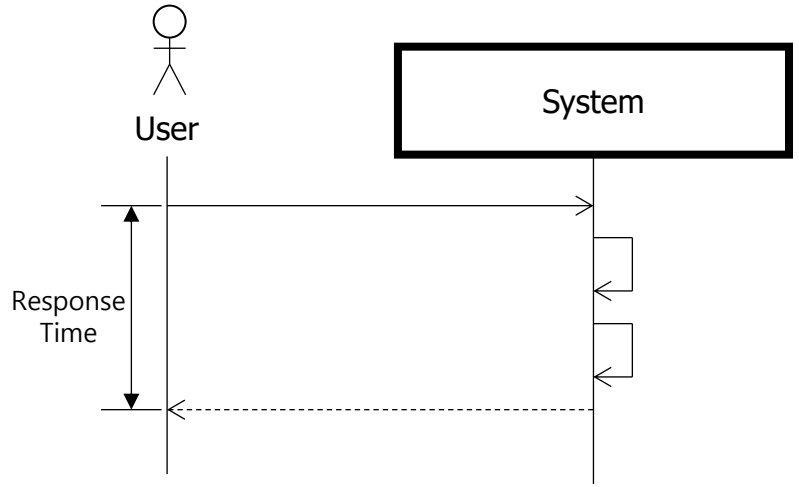
Utility Tree



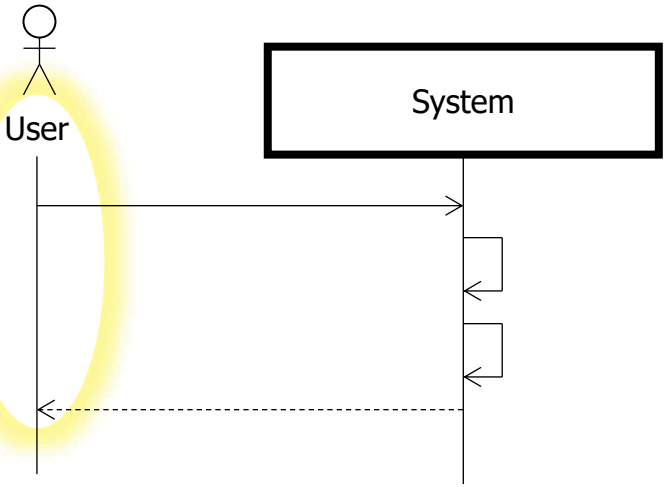
품질 시나리오의 생성

[실습3A] 동작 측면의 품질 시나리오를 생성해 보자.

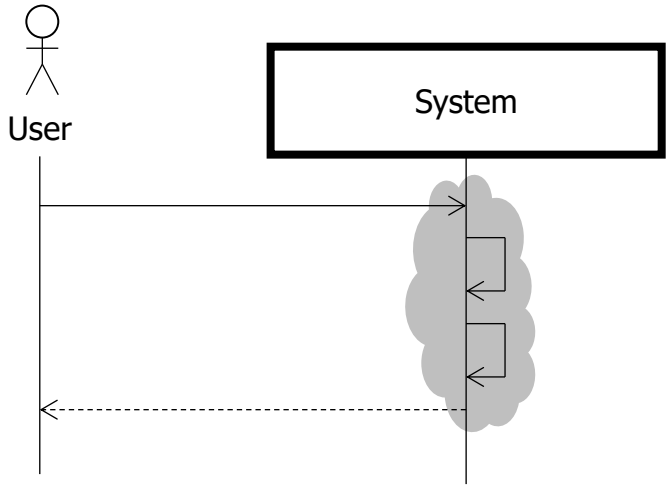
Performance



Usability



Availability

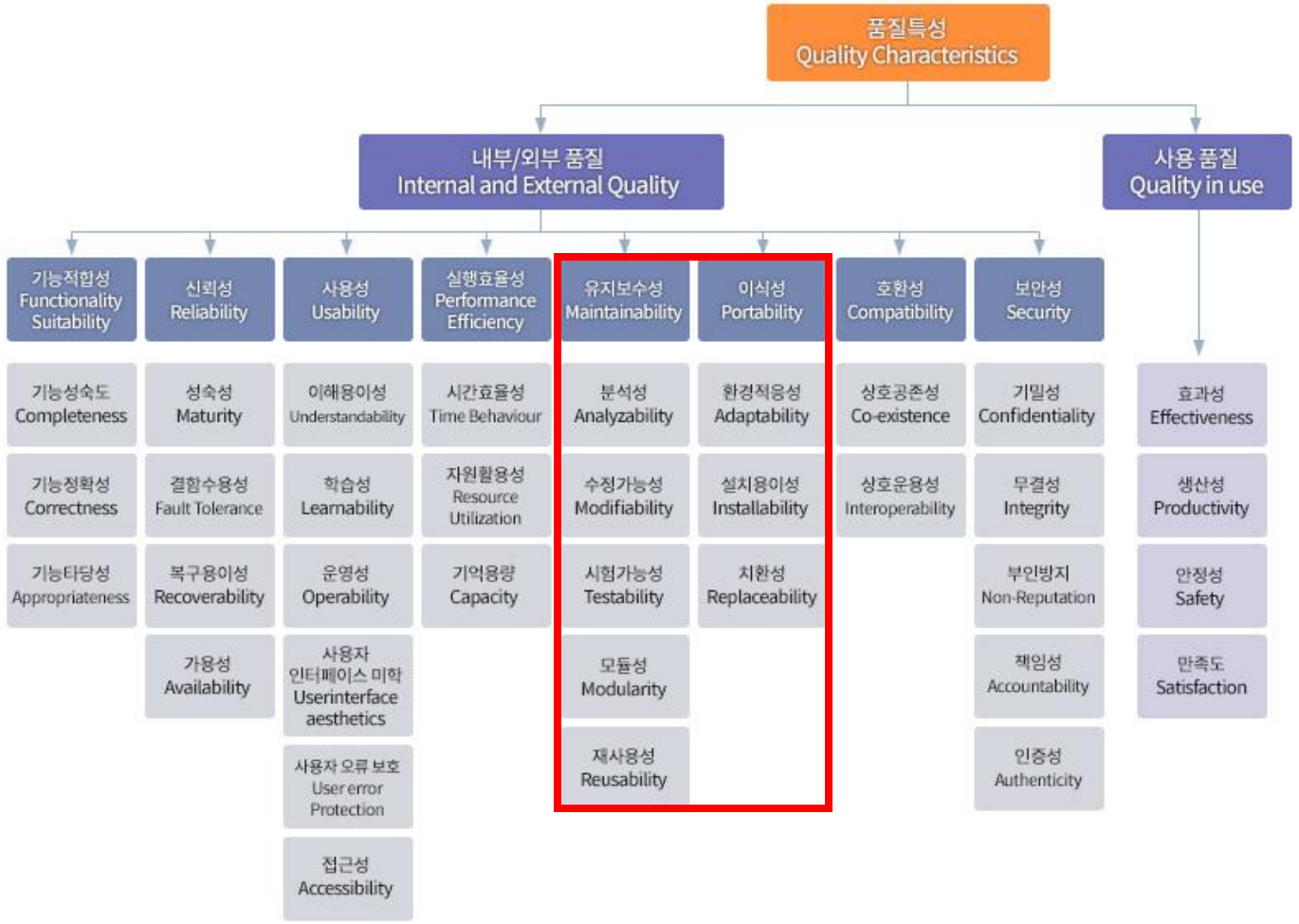


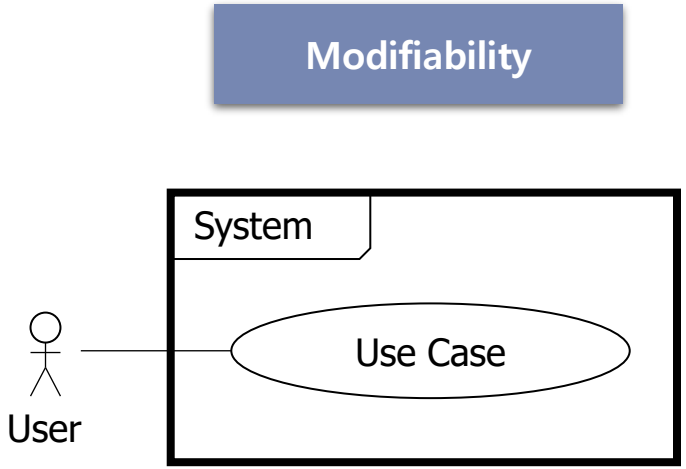
Security

Interoperability

품질 시나리오의 생성

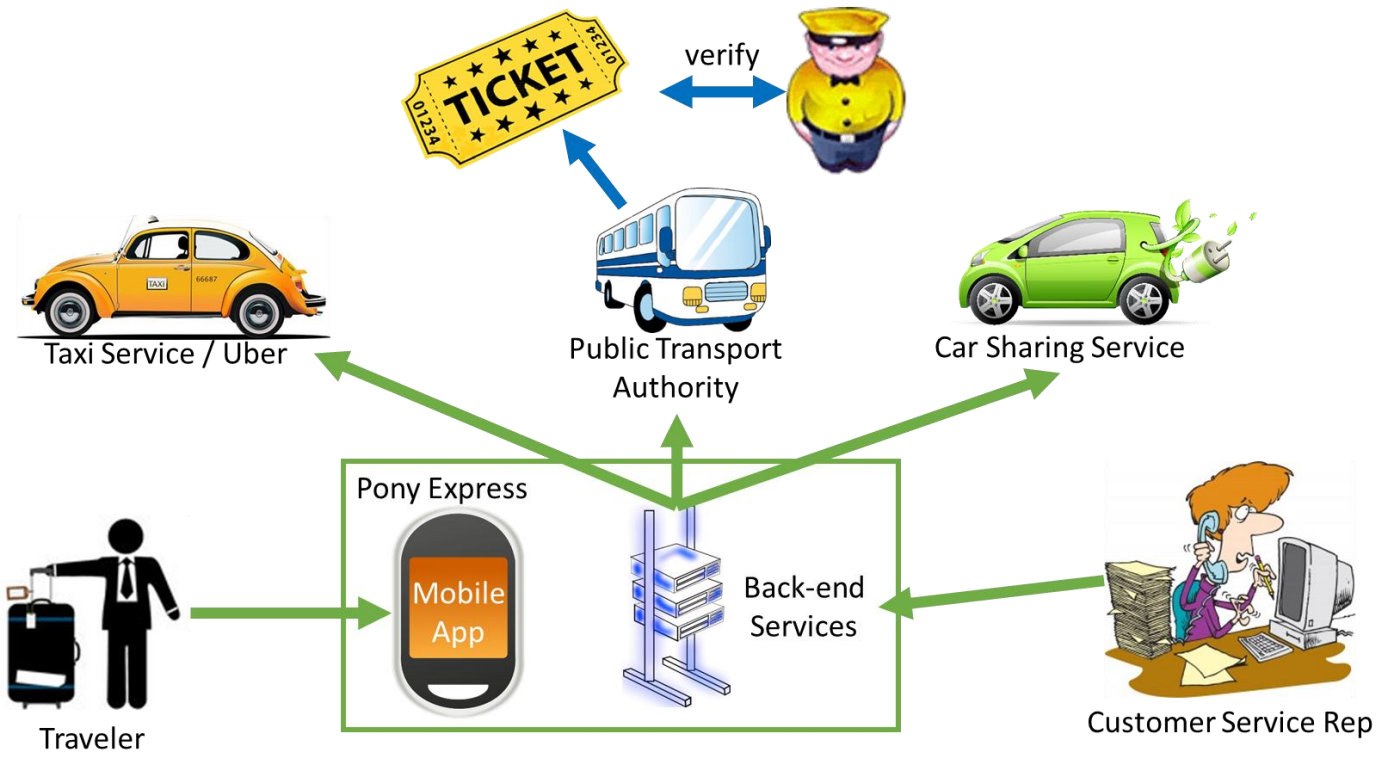
[실습3B] 개발 측면의 품질 시나리오를 생성해 보자.





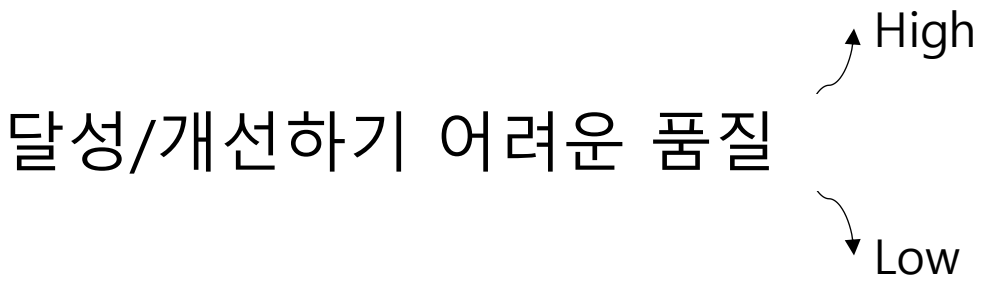
품질 요구사항의 선정

[실습3C] 품질 시나리오를 평가해 보자(중요도).



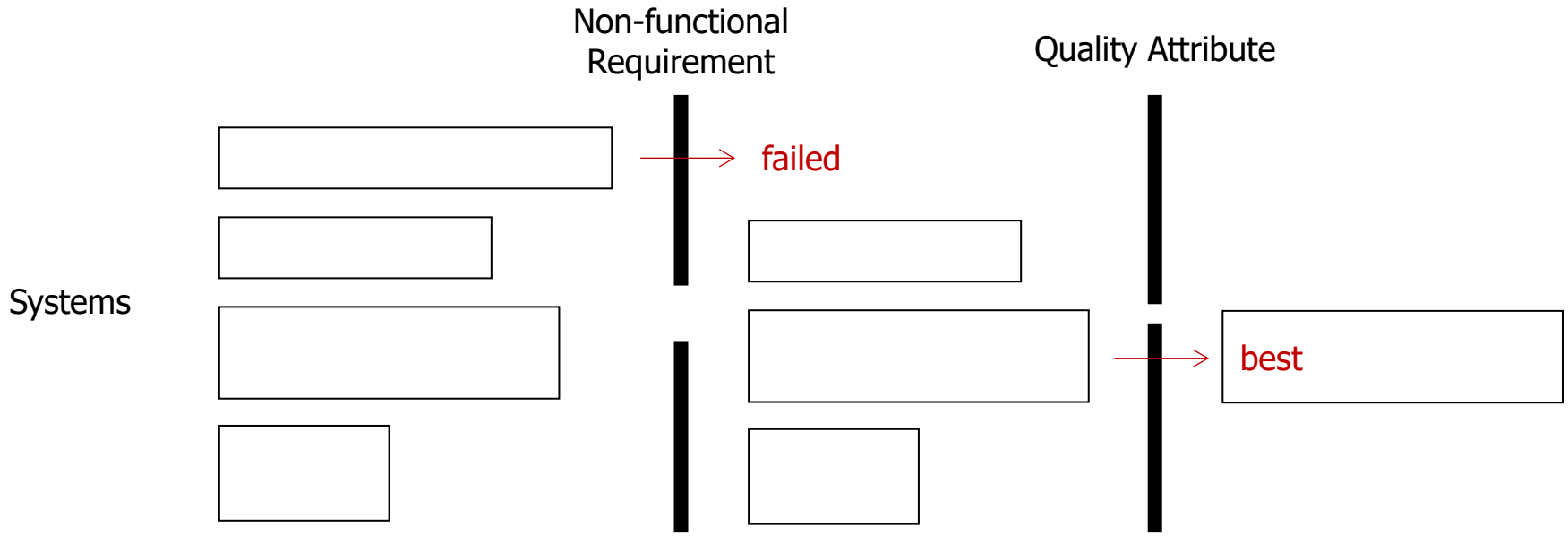
품질 요구사항의 선정

[실습3D] 품질 시나리오를 평가해 보자(복잡도/난이도).



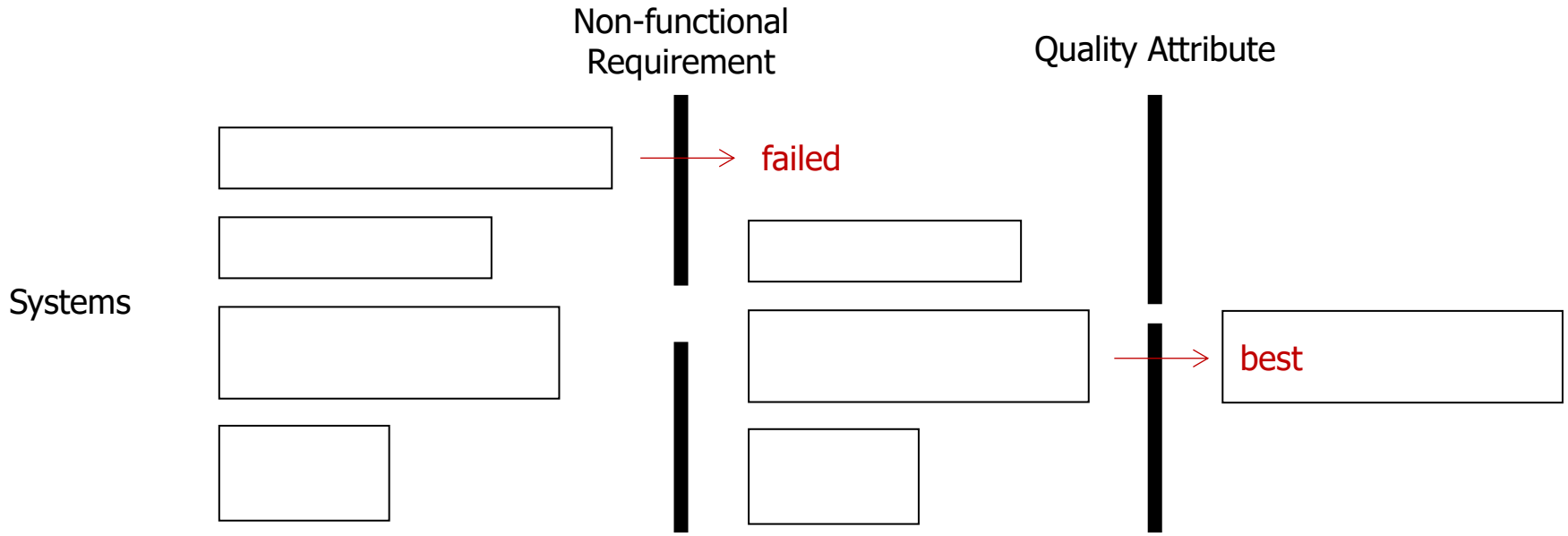
품질 요구사항의 선정

[실습3E] 비기능적 요구사항을 선정해 보자.



품질 요구사항의 선정

[실습3F] 품질 속성을 선정해 보자(우선 순위).



품질 요구사항의 선정

NFR은 business 측면에서 반드시 만족되어야 하는 품질,
우선 순위가 높은 QA는 business 측면에서 중요한 품질로 선정되는 것이 필요합니다. Business driver와의 연계성.
NFR은 기본적으로 만족해야 하는 품질로 경쟁사도 당연히 만족하는 품질,
품질 속성은 다른 경쟁사와의 비교에 사용되는 품질입니다.

품질 시나리오	중요도	난이도	품질 요구사항
QS1	H	H	
QS2	H	L	
QS3	L	H	
QS4	L	L	

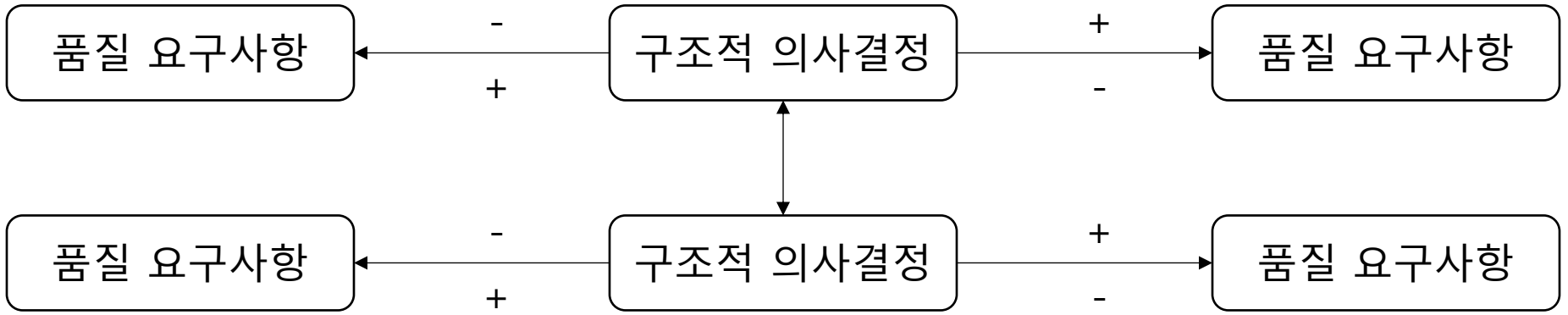
중요도는 이해관계자에 의해서 결정되지만,
난이도/복잡도는 아키텍트가 결정해야 한다.
난이도/복잡도에 대한 객관적인 검토가 필요합니다.

실습4. 구조 평가

CC BY NC ND

구조 평가(동작)

- [실습4A] 동작 측면의 품질 요구사항을 평가해 보자.



구조 평가(동작)

- [실습4A] 동작 측면의 품질 요구사항을 평가해 보자.

품질 시나리오	**NFR_01. LiDAR 데이터 처리 성능.** LiDAR 데이터를 처리 속도가 빠를 수록 좋다. 100ms 이내.
속성	성능
환경	정상적인 동작 상황
자극	LiDAR 센서가 시스템에게 거리 데이터를 제공한다.
반응	1. 시스템은 LiDAR 데이터(거리)로 사용자와의 거리를 파악한다. 2. 시스템은 사용자와의 거리를 N미터 이내로 좁히기 위한 이동 속도를 결정한다. 3. 시스템은 MCU에게 이동 명령을 전달한다. 4. MCU는 이동 명령에 따라 모터를 제어한다.
측정	[LiDAR 데이터 처리 시간] = [시스템이 MCU에게 이동 명령을 전달한 시간] – [LiDAR 센서가 시스템에 거리 데이터를 제공한 시간]

구조적 의사결정	위험요소	민감점	절충점	비위험요소

구조 평가(개발)

- [실습4B] 개발 측면의 품질 요구사항을 평가해 보자.

품질 시나리오	**QA_02. LiDAR 센서의 변경 용이성.** ROS2를 지원하지 않는 LiDAR 센서로의 변경이 용이할 수록 좋다.
속성	변경 용이성
환경	신규 제품 개발에서
자극	HW팀에서 ROS2를 지원하지 않는 신규 LiDAR 센서로의 변경을 요청한다.
반응	1. 개발팀은 신규 LiDAR 센서 드라이버를 이용해서 ROS2 노드를 개발한다. 2. 개발팀은 개발된 ROS2 노드를 검증한다. 3. 개발팀은 개발된 시스템을 릴리즈 한다.
측정	개발 비용(M/M)

구조적 의사결정	위험요소	민감점	절충점	비위험요소

재설계와 ADR

cc BY NC ND

ADR (Architecture Decision Records)

https://adr.github.io/

- Why Write an ADR?
  - Documentation: To clearly document architectural decisions and keep a record.
  - Communication: To ensure every team member, including newcomers, understands the architectural decisions made.
  - Rationale: To provide insights into why certain decisions were made.
  - Learning: To allow reflection on past decision-making processes and inform future decisions.

ADR (Architecture Decision Records)

https://adr.github.io/

- Components of ADR
  - **Title**: A concise title that reflects the essence of the decision.
  - Status: The current status of the decision (e.g., proposed, accepted, rejected, ...).
  - **Context**: A detailed description of the factors that drove the decision-making process
    - the problem statement, constraints, and the environment.
  - **Decision** : A clear statement of what was decided.
  - Decision Drivers: Criteria or reasons that are crucial in making the decision.
  - **Considered Options**: A list of alternative options.
  - **Justification**: A summary of why that option was chosen.
  - Consequences: An overview of the results and impacts of the decision,
    including benefits and drawbacks.
  - Action Items: Specific steps that need to be taken to implement the decision.
  - Approval: Who approved the decision, their title, and the date of approval.

ADR (Architecture Decision Records)

https://adr.github.io/madr/

- MADR(Markdown Any Decision Records)

```
# Use Plain JUnit5 for advanced test assertions

## Context and Problem Statement

How to write readable test assertions?
How to write readable test assertions for advanced tests?

## Considered Options

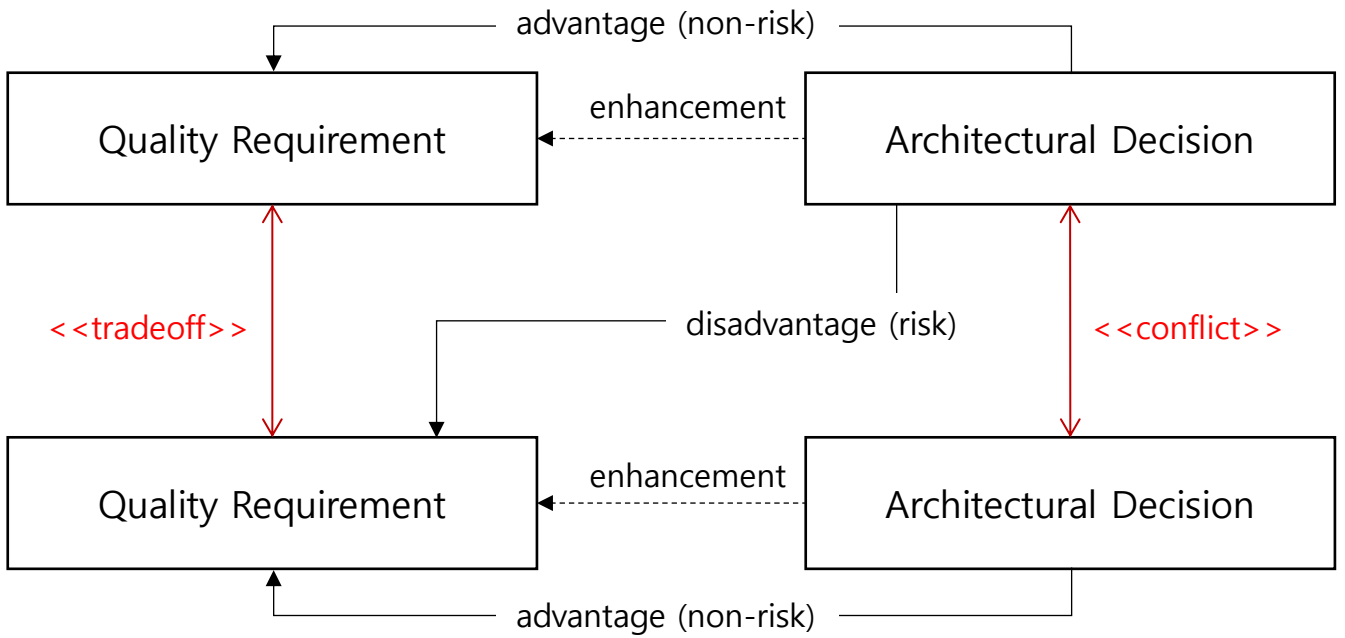
* Plain JUnit5
* Hamcrest
* AssertJ

## Decision Outcome

Chosen option: "Plain JUnit5", because it is a standard framework and the features
of the other frameworks do not outweigh the drawback of adding a new dependency.
```

ADR (Architecture Decision Records)

It's better to manage relationships between quality requirements and architectural decisions (candidate architectures).



ADR (Architecture Decision Records)

It's better to manage relationships between quality requirements and architectural decisions (candidate architectures).

	Existing AD	Candidate AD #1	Candidate AD #2
QR #1	+	+	-
QR #2	--	+	++
QR #3	-	-	+

-- ADR--

Title

Context / Problem Statement

Decision

Considered Options

Justification / Pros and Cons

Better to create ADRs

https://www.ozimmer.ch/practices/2023/04/03/ADRCreation.html

Metaphors

Journal *Action Plan* *Contract*

Executive Summary *Verdict/Scale* *Letter of Intent*

Anti Patterns (Don't)

Fairy Tale, Sales Pitch, Free Lunch, Dummy Alternative

Sprint, Tunnel Vision, Maze

Blueprint or Policy in Disguise, Mega-ADR, Novel/Epic

Magic Tricks: False Urgency, Problem-Solution Mismatch, Pseudo-Accuracy

Good Practices (Do)

Prioritization and timing by significance

Meta Qualities, objectivity and bias awareness

Tradeoff analysis, editorial quality of ADR, rightsizing

Slicing in stages, disclosure of confidence level

Author Pledge

Architectural Significance Check, Template Use, Explicit Question, Option, Criteria
Presentation Quality: Thorough, Focused, Factual
Candor/Honesty

Related Advice (Other Posts)

Architectural Significance Test, MADR or Y-Statements, Definition of Done
Good ADR review practices and anti-patterns

Better to review ADRs

https://www.ozimmer.ch/practices/2023/04/05/ADRReview.html

Roles

Peer, Coach/Mentor

Direct Stakeholders *External Design Authority*

Review Advice (Do)

Deliver what is asked for, prioritize, document scope and goals of the review

Refer to quality attributes, acknowledge context, be concrete and factual, thorough, focused

Problem-solution style, impressions but no interpretations, good and bad, fair and polite

Make comments resolvable, offer help with resolution, review the review

Anti Patterns (Don't)

Pass Through

Copy Edit

Siding, Dead End

Self Promotion, Conflict of Interest

Power Game

Groundhog Day

Offended Reaction

Reviewer Pledge

Manage scope, content, style (professional, constructive)
Avoid anti-patterns, use checklist, ensure actionability
Review like you want to be reviewed

Related Advice (Other Posts)

ASR Test, MADR or Y-Statements, Definition of Done (AD)
ADR creation practices and anti-patterns

실습5. 재설계

cc BY NC ND

재설계(동작)

- [실습5A] 동작 측면의 품질 요구사항을 개선해 보자.

재설계(개발)

- [실습5B] 개발 측면의 품질 요구사항을 개선해 보자.

구조 설계와 문서화

cc BY NC ND

구조 설계

- 시스템이 요구되는 목적(품질)에 잘 부합하도록 설계한다.
  - 품질을 좋게 하는 구조적 의사 결정을 채택한다.
  - 채택된 **구조적 의사 결정의 집합 ➔** 구조

$$\boldsymbol{Quality} = \{\boldsymbol{Q_1}, \boldsymbol{Q_2}, \ldots, \boldsymbol{Q_M}\}$$

$$\textbf{Candidate AD} = \{\boldsymbol{CAD_1}, \boldsymbol{CAD_2}, \ldots, \boldsymbol{CAD_N}\}$$

후보 구조: 채택 가능한 구조적 의사결정

$$\boldsymbol{Evaluation}(\boldsymbol{CAD_j}) = \sum_{\boldsymbol{i=1}}^{\boldsymbol{M}} \boldsymbol{Benefit}(\boldsymbol{Q_i}, \boldsymbol{CAD_j})$$

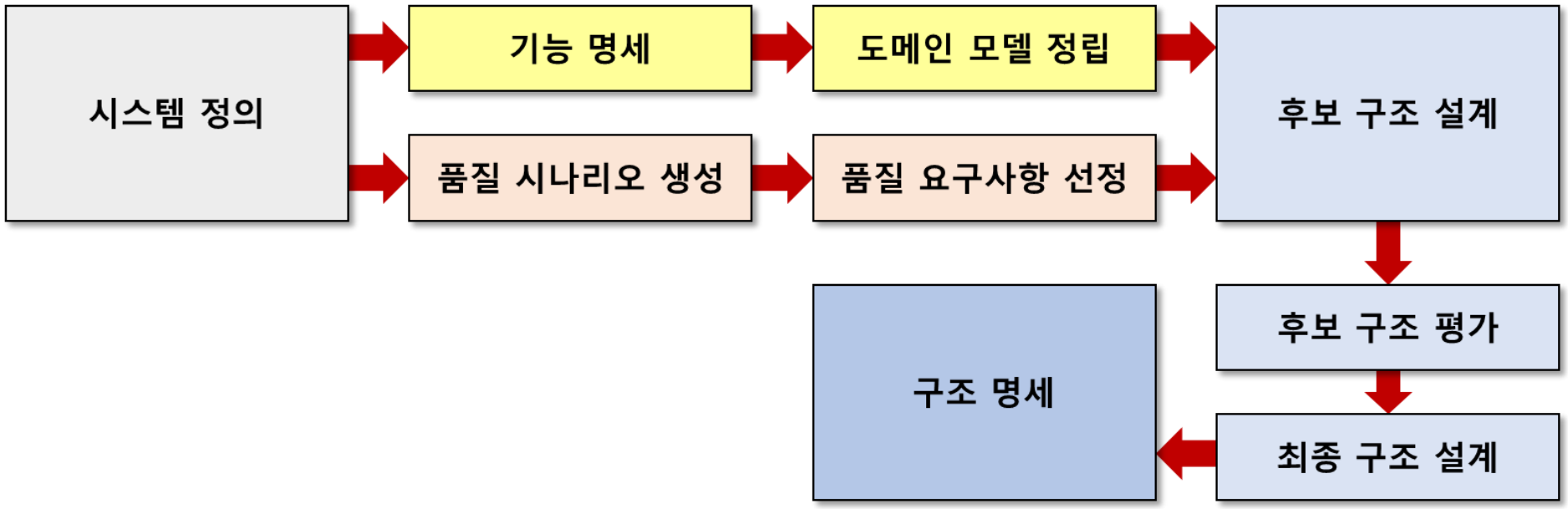
후보 구조가 품질에 미치는 영향으로 후보 구조의 채택 여부를 결정한다.

$$\boldsymbol{ARCH} = \{\boldsymbol{SAD_1}, \boldsymbol{SAD_2}, \ldots, \boldsymbol{SAD_L}\}$$

선택된 후보 구조로 구조를 설계한다.

구조 설계

- 시스템이 요구되는 목적(품질)에 잘 부합하도록 설계한다.
  - 품질을 좋게 하는 구조적 의사 결정을 채택한다.
  - 채택된 **구조적 의사 결정의 집합** ➔ 구조

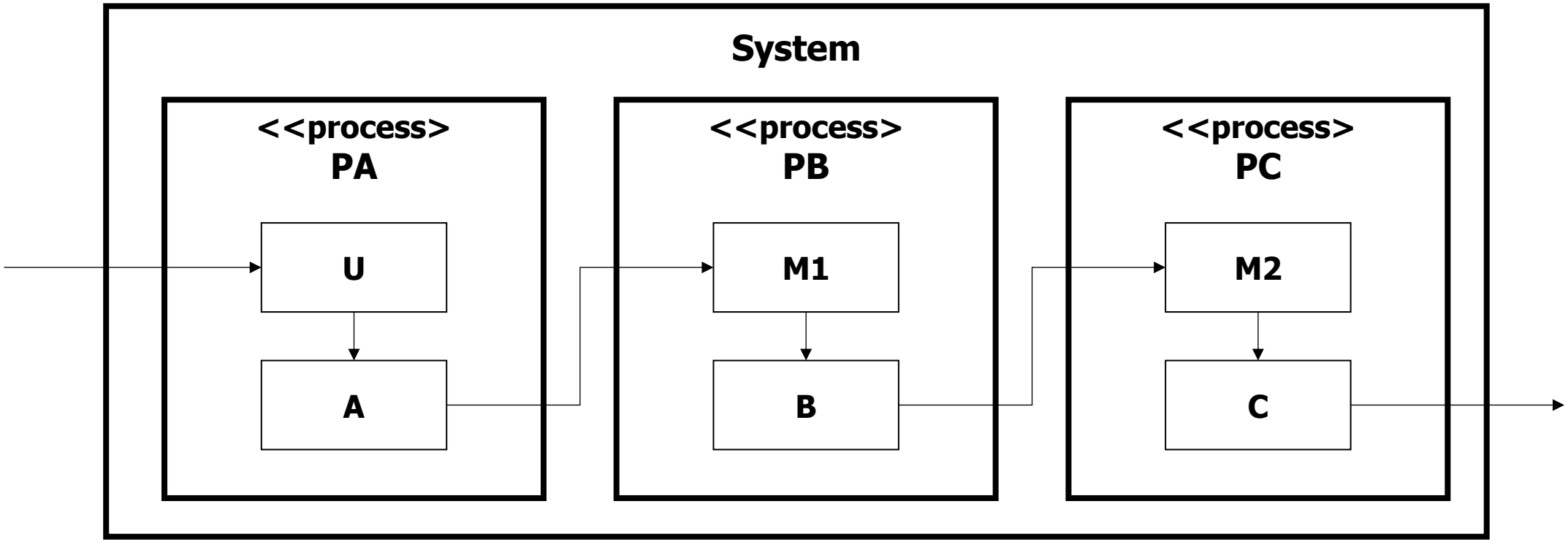


구조 설계

https://github.com/bosornd/example-module-design

- Module Design – Object-oriented Design

STEP 1. **Identify Object (Component Design)**

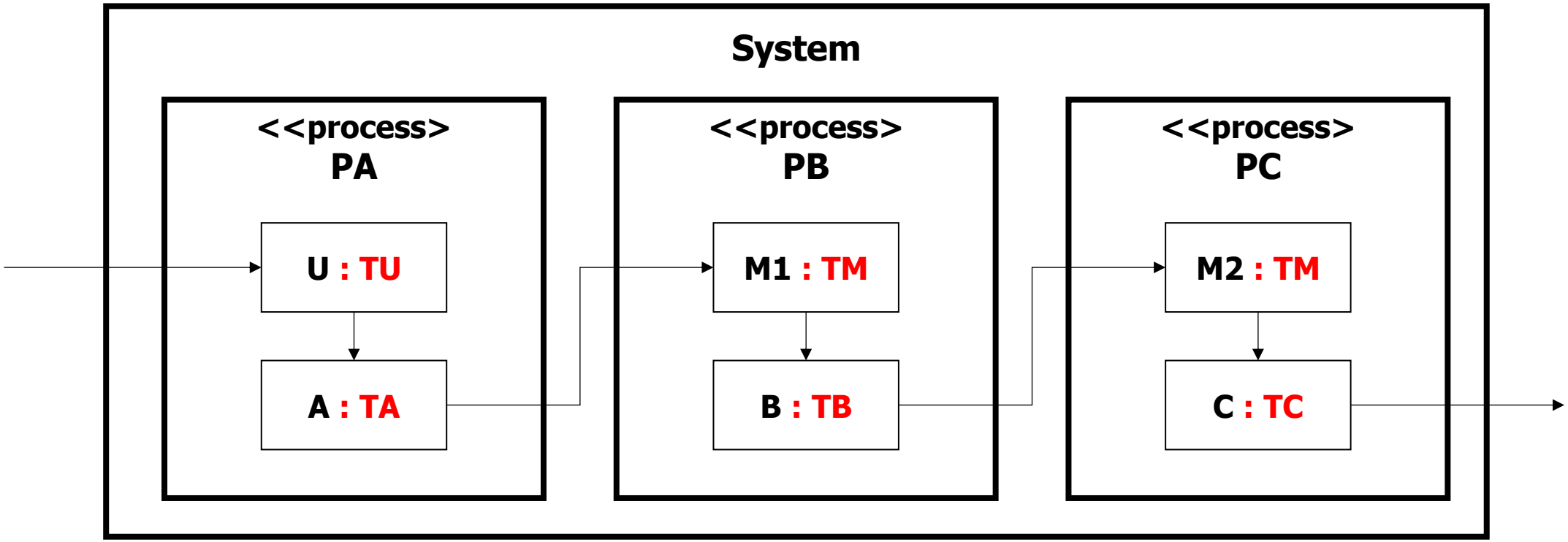


구조 설계

https://github.com/bosornd/example-module-design

- Module Design – Object-oriented Design

STEP 2. **Define Class (Module Design)**

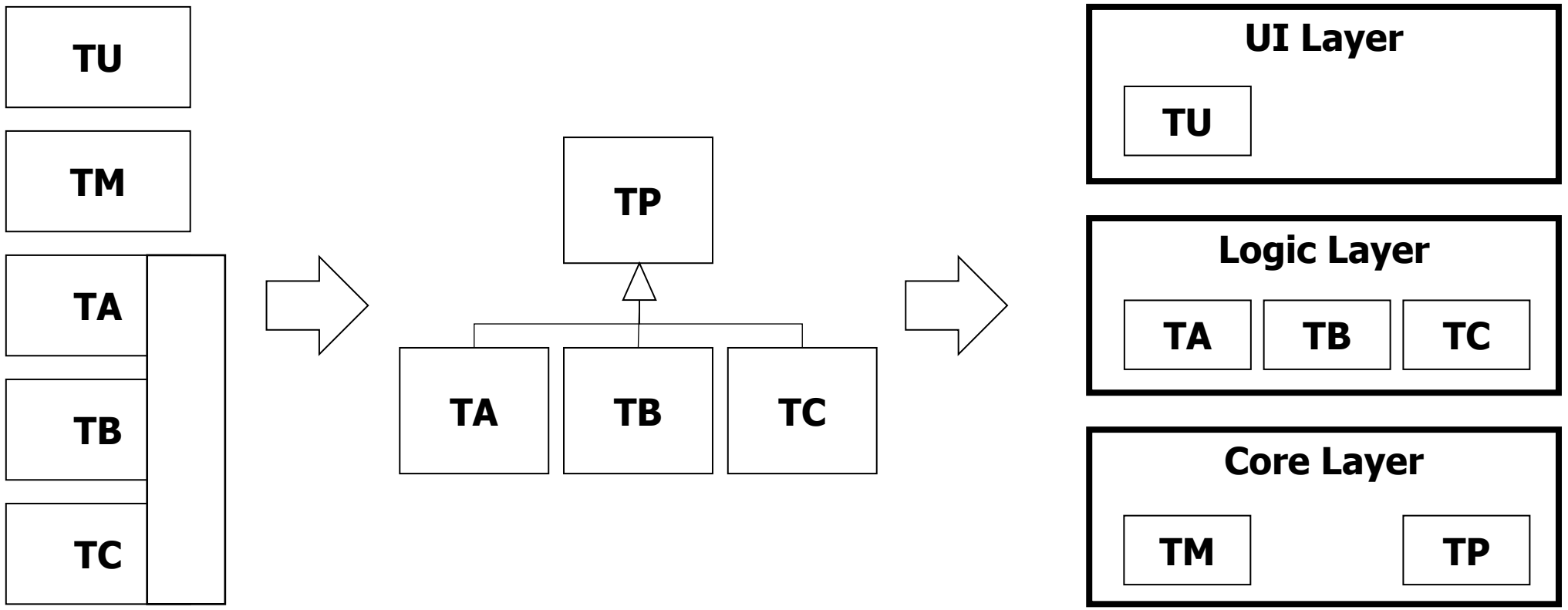


구조 설계

https://github.com/bosornd/example-module-design

- Module Design – Object-oriented Design

STEP 3. **Refine Class (Module Design)**

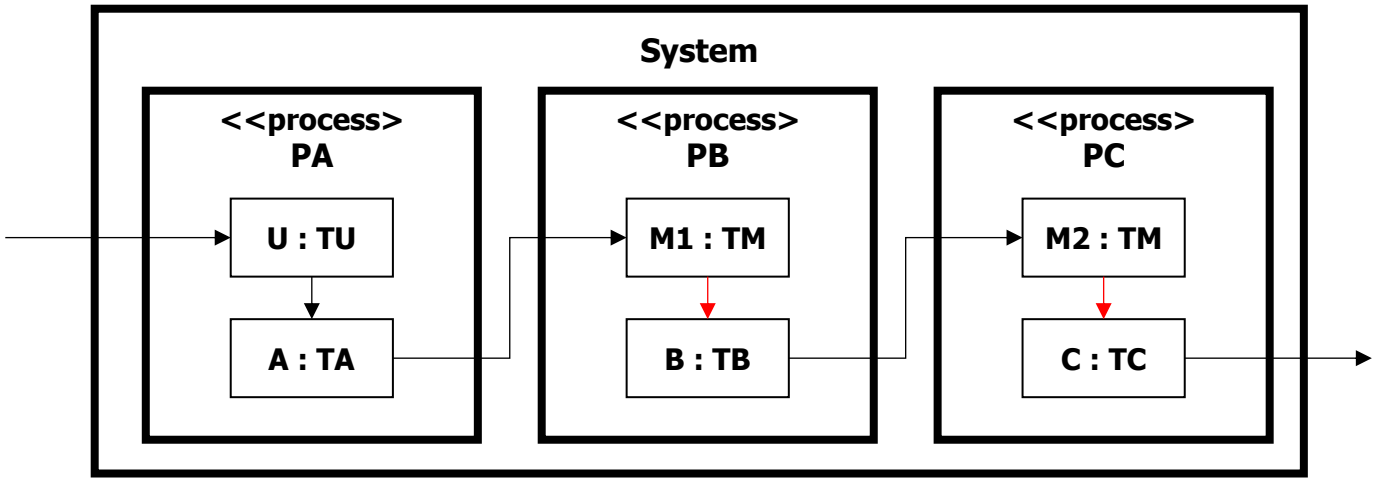


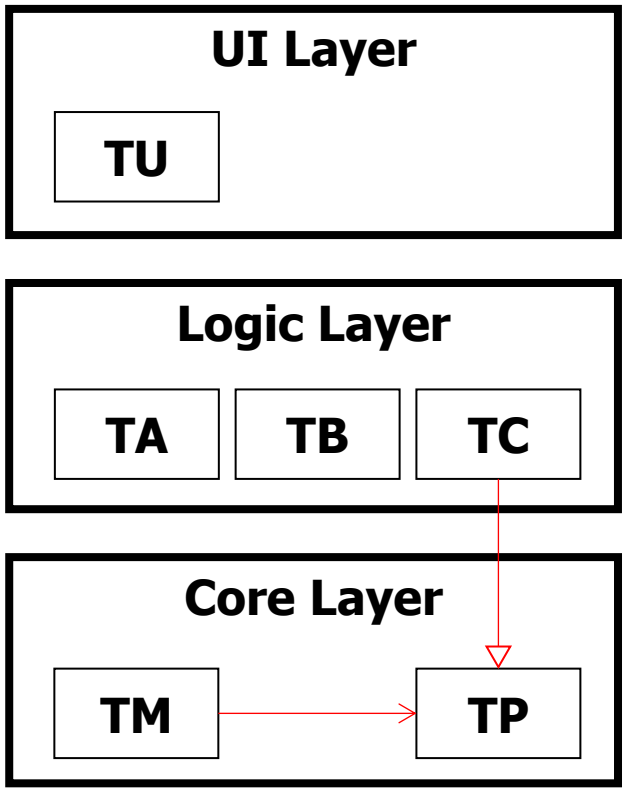
구조 설계

https://github.com/bosornd/example-module-design

- Module Design – Object-oriented Design

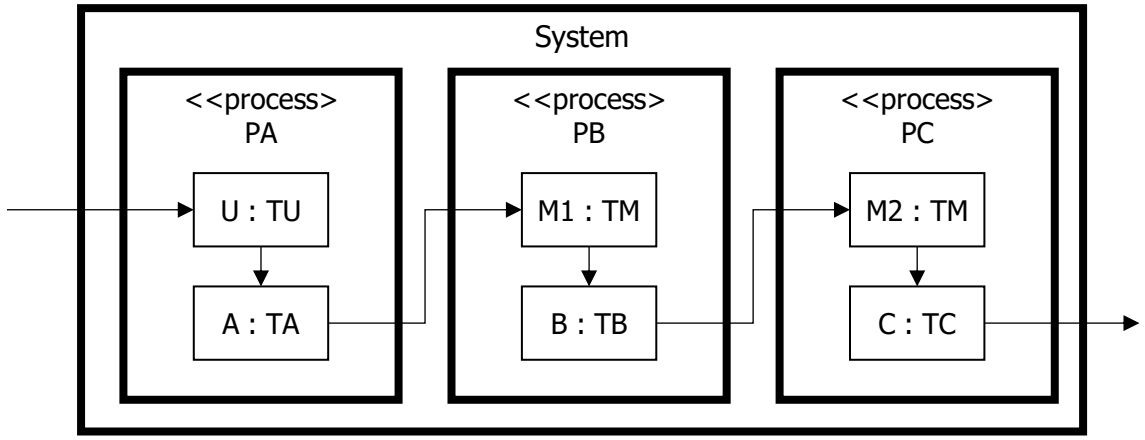
STEP 3. **Refine Class (Module Design)**

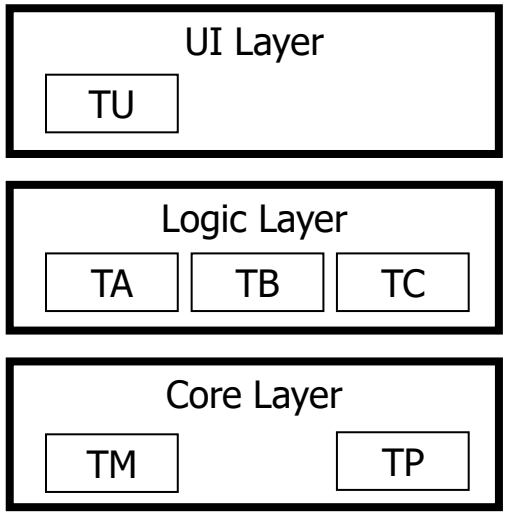




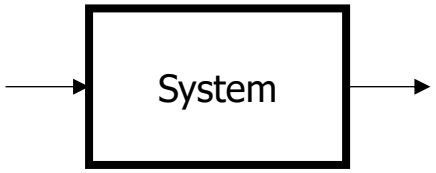
구조 명세

https://github.com/bosornd/example-module-design

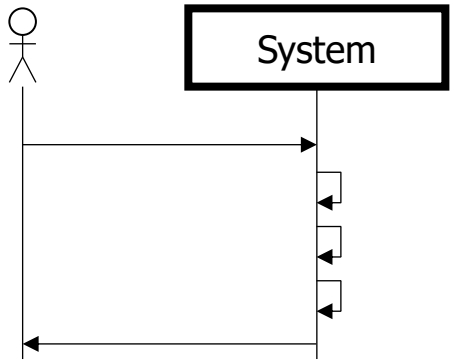




1. 시스템 정의



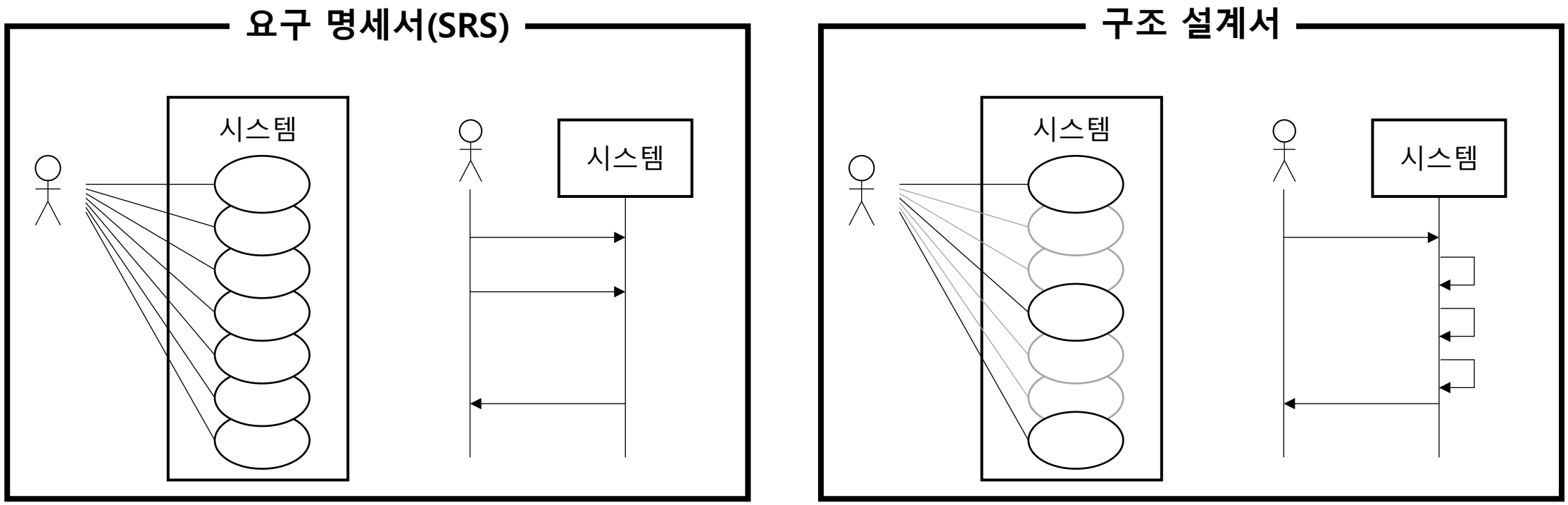
2. 요구사항



구조 명세

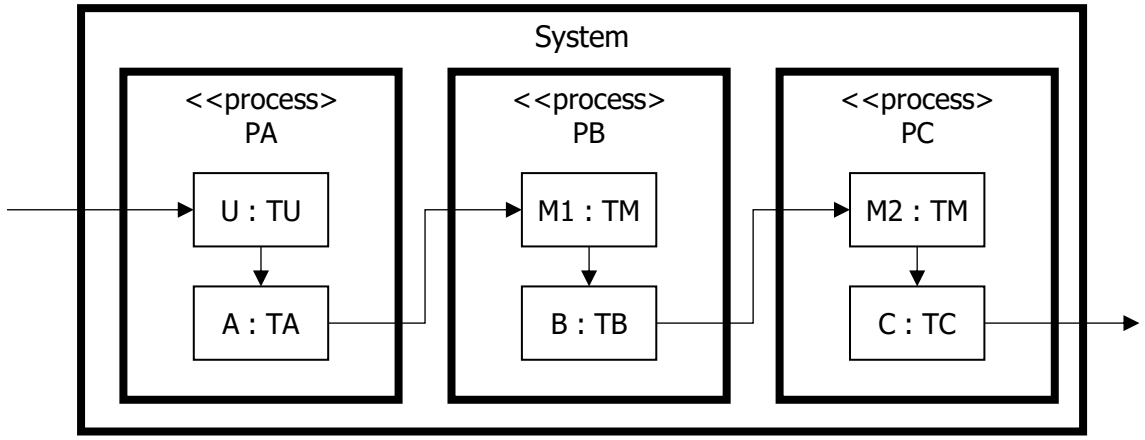
https://github.com/bosornd/example-module-design

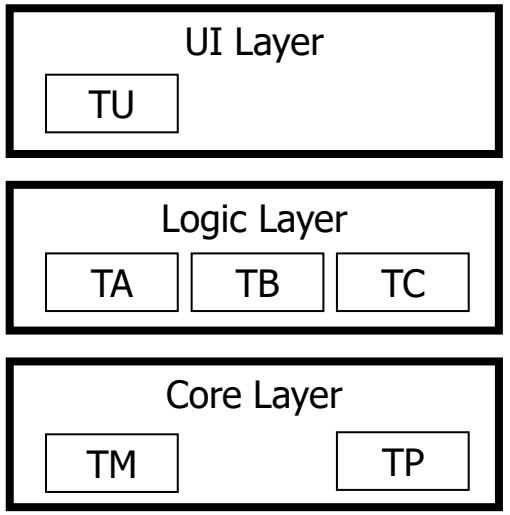
- 구조 설계서의 요구 명세는
  - 시스템이 요구되는 기능을 어떻게 제공할 수 있는 가를 기술합니다.
  - 구조적으로 중요한 기능을 구체적으로 명세하는 것이 좋습니다.



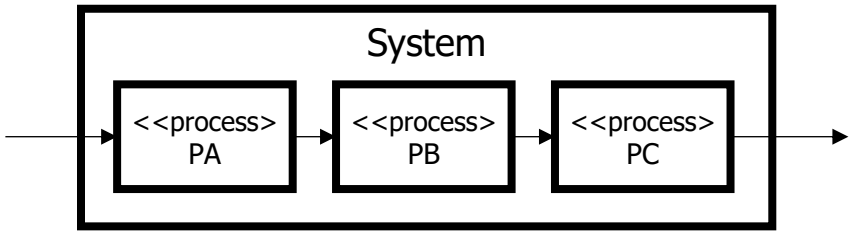
구조 명세

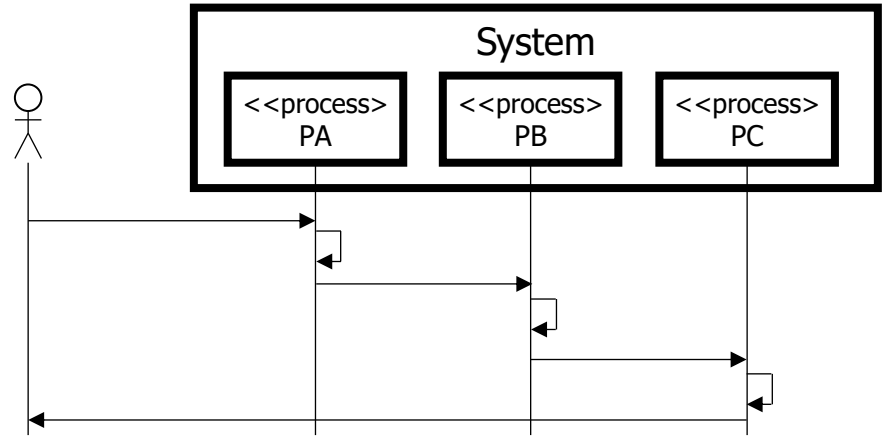
https://github.com/bosornd/example-module-design





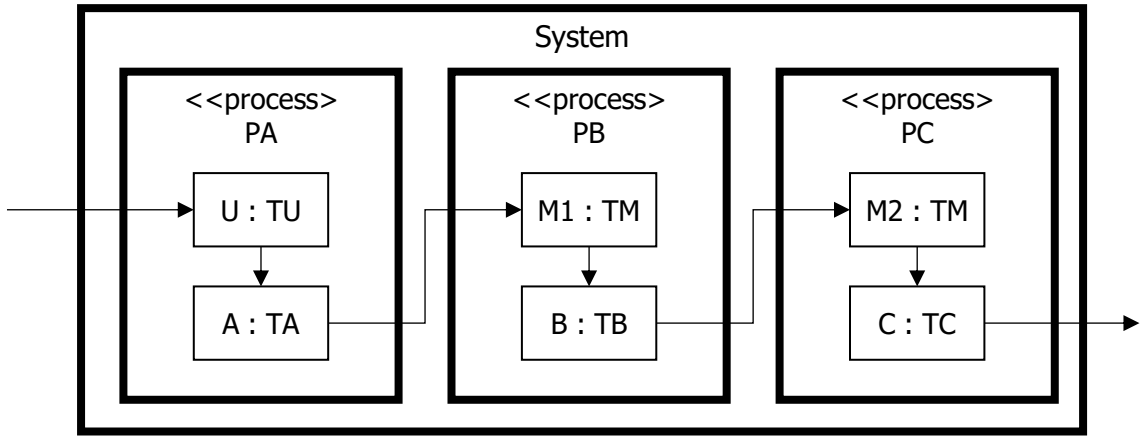
3. 시스템 구조

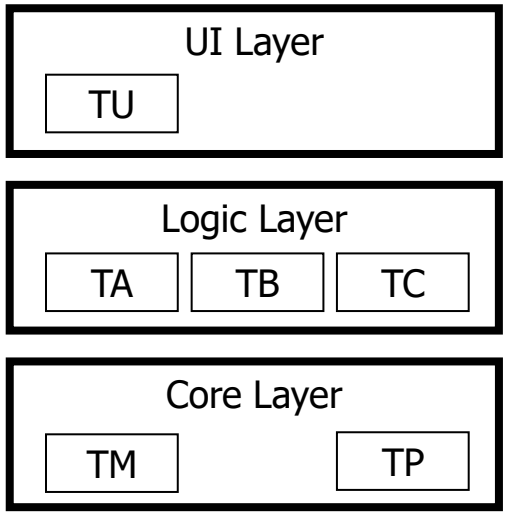




구조 명세

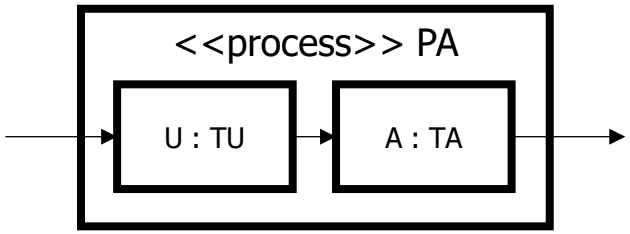
https://github.com/bosornd/example-module-design

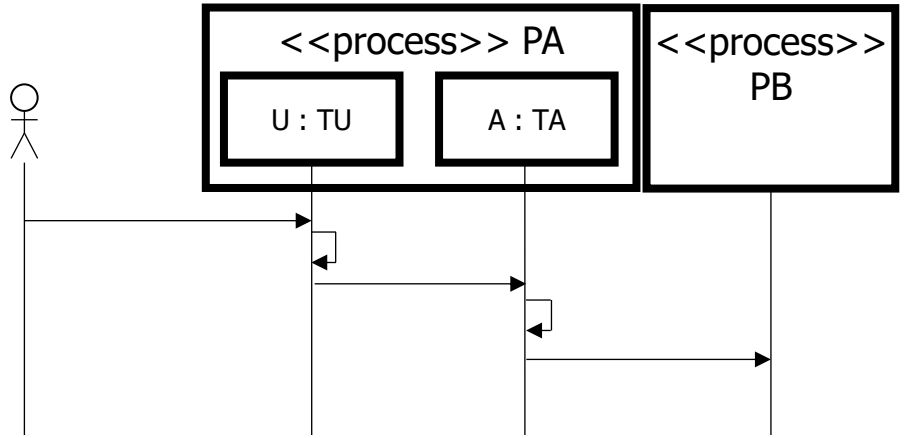




3. 시스템 구조

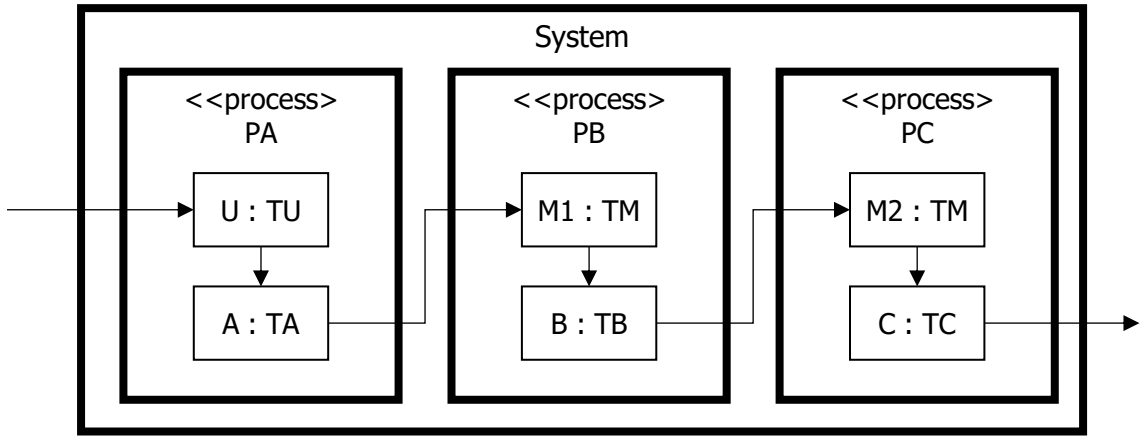
3.1 PA

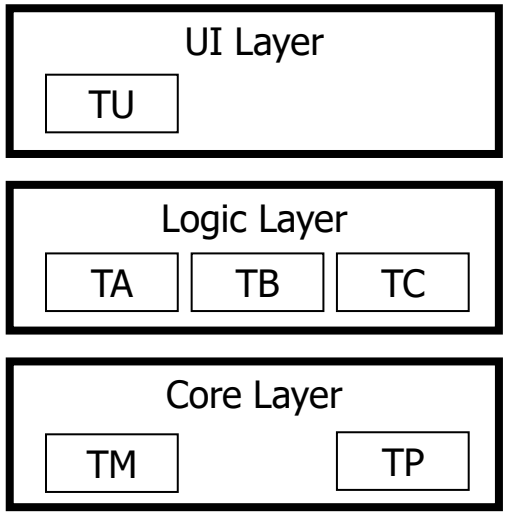




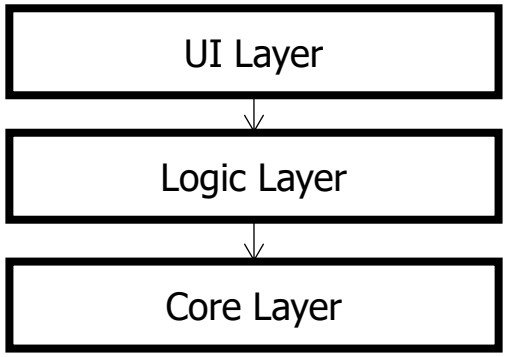
구조 명세

https://github.com/bosornd/example-module-design



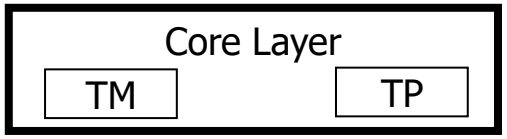


4. 모듈 명세



4. 모듈 명세

4.1 Core Layer



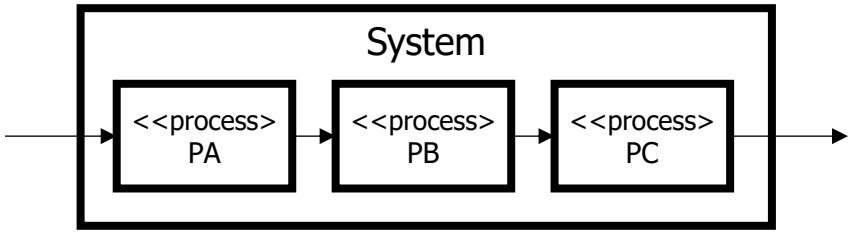
4.1.1 TM

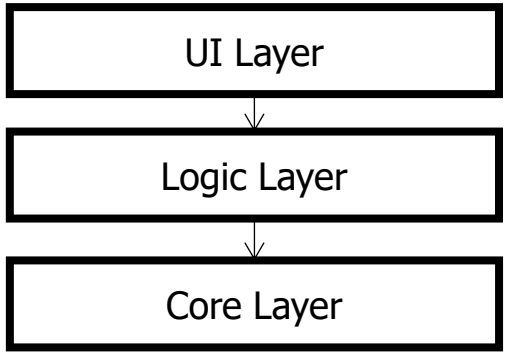


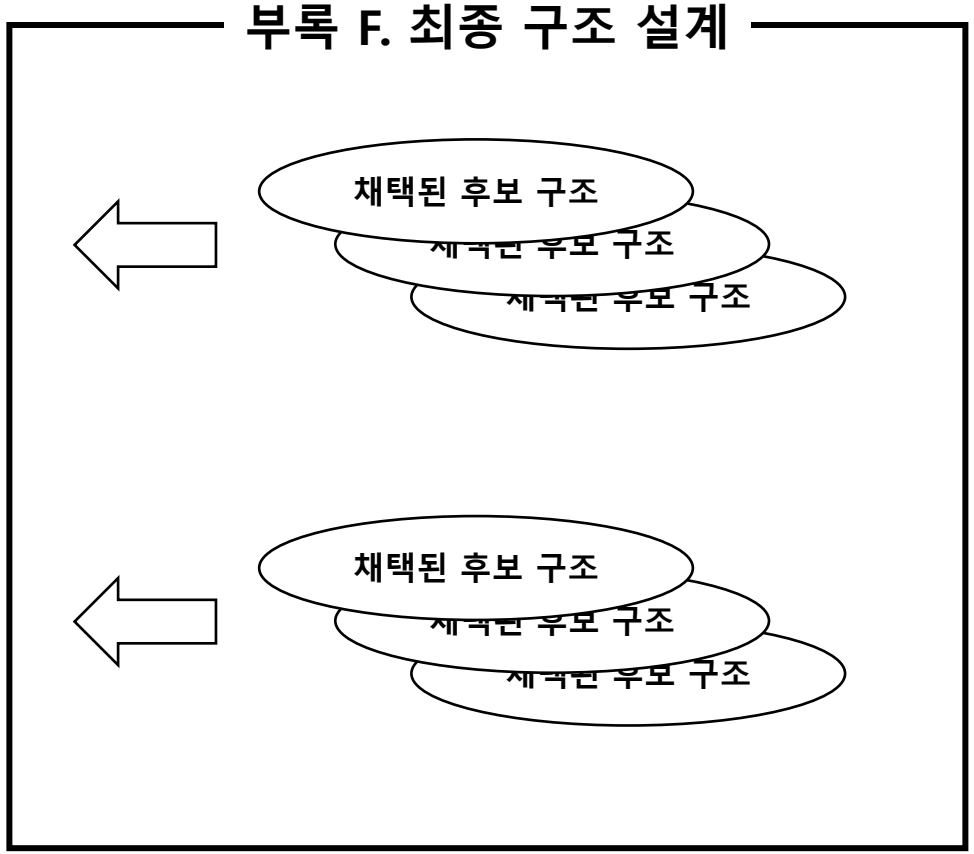
구조 명세 – 부록

https://github.com/bosornd/example-module-design

질문1. 어떤 근거로 이런 구조를 설계하게 되었는가?



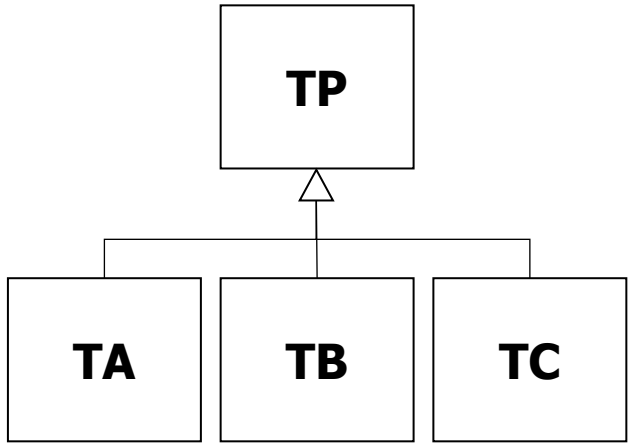




구조 명세 – 부록

https://github.com/bosornd/example-module-design

질문2. 어떤 근거로 이런 구조적 의사결정이 채택되었는가?



부록 E. 후보 구조 평가

후보 구조 목록

후보 구조 채택 근거

	TP TA TB TC	
NFR_01	+	
QA_01	++	-
QA_03	-	+

구조 명세 – 부록

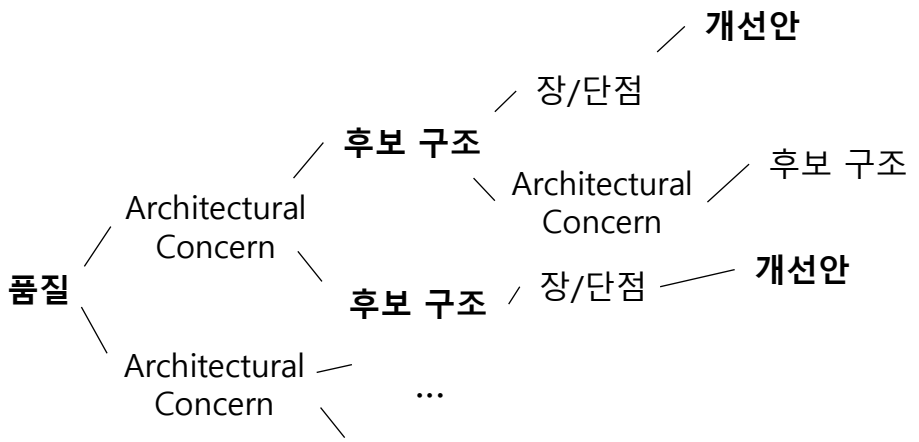
https://github.com/bosornd/example-module-design

질문3. 최적 설계라고 할 수 있는가?



부록 D. 후보 구조

충분한 후보 구조가 검토되었는가?



구조 명세 – 부록

https://github.com/bosornd/example-module-design

질문3. 최적 설계라고 할 수 있는가?

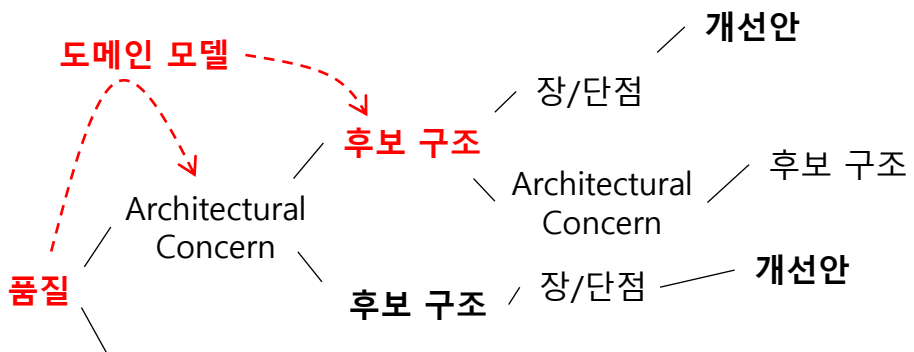


부록 A. 도메인 모델

도메인 모델 : 기능적 / 개념적 구조

부록 D. 후보 구조

후보 구조 : 품질을 고려한 도메인 모델의 변형



구조 명세 – 부록

https://github.com/bosornd/example-module-design

질문4. 품질 요구사항(품질 속성, 비기능적 요구사항)의 선정 근거는?

우선 순위가 높은 품질 속성의 선정 근거는?

우선 순위가 낮은 품질 속성의 선정 근거는?

품질 속성으로 선정되지 않은 근거는?

비기능적 요구사항으로 선정된 근거는?

부록 C. 품질 시나리오 분석

품질 요구사항 선정 결과

비즈니스 드라이버와의 연관성

ID	품질 시나리오	중요도	난이도	품질 요구사항
QS_01	...	H	L	NFR_01
QS_02	...	H	H	QA_01
QS_03	...	L	H	QA_08
QS_04	...	L	L	-

구조 명세 – 부록

https://github.com/bosornd/example-module-design

질문5. 품질 요구사항에 없는 시나리오?

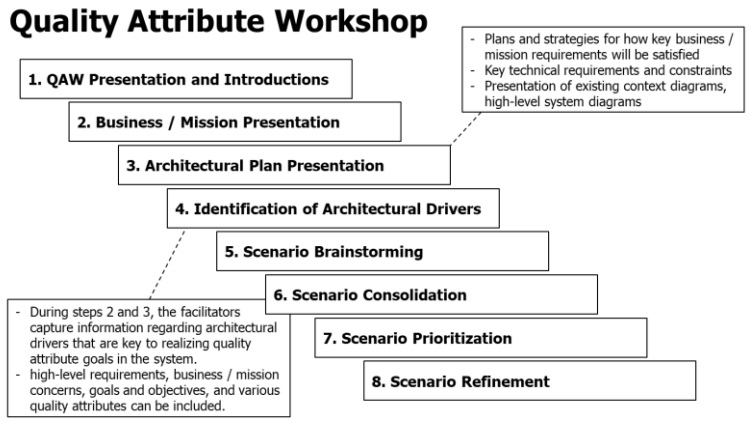
품질 요구사항으로 검토되어야 할 것 같은데
그렇지 않은 시나리오가 있다. 왜?

부록 C에 품질 시나리오 분석에 없다.
품질 시나리오 생성 방법은?

부록 B. 품질 시나리오

품질 시나리오 목록

품질 시나리오 생성 방법



구조 설계 및 평가

조 용 진

(drajin.cho@bosornd.com)

주식회사 보이는 소프트웨어 연구소
http://www.bosornd.com

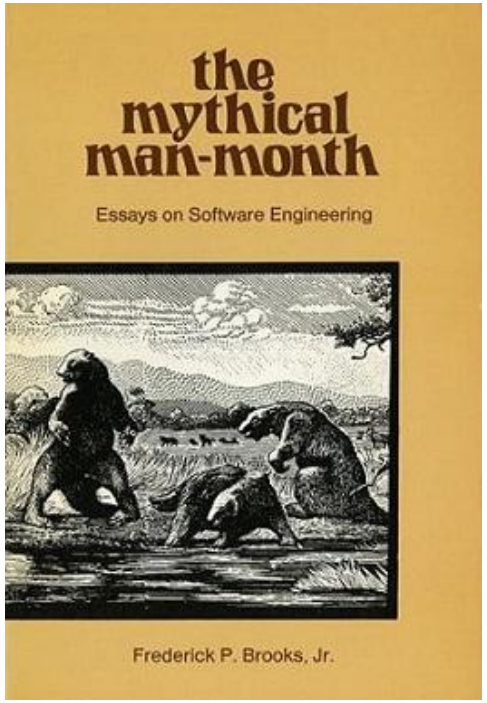
CC BY NC ND

No Silver Bullet !!!

IBM System/OS 360

Frederick P. Brooks, Jr.

There is no single development, in either technology or management technique,
which by itself promises even one order of magnitude improvement
within a decade in productivity, in reliability, in simplicity.



(1975). The Mythical Man-Month. Addison-Wesley. ISBN 0-201-00650-2.
"Adding manpower to a late software project makes it later"

(1987). No Silver Bullet: Essence and Accidents of Software Engineering
Computer, vol. 20, no. 4, pp. 10–19, Apr. 1987

(2021). D. A. Grier, "There Is Still No Silver Bullet," in IEEE Computer,
vol. 54, no. 2, pp. 60-62, Feb. 2021

Yet, almost 35 years after he wrote this contribution to knowledge,
Brooks' observation remains true.
Software engineering is a complex and demanding field that poses a host of problems
to the practitioner, and there is no single solution, that is, no silver bullet,
that will provide a simple way to reduce the work required to create a software product.

No Silver Bullet !!!

There is no single development, in either technology or management technique,

which by itself promises even one order of magnitude improvement

within a decade in productivity, in reliability, in simplicity.

Essential Complexity of SW

A scaling-up of a software entity is not merely a repetition of the same elements in larger size; it is necessarily an increase in the number of different elements. In most cases, the elements interact with each other in some nonlinear fashion, and **the complexity of the whole increases much more than linearly**.



Invisibility of SW

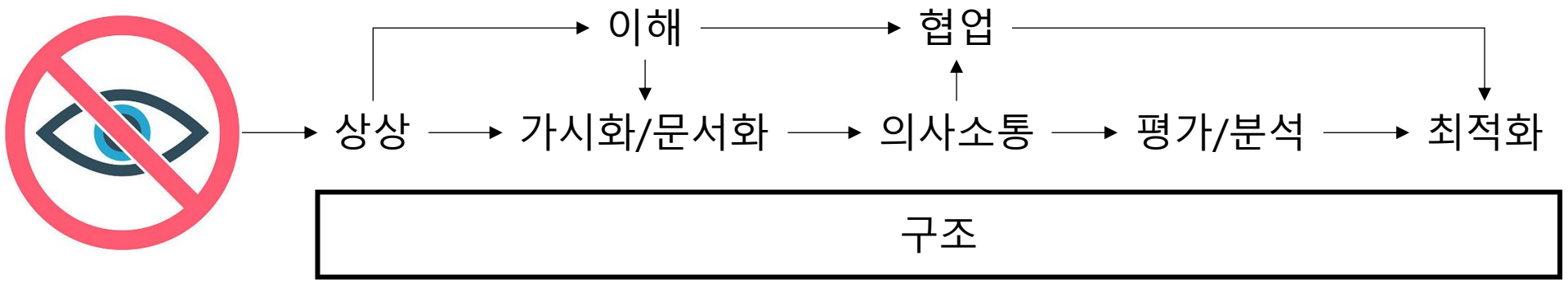
Software is invisible and unvisualizable.
As soon as we attempt to diagram software structure, we find it to constitute not one, but several, general directed graphs, superimposed one upon another.



It's the duty of the architect to solve the problems inherent in essential complexity.
- 97 Things Every Software Architect Should Know

Role of Architect

- 구조도를 그린다.



- 최적 설계/개발을 이끈다.
  - 품질 요구사항
  - 구조적 의사결정

Software Architecture

The software architecture of a system is

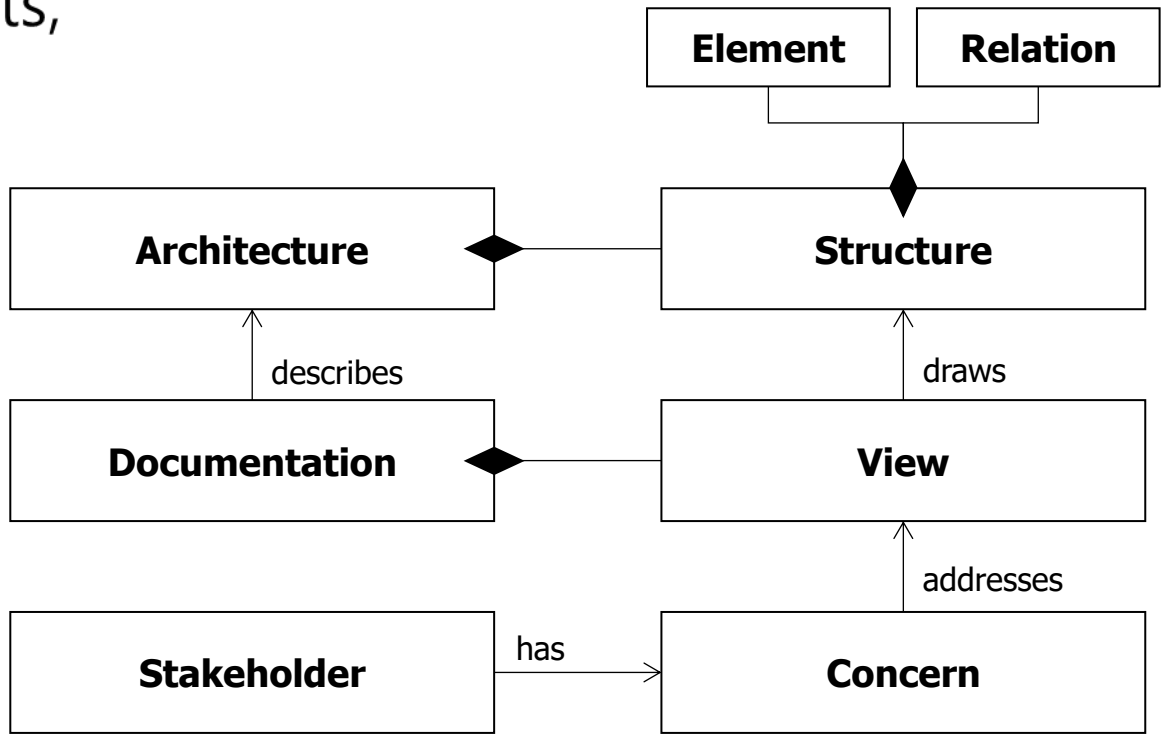
the set of structures

needed to reason about the system,

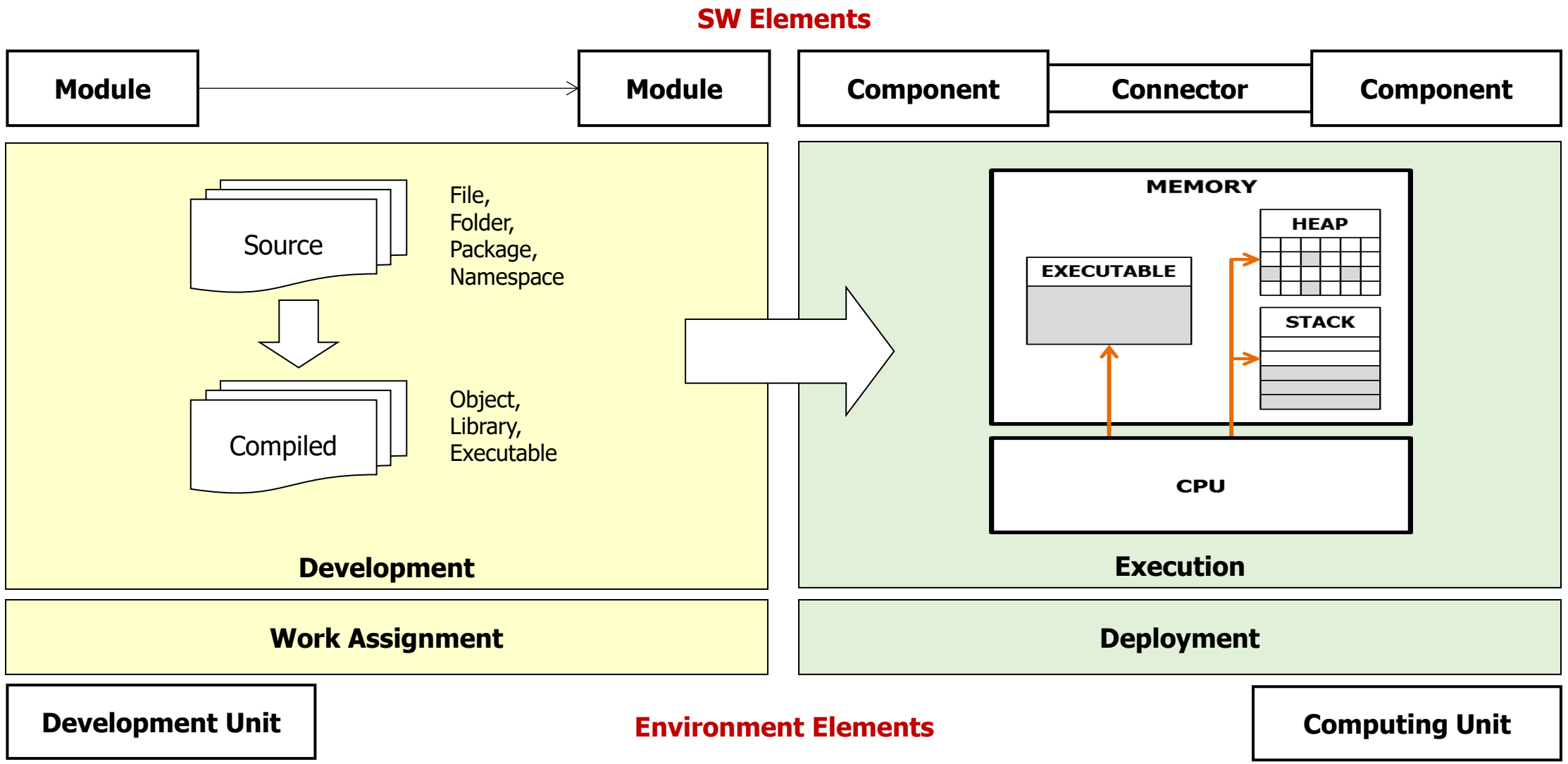
which comprise software elements,

relations among them,

and properties of both.



Software Architecture Views

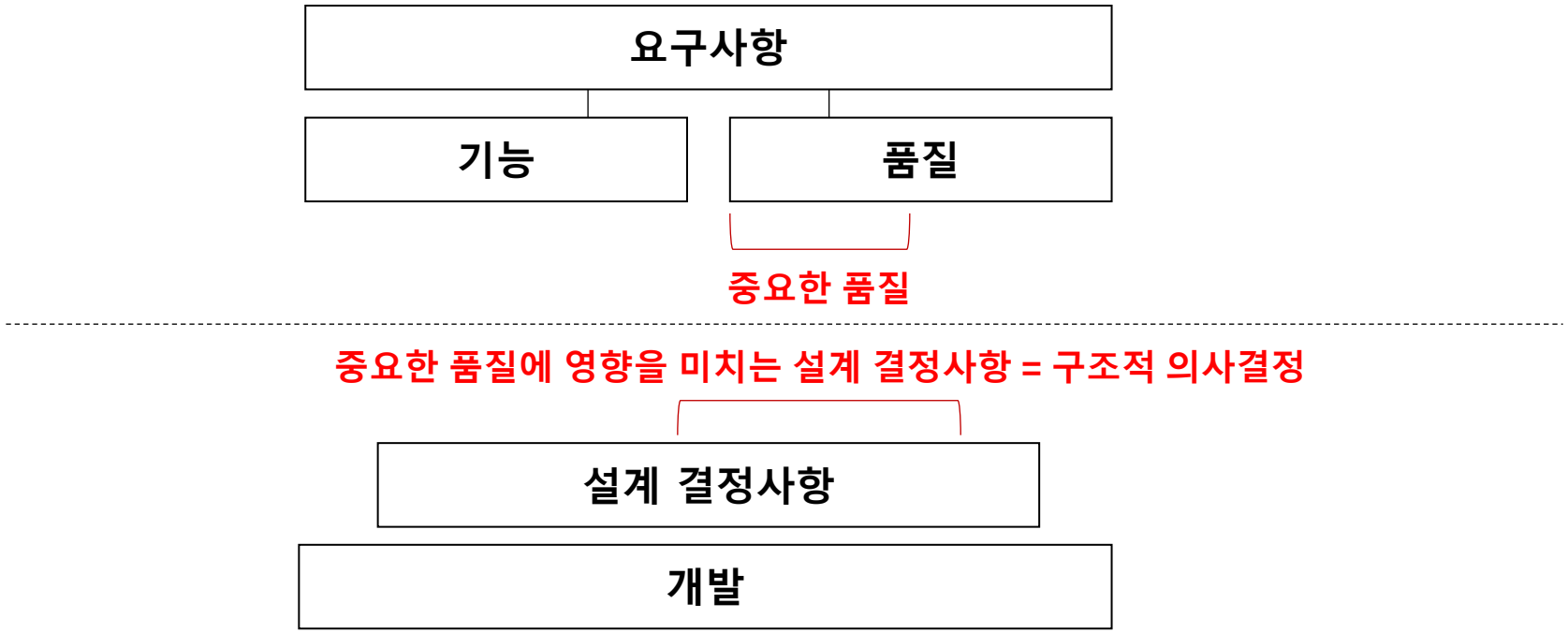


Architectural Decision

구조적 의사결정은 중요한 품질에 영향을 미치는 설계 결정사항이다.

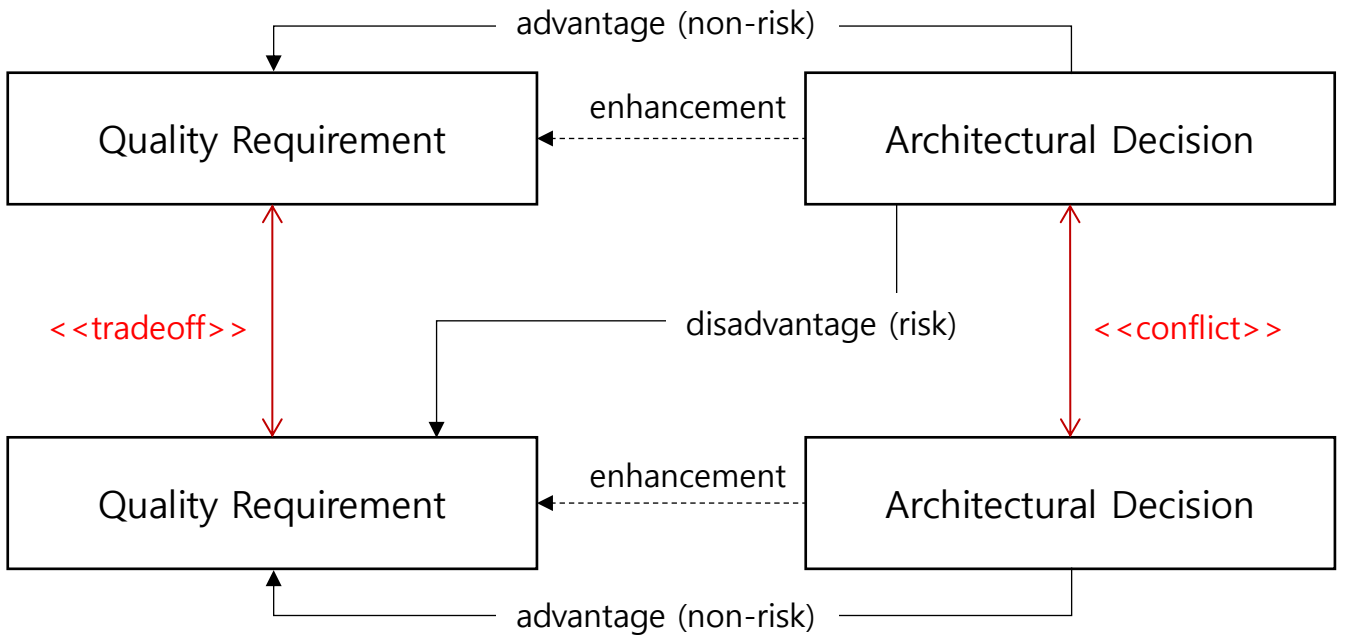
품질 요구사항과 구조적 의사결정의 관계를 관리하는 것이 필요하다.

- 품질 요구사항이 변한다. 최적화된 설계 결정사항의 채택이 필요하다.
- Tradeoff로 RISK가 있는 품질 요구사항에 대한 완화가 필요하다.



ADR (Architecture Decision Records)

It's better to manage relationships between quality requirements and architectural decisions (candidate architectures).



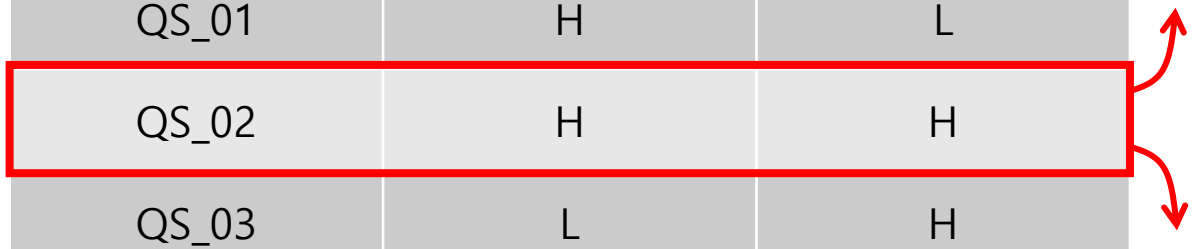
Architecture Design Guidelines

AG1. for at least 10% time

AG2. be together

AG3. focus on what are significant

Quality Scenario	important (necessary)	difficult (complex)
QS_01	H	L
QS_02	H	H
QS_03	L	H
QS_04	L	L



ARCHITECT는 SW와 [ ]을 이어준다.



개발자

관리자

고객

⋮